

Modeling of Input Devices for Natural Interaction from Low-Level Capacitive Proximity Sensor Data

Master Thesis by Andreas Braun, Department of Computational Engineering
October 1, 2009

Supervisors: Prof. Dr. techn. Dieter W. Fellner, Dipl.-Ing. Pascal Hamisu
Department of Computer Science – Interactive Graphics Systems Group



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Fraunhofer
IGD

Abstract

Within the last few years the market for input devices has seen a considerable shift towards novel technologies, using advanced sensor units to register and interpret human behavior, examples being the gaming console and mobile device market. Capacitive proximity sensors are devices that allow detecting the presence of a human body without physical contact, therefore being especially suited for unobtrusive applications. This thesis presents methods and algorithms to model input devices, using data generated by a network of wireless capacitive proximity sensors. Furthermore several input devices have been built and evaluated for several interaction techniques with the help of specifically implemented graphical applications. These devices focus on the ability for natural interaction, providing several usage scenarios within ambient assisted living context.

Der Markt für Eingabegeräte zeigte innerhalb der vergangenen Jahre eine spürbare Tendenz hin zu neuen Technologien, welche fortschrittliche Sensoren nutzen um menschliches Verhalten zu registrieren und zu interpretieren. Beispiele hierfür sind vor allem Spielkonsolen und mobile Geräte. Kapazitive Abstandssensoren messen die Präsenz eines menschlichen Körpers ohne physischen Kontakt und sind daher insbesondere für unauffällige Anwendungen geeignet.

Diese Arbeit präsentiert Methoden und Algorithmen zur Modellierung von Eingabegeräten, welche von Netzwerken kapazitiver Abstandssensoren generierte Daten nutzen. Desweiteren wurden mehrere Eingabegeräte konstruiert und mit Hilfe spezifischer graphischer Softwareanwendungen getestet. Diese Geräte legen Wert auf natürliche Interaktion, gezeigt anhand verschiedener Anwendungsszenarien für Ambient Assisted Living.

1. Table of Contents

1.	Table of Contents	i
2.	Introduction	1
3.	Motivation	2
4.	Background and Related Work	4
4.1.	Capacitive Proximity Sensing	4
4.1.1.	Background	4
4.1.2.	Problems	6
4.1.3.	History and Related Work	7
4.2.	Input Devices	9
4.2.1.	Background	9
4.2.2.	Taxonomy	10
4.3.	Natural Interaction	13
4.3.1.	Idea	13
4.3.2.	Applications	14
4.4.	Interaction in Ambient Assisted Living Environments	16
4.4.1.	Ambient Assisted Living	16
4.4.2.	Available Technologies	17
5.	Modeling Input Devices	19
5.1.	Example for the Taxonomy of Card, Mackinlay and Robertson	19
5.2.	Data Processing	20
5.2.1.	Modeling Interaction with Capacitive Proximity Sensors	20
5.2.2.	Data Fusion	22
5.2.3.	Data Processing Logic	22
5.3.	Common Model	24
5.4.	Proximity Table	25
5.4.1.	Idea	25
5.4.2.	Modeling	25
5.5.	Bed	31
5.5.1.	Idea	31
5.5.2.	Modeling	31
5.5.3.	Problems	33
5.6.	Active Floor	34
5.6.1.	Idea	34
5.6.2.	Modeling	34
5.6.3.	Problems	36

6.	Acquiring Sensor Data	37
6.1.	Hardware Description	37
6.1.1.	Generic Hardware	37
6.1.2.	Antenna Design	39
6.2.	Acquisition Pipeline	40
6.2.1.	Data Pre-Processing	40
6.2.2.	Data Transmission	42
7.	Physical Design of Sample Input Devices	44
7.1.	Proximity Table	44
7.1.1.	Concept & Design	44
7.1.2.	Dimensions	44
7.1.3.	Images	45
7.2.	Bed	46
7.2.1.	Concept & Design	46
7.2.2.	Images	46
7.3.	Active Floor	47
7.3.1.	Concept & Design	47
7.3.2.	Images	48
8.	Sample Applications	49
8.1.	Developed Software	49
8.1.1.	Requirements	49
8.2.	Structure	49
8.2.1.	GUI	49
8.2.2.	Class Structure	50
8.2.3.	Functional Structure	51
8.3.	Generic Interface	51
8.4.	Simulator	52
8.5.	Proximity Table	53
8.5.1.	Gesture Image Viewer	55
8.5.2.	Screen Lens	57
8.5.3.	3D Object Manipulation	58
8.6.	Bed	61
8.6.1.	Position and Posture Detection	62
8.7.	Active Floor	63
8.7.1.	Fall Detection	64
9.	Evaluation Proximity Table	65
10.	Amending Current Assisted Living Environments	67
11.	Conclusion and Outlook	69
11.1.	Outlook	69
11.2.	Future Works	69

A.	Interfacing with CY3271	i
A.1.	Important Classes	i
A.2.	Example Code	iii
B.	Hardware Modifications	iv
B.1.	Multifunction Expansion Board Firmware Settings	iv
B.2.	Modified FTRF Firmware	iv
C.	Bed Full Posture Listing	ix
C.1.	Sensor Activity Mapping	ix
C.1.1.	One Person	ix
C.1.2.	Two Persons	xii
C.2.	Spine Strain Mapping	xiii
C.2.1.	One Person	xiii
C.2.2.	Two Persons	xiii
D.	Software Details	xiv
D.1.	Typical Setting File	xiv
D.2.	Screen Lens Shader Code	xv
E.	Evaluation Proximity Table	xvi
E.1.	Evaluation Sheet	xvi
E.2.	Evaluation Results	xvii
	List of Tables	xviii
	List of Images	xix
	Bibliography	xxiii

2. Introduction

Considering input devices, several areas have seen a considerable change in recent years. Within the last two years the majority of smartphone manufacturers have adjusted their operating systems and handsets for input using finger gestures on touch screens, replacing mechanical buttons and pen. The console gaming market has been conquered by Nintendo's Wii and its motion sensing controller, despite a lack of processing power compared to its competitors. These two examples show that there is a huge interest in novel, more natural interaction technologies. Accordingly there is a plethora of research projects, developing human-computer interfaces based on speech-recognition [33], visual motion tracking [56], depth cameras [28] and other sensory that try to introduce interaction schemes more natural than those currently in use. This natural interaction tries to reduce the required learning curve necessary for using an input device, by recognizing elemental human behavior, like speech, gesture or mimic. Detection can occur either deliberate, i.e. the user is directly interacting with an input device, or unaware, one example being smart rooms [40], which pick up position, body temperature and other conditions of its inhabitants, adjusting room temperature and lighting accordingly. The aim is to bring the advantages of computer usage to an audience, which has been excluded previously due to complicated interaction and software, most notably elderly citizens in industrial countries.

One sensor type that is rarely used for input devices is the capacitive proximity sensor. This detector is using the electrostatic field induced in a human body to disturb an internal oscillatory circuit, which allows sensing presence without contact to any device. Therefore it is possible to use data generated by these proximity sensors to either detect deliberate gestures by a user, or passively detect presence in general. This thesis presents methods and algorithms to model input devices, using data generated by a network of wireless capacitive proximity sensors. Furthermore several input devices have been built and evaluated for several interaction techniques with the help of specifically implemented graphical applications. These devices focus on the ability for natural interaction, providing several usage scenarios within ambient assisted living (AAL) context.

The thesis begins with motivating the necessity for novel input devices in AAL environments in chapter 3. Afterwards a short introduction is given on related research area in chapter 4, ranging from basics of capacitive sensing, introduction to input device taxonomy, research in natural interaction and state-of-the art regarding interaction in AAL environments. The methodology required to model input devices from capacitive sensor devices is described in chapter 5, including data abstraction and semantic mapping. Chapter 6 outlines the pipeline generating discrete integer values from continuous sensor data, including hardware description, data pre-processing on the sensor's microcontroller, design of the detection antennas and transmission of the sensor data throughout a network of units. The physical input devices that were built are introduced in chapter 7. Possible usage scenarios, along with software evaluating these are described in chapter 8. Details of a user study held to evaluate the Proximity Table and its applications are given in chapter 9. The thesis closes mentioning several options to amend current AAL environments using capacitive proximity sensing in chapter 10 and the conclusion in chapter 11. The appendices feature an introduction to interfacing with the supplied hardware, modifications applied to firmware, an overview of the semantic mapping used in certain devices and additional details about the developed software.

3. Motivation

If a random person is asked, “Could you please draw a computer in 20 seconds?” the result will most likely resemble the drawing in Figure 3-1.



Figure 3-1 Speed Drawing of a Computer

The vast majority depicts a computer as screen, keyboard and mouse. Observing this result, several consecutive questions seem logical:

- How come, most drawings show personal computers?
- Why do most drawings lack the tower/desktop?

Personal computers, in the form of desktop systems and laptops are ubiquitous in industrial countries. Starting from the mid-1970s the PC has been an unequalled success, conquering industry, commerce and personal life within a single generation. Although computers are present in almost any modern electronic device, the public image of it is strongly inclined towards the classical PC. If the same question would have been posed around 1970, the drawing might have shown a huge box with a magnetic tape drive. The deciding factor, determining what is drawn, is the limited time. Participants will only draw what is most important and defining, regarding the desired object. Interestingly, most drawers will focus on devices used for human-computer interaction (HCI), while the computing hardware and its casing are ignored. Simplifying objects to their inputs and outputs is common, even for mechanical devices. A quickly drawn clock will show its face and hands, a phone its handset and dial. End-users are rarely interested in the mechanics/electrics of devices they use, instead the focus is on functionality and interactions required to achieve the desired result. In consequence it is a pressing question how future interaction devices might look like and how they will affect our handling of machines.

Western societies do face a challenging demographic change in the near future. Improvements in nutrition and medical supplies have increased the life expectancy for males and females in Europe from 45.7 and 49.6 years respectively in 1900 to 75 and 79.9 years in the year 2000. It is expected that by the year 2020 21% of the total population is over 65 years in age compared to 11% in 1960 [53]. It is commonly agreed on that an ageing population together with an increasing survival rate after accidents and operations does lead to an increased proportion of the population having disabilities and impairments that affect the participation in daily life. In order to maintain the viability of welfare systems it is necessary to minimize the economical and social effects of these demographic changes. Ambient assisted living is providing concepts, technologies and services to help people requiring assistance in maintaining a self-determined, socially included lifestyle within their familiar surroundings as long as possible. This approach is user-centric, i.e. technology has to adapt to user behavior and should integrate itself into the living environment unobtrusively, including processing

units and sensor devices. Capacitive proximity sensors are specifically suited for hidden appliances, requiring no physical contact or line-of-sight with a user to register values. In the scope of this project they are used for two specific applications. The first is integration into furniture as passive monitoring devices, detecting presence or pressure. The second is a device for deliberate interaction with the AAL environment. Designing interaction devices for the disabled and elderly is facing a specific set of challenges, e.g. blind or deaf people are restricted to a certain group of output devices. Persons suffering from carpal tunnel syndrome or arthritis might not be able to give their input using mouse, keyboard or touch screen.

This thesis is presenting several independent input devices based on capacitive proximity sensors, focusing on showing the broad spectrum of applications for those devices and the possible integration into existing environments for AAL.

4. Background and Related Work

4.1. Capacitive Proximity Sensing

4.1.1. Background

Any living organism produces a small electrical current that is caused by cell activity and ionic currents in the nervous system [11]. Measuring this electric field requires very sensitive sensors and is limited to close proximities. It is easier to use external electric fields of a given value or charge the body with a small current, in order to measure the disturbance in the field caused by the presence of a person. Furthermore the human body is mostly composed of ionized water. Differences in the proportion of water in specific types of body tissue are causing distinct electrical properties that can be distinguished. The response of a living organism to an external electric field is called bioimpedance [23].

Capacitance Model

A complete overview about the electrostatic principles of capacitive proximity sensing can be found in [5], chapters 2 and 6. In order to keep this thesis self-contained this short introduction is focusing on the sensor devices used [48,13].

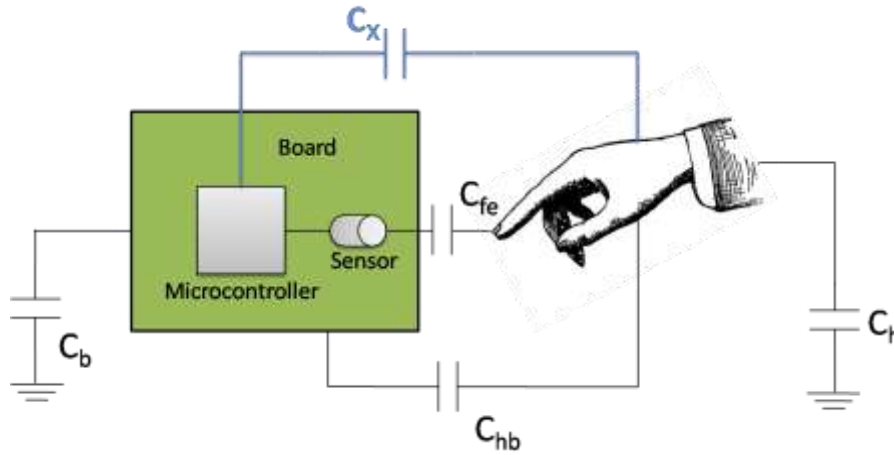


Figure 4-1 Black box setup of a capacitive proximity sensor

The basic setup of the sensor used is shown in Figure 4-1. The proximity capacitance C_x can be determined using serial and parallel circuit combinations of capacitors, resulting in the following equation:

$$C_x = \left(\left(C_{hb} + \frac{C_h \cdot C_b}{C_h + C_b} \right)^{-1} + \frac{1}{C_{fe}} \right)^{-1}$$

Additionally there are parasitic capacitance components, i.e. disturbing capacitance values within the system. Sources are:

- Sensing electrode capacitance
- Capacitance between sensing electrode and ground plane
- Intercapacitance between neighboring traces on the board

The present parasitic capacitances C_{par} amount to values approximately between $10pF$ and $300pF$ and are therefore considerably larger than the value of the proximity capacitance C_x , being between $0.1pF$ and $10pF$. The total capacitance sensed is the sum of parasitic and proximity components.

$$C_S = C_x + C_{par}$$

Capacitance of Detectable Objects

Any object exhibits capacitance in respect to infinity. Surveying simple geometric shapes this capacitance is analytically determinable, e.g.:

$$\begin{aligned}C &= 8\varepsilon_0 r && \text{Disk} \\C &= 4\pi\varepsilon_0 r && \text{Sphere}\end{aligned}$$

ε_0 is the vacuum permittivity and r the respective radius. This free space capacitance is increasing as soon as another object is approaching, caused by the capacitance of this second object, resulting in mutual capacitance. Looking at generic formulas, determining capacitance between parallel plates this behavior can be described analytically.

$$C = \frac{Q}{V} \qquad C = \varepsilon_0 \varepsilon_r \frac{A}{d}$$

The capacitance is directly proportional to the plate area A and inversely proportional to the distance d between the plates, with ε_r being the relative static permittivity of the dielectric between the plates. Sensor electronics are grounded with the body acting as ground itself. The sensor plate is continuously charged using a constant voltage V . A higher capacitance allows the system to hold a larger charge. If the system is connected to the ground, the sensor capacitor is discharged through a resistor. The resulting voltage is depending on the available charge, shown in the equation above. Furthermore the required time to discharge the capacitor is increased. This process is symbolized in Figure 4-2.

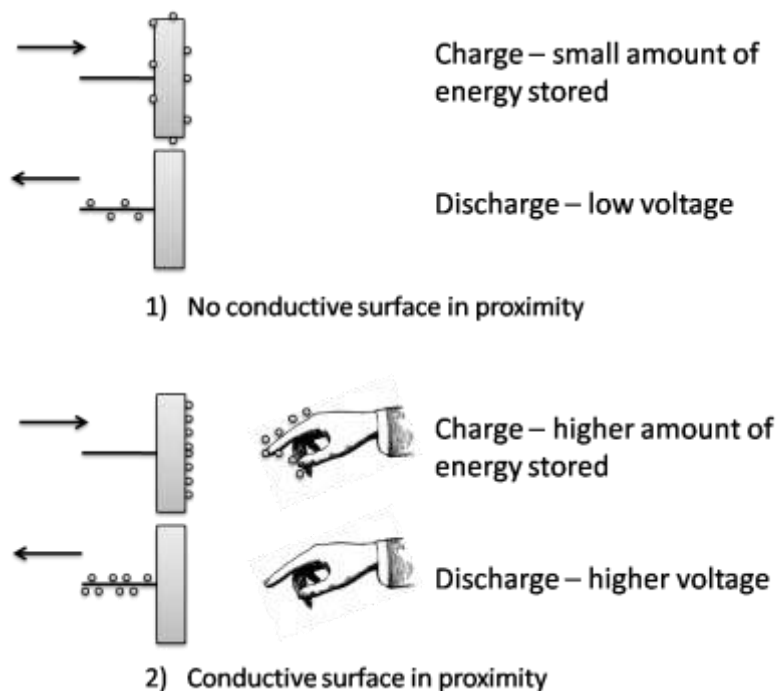


Figure 4-2 Capacitive sensing procedure

Capacitance Measurement Operation

It is not possible to measure capacitance directly. Electronics are required that are able to detect properties that can be derived from capacity, e.g. voltage. A possible circuit based on voltage detection, is the sigma-delta modulator. It is able to transfer the capacitance acquired from a switched capacitor cell and transfer it to a duty cycle. This duty cycle is transformed to an integer value using a digital logic called bit stream integrator. A simplified version of the circuit is shown in Figure 4-3.

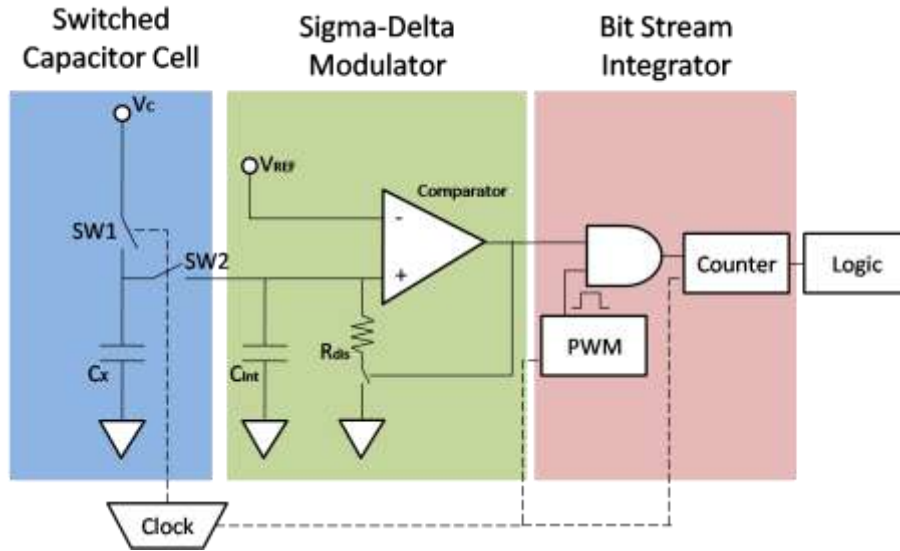


Figure 4-3 Measurement circuit

The switches $SW1$ and $SW2$ are closed inversely at a given frequency. If $SW1$ is closed the sensor capacitor C_X is charged. If it is re-opened $SW2$ is getting closed and the charge is transferred to C_{Int} . The comparator goes high as soon as the voltage through C_{Int} exceeds V_{REF} . This connects R_{dis} , which is discharging C_{Int} until the voltage drops below V_{REF} . The charge on C_X is proportional to the time the comparator is high. To convert this duty cycle to a usable number a bit stream integrator is used. This digital logic is clocked with the same frequency as the switches $SW1$ and $SW2$ and is using pulse width modulation (PWM) to feed a digital counter, which is running as long as the comparator is high. The difference in subsequent counter values is proportional to the sensed capacity and therefore usable by the logic attached to the circuit.

4.1.2. Problems

Detectable Objects

Capacitive sensing using this simple sigma-delta-modulator setup is limited to the detection of conductive grounded objects. Other concepts using parallel plate capacitors are able to detect non-grounded objects brought between the electrodes by calculating their relative permittivity, which is unique to each material. Yet these detectors are restrained in resolution and require a more sophisticated setup. Since this project is focusing on sensing the human body, this limitation is advantageous, because non-grounded objects within the sensor field do not disturb the measurement.

Disturbance in measurement

The actual capacity of an object depends on various environmental parameters. Temperature is affecting the number of free electrons in any material thus changing electrical properties [35]. The strength of this effect is depending on the specific material. Another influencing parameter is humidity, leading to a microscopic water layer on any object, which has a high relative permittivity, leading to

certain sensor units that use capacitance to determine humidity [44]. The strongest influence is the susceptibility of capacitive sensors to other electric fields in range, decreasing sensitivity considerably. Disturbing fields should be avoided as far as possible. Using sensor arrays, it is necessary to shield units from each other, if they are supposed to be placed in close proximity. Concerning this project, it was decided to refrain from attempting to convert sensor data to absolute physical values, which would have required thorough calibration on each usage. Instead the focus is on robustness of the devices in real-world usage, which is achieved by using relative distance values. Instead of trying to calculate an absolute physical distance measured in centimeters, the software is using semantic variables, i.e. the object is considered far away, close, etc. This is achieved either by using threshold levels or a transfer function, explained in more detail in chapter 5.

Signal-to-Noise Ratio

Any electric measurement is suffering from noise. The signal-to-noise ratio (SNR) is the main indicator for practical resolution of any sensor device. Concerning capacitive sensing the major sources are interfering external electric fields, including those generated by radio transmission, electric power lines, etc. Consequently it is suggested to use a working frequency, avoiding commonly used wave bands. Noise is a major problem in the used setup. Therefore filtering algorithms are applied in both pre- and post-processing to achieve a good effective SNR and robustness. A detailed description of noise reduction at a post processing level is found in chapter 5.4.2, which describes the modeling of an input device strongly affected by noise.

4.1.3. History and Related Work



Figure 4-4 A young Leon Theremin playing his instrument¹

Using the electric properties of a human body to control electronic devices surprisingly is a rather old application. In 1920 Russian physicist Léon Theremin developed a musical instrument that he called etherphone, which would later become widely known as the theremin. It consists of two metal antennas connected to the electric system and is played without physical contact. The musician moves his hands in proximity to the antennas, the first controlling pitch and the other volume [20].

The principle is similar to the capacitive proximity sensors used in this project. Both antennas are connected to radiofrequency oscillators. The musician's hands act as grounded plates, with the body being the connection to the ground. Therefore he can be considered as a variable capacitor in an inductance-capacitance (LC) circuit. Pitch is generated using the heterodyne principle. The circuitry contains two RF oscillators, one working at a fixed frequency. The pitch antenna is connected to the

¹ Source: http://upload.wikimedia.org/wikipedia/commons/7/71/Leon_Theremin.jpg

other, the frequency of which is depending on the hand position. Current frequency difference between those two is used to generate a difference tone in the audible range. The volume antenna is connected to another oscillator controlling the amplification [3].

Electrical impedance tomography (EIT) is a medical imaging technology that was first proposed in 1978 and developed successfully in 1984 [4]. Electrodes are attached to the body and charged with alternating currents. Electrical voltages are measured in order to reconstruct a three-dimensional image of the examined area. Different body tissues vary in conductivity and permittivity and therefore influence the applied electric fields differently. Using tomographic image reconstruction methods a three-dimensional dataset of the examined body part can be generated. One suggested application is lung monitoring. Air has a high conductivity contrast to the lung tissue and can be easily differentiated. It has also been proposed to replace mammography in breast cancer detection using the distinct electrical properties of cancer tissue. However even though several studies [2,30] have shown the use of this less invasive technology, it is not widespread.

Throughout the 1990s electric field sensing has been a research interest at the Massachusetts Institute of Technology (MIT) Media Laboratory. Attempting to mimic the perception of several weakly electric fish a variety of sensor units has been created. Single devices are named LazyFish, while an array of those is called School of Fish [50].

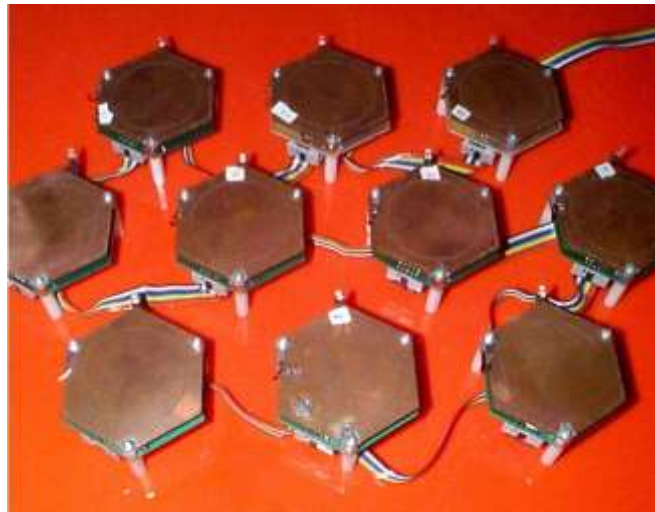


Figure 4-5 School of Fish sensor array²

The developed methodology includes two operational modes. Human shunt mode is using a single electrode to create a dipole field, which is disturbed by the presence of a body part, thus working similar to the theremin. Human transmitter mode is using the whole body as RF transmitter by capacitive coupling it to the transmitter electrode, increasing the generated electric field registered by the receiver. The person has to be close to the transmitter electrode for this mode to work [63]. Cooperation with NEC Automotive Electronics resulted in the development of a passenger sensing system that is able to detect a rear-facing child seat on the passenger side. In case of an accident this unit prevents deployment of the airbag to protect the child from serious injuries. Numerous other applications have been suggested, including the array in Figure 4-5, which is able to detect position and orientation of a hand in close proximity and a gesture wall, modifying its content based on recognized user movement [51].

A recent research project is the Thracker system that is using capacitive sensing to detect hand gestures in front of a screen. The hardware was developed at the University of Munich in 2006 [61].

² Source: [50] – page 51



Figure 4-6 Thracker attached to a LCD screen³

The system is able to detect hand position relative to the screen in three dimensions and simple gestures that are performed at the center of the detection area. It can be cheaply produced and suggested applications include interactive underground plans and advertisement walls. In subsequent work the project has been extended, defining active regions on a table, amending a keyboard with sensors to simplify scrolling and measuring fluid levels in a bottle [60].

4.2. Input Devices

4.2.1. Background

Input devices are the interface between human and machine. One of the earliest devices, controlling mechanical machines and the first digital computers, was the punched card [29]. Input devices in their modern sense have been suggested as early as 1943 in the von Neumann architecture for stored-program computers [59].



Figure 4-7 (1) Keys on the Zuse Z3⁴ (2) The first computer mouse⁵ (3) Ivan Sutherland in front of his Sketchpad⁶

³ Source: [61] – page 1

⁴ Source: http://www.computerhistory.org/timeline/images/1941_zuse_z3_large.jpg

⁵ Source: http://www.cc.gatech.edu/classes/cs6751_97_fall/projects/follow_me/hw4/first_mouse_large.jpg

⁶ Source: <https://webpace.utexas.edu/alp425/CS%20329/Final/images/sketchpad.jpg>

The keyboard as prime instrument for text entry is an extended electronic version of typewriter keys. First usage of keys on digital computers dates back to 1941, when the Zuse Z3 was controlled by a multitude of keys. Many of the common input devices of our time are derived from research projects at American universities in the 1960s. The computer mouse was developed in 1963 by Douglas Engelbart at Stanford Research Institute, though it wasn't presented to the public until 1968 [18]. The predecessor of touch screen and touchpad devices, the Sketchpad, was developed by Ivan Sutherland at the MIT in 1963.

Aforementioned technologies are the most common input devices of modern computers. However there exists a huge diversity of other, less prevalent input peripherals, including but not limited to, web cams, space mice, microphone arrays, fingerprint readers or image scanners.

4.2.2. Taxonomy

The multitude of input devices requires some sort of systematic system to categorize each unit. Over the years several systems have been suggested. Early systems of the late 1970s define input devices in terms of the values they return to the application, e.g. locator returning a real-world coordinate, string returning a sequence of characters or valuator returning a single number. Foley, Wallace and Chan took an approach based on the user's intention.



Figure 4-8 Transaction tree categorization⁷

Six generic transactions are identified. Three manipulations, namely selection, location and orientation, together with three intended actions, called ink, i.e. drawing a line, text, intending to enter text, and value, where a scalar value is specified. Input devices are then assigned to the different transactions, resulting in a tree-structure, as shown in Figure 4-8.

Buxton is using a tabular system that is restricted to hand-operated devices that submit continuous data. The primary rows depict the sensed property, e.g. position and motion, while the primary columns show the respective degrees-of-freedom (DOF). Resulting cells are divided into sub columns,

⁷ Source: [7] – page 3 (redrawing)

based on the motor skills used, and sub rows, which differentiate between direct touch of the sensing mechanism and usage of a mechanical intermediary in between (Figure 4-9). A thorough overview of mentioned taxonomies and an additional state-transition categorization can be found in [7].

		Number of Dimensions							
		1		2				3	
Property Sensed	Position	Rotary Pot	Sliding Pot	Tablet & Puck	Tablet & Stylus	Light Pen	Floating Joystick	3D Joystick	M
				Touch Tablet		Touch Screen			T
	Motion	Continuous Rotary Pot	Treadmill	Mouse			Trackball	3D Trackball	M
			Ferinstat				X/Y Pad		T
	Pressure	Torque Sensor					Isometric Joystick		T

Figure 4-9 Buxton table for continuous, hand-controlled input devices⁸

Card, Mackinlay and Robertson (CMR-Model in future references) depict the input device as six-tuple $\langle M, In, S, R, Out, W \rangle$ [8]. M is a manipulation operator, representing the detected property. Any possible operator is shown in Table 4-1.

	Linear	Rotary
Position		
Absolute	Position P	Rotation R
Relative	Movement dP	DeltaRotation dR
Force		
Absolute	Force F	Torque T
Relative	DeltaForce dF	DeltaTorque dT

Table 4-1 Possible manipulation operators

Input domain is defined by the In attribute. One example for an absolute position rotary input device is a volume knob, allowing any position in the continuous interval of angles between 0 and 270. S represents the current state of the device, which is an element of the current input domain. In above example it could be 80 degrees. Mapping between the input domain set and the output domain set occurs using a resolution function R . Concerning the volume knob, no transfer function within the input device is required. However the application is mapping detected angles to a volume level, requiring a simple linear transfer function. Mentioned output domain set is attributed via Out and describes the transferred values that are consumed by an application. Finally device parameters that might describe additional functionality are collected in the attribute W . The volume knob example is specified in Table 4-2.

⁸ Source: [7] – page 5

Manipulation	R_z
Input	$[0, 270]$
State	s
Resolution function	$R(s) = s$
Output	$[0, 270]$
Works	-

Table 4-2 Six-tuple of a 270-degree volume knob

In order to describe more complex input devices three different composition operators are proposed. Merge composition is the combination of two devices, resulting in a new input domain that is the cross-product of both involved devices. Layout composition is used to join two devices on a common platform. Connect composition is linking the output domain of one unit to the input domain of another.

Comparison Chart

The following chart is comparing several qualitative features of the presented input taxonomies.

- Flexibility: Adjustability towards layout changes in devices. E.g. traditional button mouse vs. scroll wheel mouse.
- Generality: The scope of input devices included. A generic taxonomy should include any possible device
- Extensibility: Adaptability towards new device categories. Preferably easy to integrate.
- Definiteness: Ambiguous device categorization should be avoided.

	Foley, Wallace & Chan	Buxton	Card, Mackinlay & Robertson
Method	User's intentions	Human Sensory System	Manipulation & Composition
Flexibility	No combined devices	Only continuous hand devices	Combined devices via composition Virtual input devices
Generality	No Audio/Vision	No Foot/Audio/Voice No Discrete Input	No Audio/Vision
Extensibility	Only easy if in six categories	Only if continuous hand-controlled device	Easy if input, output domains can be defined
Definiteness	Any device in multiple categories based on application	Given for set amount of input devices	No categories but device specific unique modeling

Table 4-3 Comparison chart of input device taxonomies

Conclusively the taxonomy of Card, Mackinlay and Robertson is used to model the input devices designed in this project. It simplifies modeling of several input devices based on a single platform and is application independent. Additionally it is the only system that allows modeling of virtual input devices, which are used in this project to encapsulate high-level data processing algorithms applied within the input device. The taxonomy of Card, Mackinlay and Robertson is extended in this thesis, by integrating capacitive sensors and their isotropic proximity sensing in spherical coordinates, as well as the virtual data processing described earlier. The missing categorization is not very important for the devices modeled in this project, because the modeling itself is independent of a device category.

Furthermore the devices are not using visual and audio input, which are not supported by their taxonomy. A further introduction to the taxonomy and its modifications, including examples is given in chapter 5.

4.3. Natural Interaction

4.3.1. Idea

Alex Pentland, head of the MIT Human Dynamics Lab, described the idea behind his *Smart Rooms* project in 1996 [47].

“The problem, in my opinion, is that our current computers are both deaf and blind: they experience the world only by way of a keyboard and a mouse. [...] They do not understand the meaning behind the characters, sounds and pictures they convey. I believe computers must be able to see and hear what we do before they can prove truly helpful. What is more, they must be able to recognize who we are and, as much as another person or even a dog would, make sense of what we are thinking.”

Natural interaction is an expression without fixed definition, despite being occasionally used in research papers. A collection of the general ideas and current developments behind this idea resulted in the following definition [58].

“Natural interaction is defined in terms of experience: people naturally communicate through gestures, expressions movements, and discover the world by looking around and manipulating physical stuff; the key assumption here is that they should be allowed to interact with technology as they are used to interact with the real world in everyday life, as evolution and education taught them to do.”

Research in natural interaction is focusing on two areas:

- Development of sensor devices sensitive enough to recognize subtle gestures, distinguish facial expressions, perceive variations in tone of voice and are able to interpret the semantics of these characteristics. These units also need to be cheap in order to achieve a high adoption rate in industry, commerce and at home.
- A new paradigm in user interface development that makes use of novel input options. Many operating systems feature small icons and active elements specifically suited for use with a mouse, which allows very precise cursor placement. Gesture-based user interfaces, either need to use techniques that allow selecting a location with similar precision or get rid of a cursor altogether.

Natural interaction in the context of this project is the deliberate or passive interaction with capacitive proximity sensing devices and the development of applications with the ability to use this sensor data to detect natural behavior and gestures. To achieve the first goal it is necessary to design devices that are able to detect gestures within their active area and develop software, able to consume those gestures to control user interfaces. Passive interaction is using the body sensing ability of capacitive sensors to unobtrusively locate a person at specific positions or register certain body properties, in order to trigger certain events amending assisted living environments. Developed demonstration applications are controlled using simple, easily learned gestures and give direct visual feedback on any detected event.

4.3.2. Applications

Scientific



Figure 4-10 (1) Person in perceptive space⁹ (2) Interactive workbench¹⁰ (3) HoloWall¹¹

Various research projects of the last 15 years have been investigating the next evolution of HCI, regularly following several concepts of natural interaction. These projects attempt to replace traditional input devices with multi-modal, perceptual systems that register user movement, gesture, mimic or speech. They are designed to allow a user to interact freely with a computer system. Common approaches include the usage of several video cameras, using feature extraction algorithms to identify user movement [62], microphone arrays, calculating user position [6] and large display areas, acting as output device. Several examples are shown in Figure 4-10. However, neither of those systems has been able to gain a wide audience.

Gaming

One sector, achieving great success in adapting devices recognizing natural interaction is the gaming industry. Especially the console market has spawned a vast amount of controlling devices and accessories that perceive natural interaction in a limited way. Sony's EyeToy [52] is a camera system that is able to detect user movement in order to control several games. Dance mats [31] that use pressure sensors to locate a player's position have been successful in Japan and the USA. Nintendo is the market leader of the current console generation, thanks to the Wii Remote [42], which is sensing motion and position, and other peripherals like the pressure sensing Wii Balance Board [43], used in fitness games. Music games, using devices that mimic actual instruments and microphones are selling millions of units each year [55]. Microsoft is currently developing a multi-modal input device that will include a depth-camera, a microphone-array and a RGB camera in order to register voice, gestures and location, preliminary called Project Natal [36].

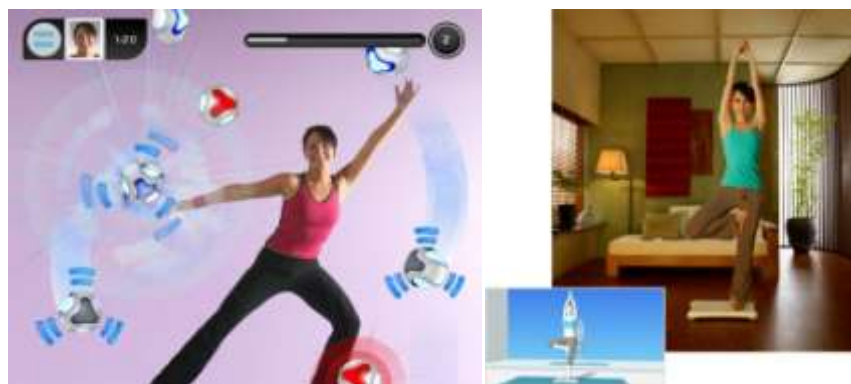


Figure 4-11 (1) Mixed-Reality game using the Sony EyeToy¹² (2) Woman using Nintendo Wii Balance Board¹³

⁹ Source: [62] – page 1

¹⁰ Source: [32] – page 6

¹¹ Source: [49] – page 2



Figure 4-12 (1) Group of people playing Rock Band game¹⁴ (2) Woman using Microsoft Project Natal demonstration unit¹⁵

Each of those devices has the ability to recognize natural behavior, albeit with limitations. The controllers are designed for a specific task, suited for usage with specific consoles, which affects the amount of available software. However there are projects that allow the use of the Nintendo Wii Remote on a PC [10] and Microsoft is thinking about making the Project Natal hardware available for Windows computers [19].

Handheld devices

Recent years have seen a considerable shift in the market for mobile music players and smart phones. Touch screen and finger gesture control are becoming the new standard, replacing mechanical buttons and stylus-based touch input [21]. The technology enables manufacturers to increase screen size without affecting overall dimensions and weight, enhancing applications like video playback, navigation and web surfing.



Figure 4-13 (1) Apple iPhone¹⁶ (2) Cowon S9 personal media player¹⁷ (3) Palm Pre phone¹⁸

¹² Source: <http://www.videogamesblogger.com/wp-content/uploads/2008/04/eyetoy-kinetic-screenshot-big.jpg>

¹³ Source: <http://www.cheatmasters.com/blog/wp-content/uploads/2008/04/wii-fit-big.jpg>

¹⁴ Source: <http://multiplayerblog.mtv.com/wp-content/uploads/2008/07/bloggers-rock-band.jpg>

¹⁵ Source: <http://media.industrygamers.com/editorial/2009/06/project-natal-demo.jpg>

Despite using finger gestures and avoiding a mechanical intermediary, it is discussable if this interaction is natural. Limited by the available active control area and the necessity of physical contact, the operating systems of those devices use the traditional point-and-click interface, adjusted for imprecise finger pointing, augmented with artificial gestures that simplify common tasks, including zooming and scrolling. Several companies offer attached mini-keyboards as text input via touch screen lacks haptic feedback and therefore is considered inferior to mechanical keys by many users.

Interestingly, several of these mobile phones also feature capacitive proximity sensors, designed to turn off the screen when in close proximity to the head, in order to save energy and avoid accidental triggering of the screen.

4.4. Interaction in Ambient Assisted Living Environments

4.4.1. Ambient Assisted Living

Ambient assisted living is a research area that emerged in the mid-2000s. The idea has been described in [53].

“AAL aims to prolongate the time people can live in a decent way in their own home by increasing their autonomy and self-confidence, the discharge of monotonously everyday activities, to monitor and care for the elderly or ill person, to enhance the security and to save resources.”

AAL research is developing concepts, methods, electronic systems, devices and services that provide support for a person in need of assistance, allowing them to continue a self-managed, socially included and healthy lifestyle in their familiar surroundings. It is adapting ideas from earlier research in ambient intelligence and ubiquitous computing, as well as evaluating new technologies for usage in AAL context. Numerous sensor devices unobtrusively located in the living environment of the assisted person are operating in a network, connected to a central control server, which is orchestrating functionality and applications.

The following table categorizes necessary services and shows existing technologies that can be used in AAL environments [41,39].

Categories	Subcategories	Example Technology
Emergency Treatment	Emergency prediction	Personal health monitor
	Emergency detection	Fall detection
	Emergency prevention	Smoke alert
Autonomy Enhancement Services	Cooking assistance	Service robot
	Cleaning assistance	Autonomous cleaning devices
	Medication assistance	Automated reminder
	...	
Comfort Services	Logistic services	Intelligent refrigerator
	Infotainment services	Home media center

Table 4-4 Necessary services and available technologies in assisted living

¹⁶ Source: <http://www.rca.ac.uk/UploadedImages/apple-iphone-in-hand-thumb.jpg>

¹⁷ Source: <http://feelgizmos.com/wp-content/uploads/2008/12/cowon-s9.jpg>

¹⁸ Source: <http://fredillies.files.wordpress.com/2009/06/palm-pre-1.jpg>

4.4.2. Available Technologies

Personal health monitoring systems use smart sensors that detect important vital signals, including blood pressure and glucose levels. This data is sent to a central system within the patients environment, which in turn is connected to a phone network, in order to call for emergency if necessary [54]. The central system is connected to a database and is using stored values to predict impending medical emergencies.

Fall detection registers unintentional falls, which are a major cause for injury in the elderly population. Most commonly a wearable device based on interpreting accelerometer data is used [9]. Detected falls can be forwarded to emergency physicians.

Smoke alerts are a common system, detecting smoke in buildings. Concerning elderly people with an increased risk of fire hazard, most notably persons cooking for themselves and smokers, a smoke detection can automatically alert the closest fire department, as in many cases those people might not be able to leave the building by themselves in time.

Several companies are providing autonomous service robots designed to assist impaired persons in their everyday tasks [20,27,24]. Japan, having the most imminent issue of an ageing population, is adapting service robot technology at a fast pace. Those universal robots can be used to fetch objects for the user, provide simple social interaction, cook, clean, etc.



Figure 4-14 (1) Tunstall Fall Detector¹⁹ (2) Fujitsu Service Robot enon²⁰ (3) iRobot Roomba vacuum cleaner²¹ (4) DFKI intelligent refrigerator²²

Simple domestic robot systems designed for specific tasks are a market that is growing rapidly [26]. The most common systems are robotic vacuum cleaners and robotic lawnmowers. While those tasks

¹⁹ Source: http://www.atdementia.org.uk/content_files/images/TunstallFalldetector.jpg

²⁰ Source: <http://www.frontech.fujitsu.com/imgv3/jp/group/frontech/en/forjp/robot/servicerobot/pic-servicerobot.jpg>

²¹ Source: http://www.xconomy.com/wordpress/wp-content/images/2007/08/roomba560_sideview.jpg

²² Source: <http://www.dfki.de/web/living-labs-en/RetailLaboratoryIRL.jpg>

could possibly be done by generic service robots it is often cheaper and therefore viable to use numerous simple robots that are controlled by an intelligent central server.

Intelligent refrigerators are a prominent example for advanced domestic electric devices. They systematically monitor their content and are able to automatically create shopping lists or even order food autonomously [16].

Home media centers are dedicated computer appliances designed for the living room, using a television as output device in order to simplify access to media sources. Elderly people often have problems controlling the multitude of devices present, as there is no common standard for remote controls and user interfaces. Media centers can combine all possible sources and provide a generalized surface controlled by a single input device. It can also be attached to health monitoring to give visual and audible reminders for medication or allow visual communication with the user's physician or family.

5. Modeling Input Devices

Based on the decision made in chapter 4.2.2 this chapter presents the modifications made to the input device taxonomy of Card, Mackinlay and Robertson, as well as the modeling of three different input devices.

5.1. Example for the Taxonomy of Card, Mackinlay and Robertson

To exemplify the modeling of Card, Mackinlay and Robertson the following section outlines the steps required to model a mechanical mouse with two buttons and a scroll wheel. Detection of movement necessitates two linear movement sensors. In case of a mechanical mouse these are rollers connected to a ball that turn optical disc encoders. The results of both sensors are merged, generating a two-dimensional movement. Buttons can be modeled as simple two-state position sensors, the scroll wheel as rotary angle difference detector. Respective six-tuples are displayed in Table 5-1.

	x-Movement	y-Movement	Button	Scroll Wheel
M	dPx	dPy	Pz	dRx
In	<i>Real number</i>	<i>Real number</i>	$\langle 0,1 \rangle$	<i>Real number</i>
s	s	s	S	s
R	<i>Linear function</i>	<i>Linear function</i>	<i>Static function</i>	<i>Linear function</i>
Out	<i>Real number</i>	<i>Real number</i>	$\langle 0,1 \rangle$	<i>Real number</i>
W	-	-	-	-

Table 5-1 Mechanical mouse sensors

The input domains of both movement sensors have to be combined using the merge operator. The final unit contains the four resulting inputs, combined via the layout operator. Finally in many applications the output of the mechanical mouse is linked to the mouse cursor on the screen, which acts as virtual input device detecting the absolute position on the display and is modeled using the connect operator. The resulting composition tree is pictured in Figure 5-1.

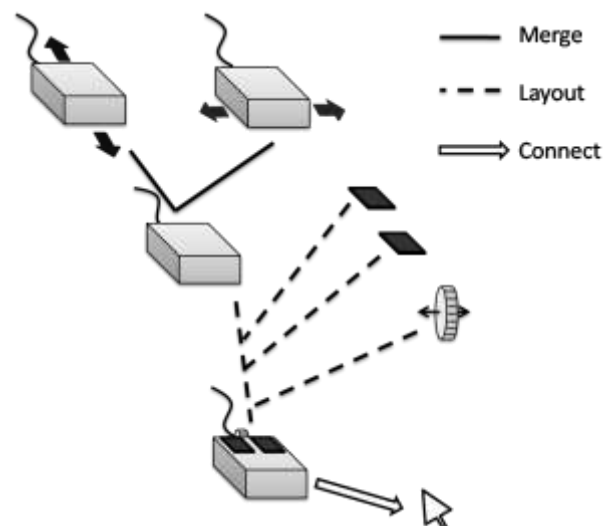


Figure 5-1 Modeling mouse and linking to cursor

5.2. Data Processing

Any input device consists of one or more sensor units and is processing their raw data in order to generate events, symbolized in Figure 5-2. The processing unit of an input device needs to abstract this sensor data to a format that is usable by attached machines. If several units are working in a single input device it is often necessary to fuse the measured data to acquire useful information. A common example is a web cam, which might consist of several million photon sensors that are attached to an image processing unit, using all available data to generate a single image. Another optional functionality is applying logic to the acquired data, e.g. optical mice, which use an image sensor to detect movement. In addition to fusing the single photon sensors this requires the application of optical flow algorithms [46] in order to generate two-dimensional movement data. It is also possible to configure the processing in several input devices, an example being modern mice that allow on-the-fly adaption of their sensitivity, which allows the user to change pointer behavior in specific applications.

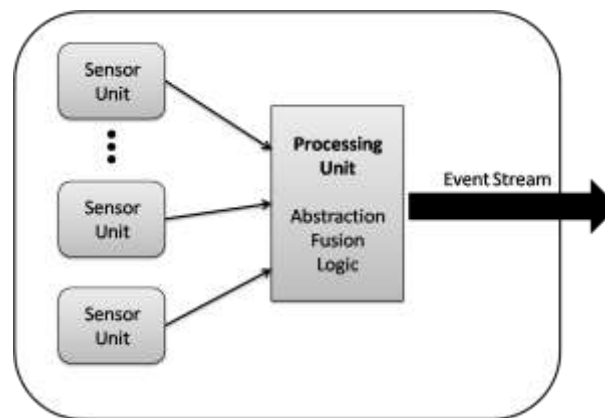


Figure 5-2 Input Device Processing Model

Common methods for sensor data processing are the Kalman filter [12], which estimates the state of a dynamic system based on preliminary assumptions, or sequential Monte Carlo methods [17], simulation based estimation techniques. In order to estimate the position of an object, Smith used a numerical approximation to the Laplace equation of electromagnetic fields [50]. Concerning the actually designed input devices of chapter 7 this processing is occurring on the CPU of the attached machine, though it was attempted to use light-weight algorithms that in future iterations can be easily delegated to a dedicated processing unit within the input device. Future iterations might rely on more sophisticated data processing algorithms.

Generally, any attached sensor is transforming a physical property to an electrical signal that is changing the current state of the input device, e.g. a mouse button is set active if force exerted on the button area is exceeding the threshold required to push the underlying pressure sensor to its on-position, which is causing a signal. Changing the current state of an input device requires at minimum a changed measurement of a single sensor. Those minimum changes in sensor state are subsequently called fragments.

5.2.1. Modeling Interaction with Capacitive Proximity Sensors

CapSense units are isotropic proximity detectors. They do not detect a position in Cartesian coordinates but detect a radius in spherical coordinates with undetermined angles. As mentioned in chapter 4.1.2, it is difficult to acquire absolute distance parameters. Instead two operational modes generating fragments are suggested.

Proximity Mode

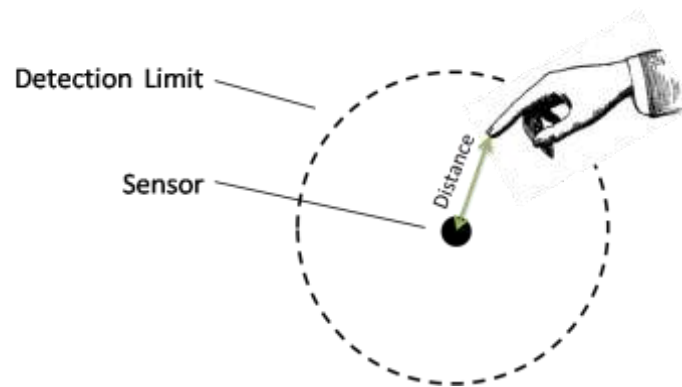


Figure 5-3 Proximity Mode

Proximity mode, displayed in Figure 5-3, is the continuous recording of a relative distance measure. Depending on current capacitance data a detected object is assumed to be at a specific distance to the sensor. Instead of mapping the data to an absolute measurement they are transferred to an arbitrary number. Conclusively proximity mode is continuously detecting the radii of spherical coordinate locations and is mapping those to a range of real numbers using a transfer function.

State Mode

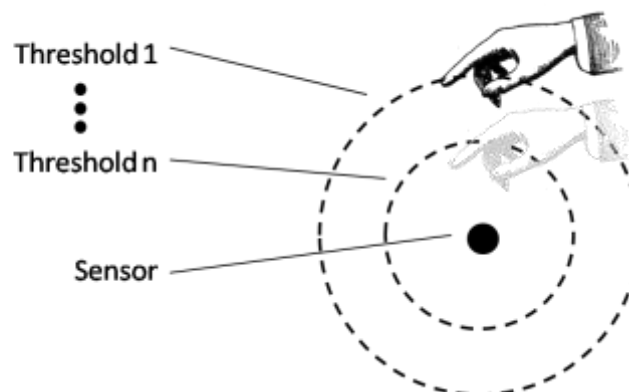


Figure 5-4 State Mode

State mode, displayed in Figure 5-4, is using one or more threshold levels in order to map proximity values into distinct states. It is basically the quantization of gathered capacitance data on sensor level, abstracting the data for later usage in input devices. Exceeding or falling below any of those thresholds is registered as fragment, while remaining between thresholds is ignored. Therefore state mode is detecting a number of specific radii in spherical coordinates and mapping those to certain integer numbers using a static function.

Singular CMR Sensors

Transferring these findings to the CMR-Model, a capacitive proximity sensor is able to act as different singular input devices. Respective six-tuples are displayed in Table 5-2. P_r represents a radius in spherical coordinates.

	Proximity Mode	State Mode
M	P_r	P_r
In	<i>Real number</i>	$\langle 0, \dots, n \rangle$
s	S	S
R	<i>Transfer function</i>	<i>Static function</i>
Out	<i>Real number</i>	$\langle 0, \dots, n \rangle$
W	-	-

Table 5-2 Capacitive proximity sensor input devices based on mode

5.2.2. Data Fusion

Sensor data fusion is the process of combining sensory data generated by several sources in order to increase the resulting information [12]. While it is possible to use a single capacitive proximity sensor as input device, it is often beneficial to use an array in order to acquire additional data, most notably the relative location between sensors. In order to successfully fuse sensor data it is necessary to provide a structure that is storing this data and includes information necessary to identify single sensors free of doubt. In case of a computer mouse this structure is a simple two-element array containing the detected two-dimensional movement, while a web cam combines thousands of gray level or RGB values to a single image using knowledge of each pixel location.

5.2.3. Data Processing Logic

Modern powerful microcontrollers allow delegating processing to the input device itself. Professional web cams allow the application of color space conversion and image manipulation operations before the image is submitted to the connected machine [25]. Concerning the sensor units of this project, noise reduction algorithms are outsourced to the microchips of the devices, described more detailed in chapter 6.2.1. However, so far there is no further processing of the fused data within the input devices.

Methodically this post-processing is a transformation of the data fusion structure into an output structure. This process can be integrated into the CMR-Model if the data processing is regarded as a black box virtual input device that acts as an intermediary between the physical input device and a connected PC.

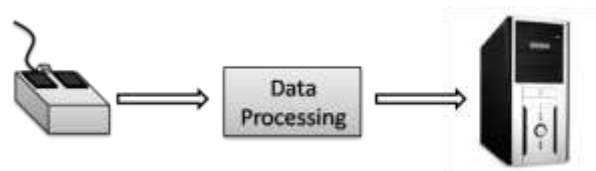


Figure 5-5 Data processing in the CMR-Model

Referring to the input device processing model, any change in this output structure is triggering an event.

Semantic Mapping

The processor may choose to ignore certain changes in sensor activity or amplify data acquired by other sensors depending on the programmed logic. As this process is adding an implied significance to acquired values, it is called semantic mapping.



Figure 5-6 Data processing pipeline

Generally it is useful to apply semantic mapping to specifically designed, intermediary data structures created by the post processing. A device consisting of two capacitive proximity sensors with 16-bit resolution possibly creates 2^{32} different combined values. If it is desired to determine which sensor is closer to a detectable object only two results are possible. If the application attempts to semantically map each possible value this would be a waste of processing time and storage. Instead post processing should use the input values to calculate a single average, which leads to a single value that needs to be processed by the semantic mapping routine.

In order to formally describe semantic mapping a structure S is defined as a set of n attributes A_i .

$$S = \{A_1, A_2, \dots, A_n\}$$

Any attribute A_i has a domain D of possible values v . The current status of the structure therefore is the set s of all current values of each attribute.

$$s(S) = \{v(A_1), v(A_2), \dots, v(A_n)\}$$

The amount of possible states is the cross product of all value domains.

$$N_{S_{in}} = D(A_1) \times D(A_2) \times \dots \times D(A_n)$$

$$N_{S_{out}} = D(A_1) \times D(A_2) \times \dots \times D(A_m)$$

The semantic map is the function associating any possible state of the input structure to a state of the output structure.

$$f_S: N_{S_{in}} \rightarrow N_{S_{out}}$$

Semantic Mapping Notation

If the number of states is sufficiently small, a graphical notation of the semantic map is reasonable. For the purpose of describing a structure, a simple tabular notation including name, attributes and domain is adequate.

Structure Name	
Attribute 1	$D(A_1)$
Attribute 2	$D(A_2)$
...	...
Attribute n	$D(A_n)$

Table 5-3 Structure setup

The domain is either a limited or unlimited set, e.g. a Boolean value or a real number. Single mappings are noted as shown in Figure 5-7.



Figure 5-7 Generic semantic mapping

Participating attributes of the input structure and their respective values are noted in white boxes on the left side. Those are mapped to certain attributes of the output structure, indicated by grey boxes on the right side, and manipulate their values. Non-participating attributes are not displayed to avoid cluttering. The number of attributes on each side depends on the desired effect and does not have to correlate. Therefore the semantic map consists of a certain amount of single mappings. Any combination of the input structure that is not displayed is ignored and does not lead to a change within the output structure.

Using this notation it is possible to formally design input devices based on capacitive proximity sensors. In the scope of this project three different units have been designed and implemented and are presented in the following sections.

5.3. Common Model

CMR-Model

Every designed input device is using six capacitive proximity sensors to gather data. That amount is deemed reasonable for the desired applications as it combines a good spatial resolution with sufficiently fast data transfer. Even though the software is able to handle additional sensors, the PC Bridge is prone to freezing with increased traffic, which is going to be detailed in chapter 6.2.2.

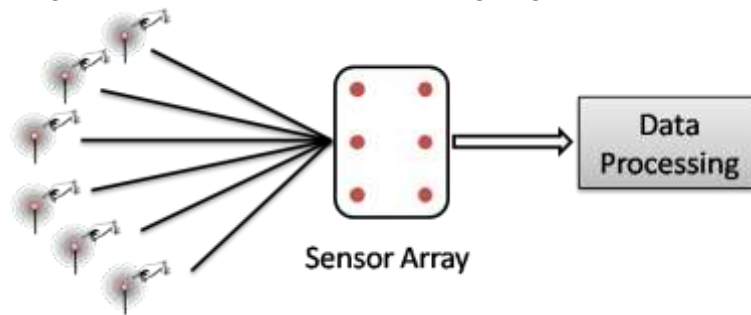


Figure 5-8 Full CMR model

Every sensor is running in proximity mode. The proposed devices need direct access to the acquired data in order to conduct required calculations. All input devices therefore share a common initial CMR-Model consisting of six proximity mode capacitive sensors merged to a single sensor array. This unit is connected to a virtual data processing input device that is unique to each designed input device and detailed in their appropriate sections.

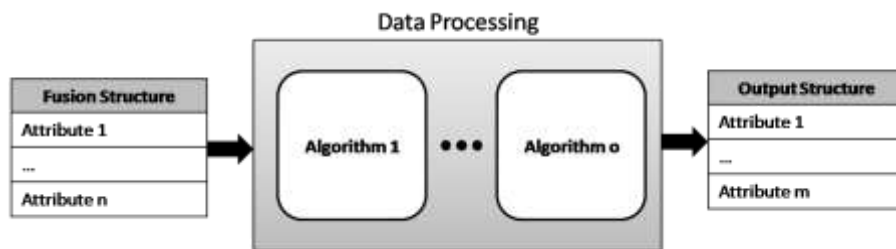


Figure 5-9 Data processing black box detailed

The generic method to detail the data processing is shown in Figure 5-9. The data acquired from the sensor array is amended with additional information to generate a data fusion structure that acts as input domain for the virtual data processing input device. The type of data to amend is depending on the required task. In order to apply a localization algorithm, the location and sensor ID needs to be known. This structure can also be used to group sensors. Detailed examples are following in the next sections. The data processing itself applies several algorithms that in conclusion map the fusion structure into an output structure that has its own set of attributes. The number of attributes and

algorithms are independent from each other and may even be changed on run-time if require. In the scope of this project the modification of the data processing is used frequently by modifying attributes used by the algorithms. Any modeled input device has a distinct set of fusion structure, algorithms and output structure that are detailed in the appropriate sections.

5.4. Proximity Table

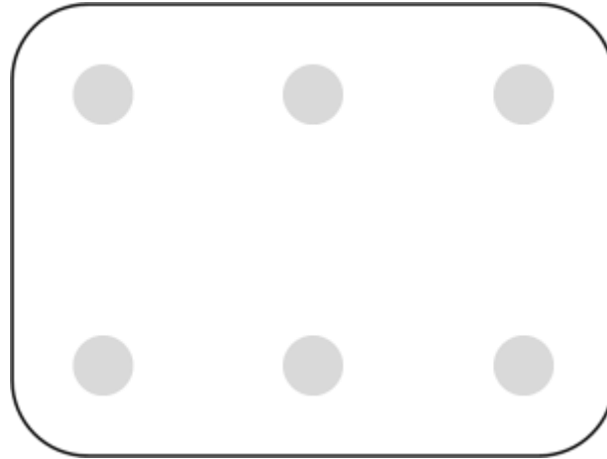


Figure 5-10 Design model of Proximity Table input device

5.4.1. Idea

The Proximity Table is an input device designed for explicit, spatial interaction. It uses the acquired data to detect natural, three dimensional gestures. It is desired to integrate such arrays into furniture of the living room, replacing the plethora of remote controls and combine it with software giving visual and audible guidance. This should allow elderly people to control their electronic devices with gestures that can be easily learned. Additional applications include sterile environments, e.g. an operating room where the surgeon is not allowed to touch any surfaces but might want to interact with a visualization of medical data, made possible by using touchless proximity sensing devices.

5.4.2. Modeling

The desired output is a three dimensional location that can be consumed by applications. Sensor data and amended sensor positions are used to interpolate this location. Additional filtering algorithms are applied to achieve a more stable localization.

Data Fusion

The structure required to hold data acquired by the capacitive proximity sensors of this project is using additional information about sensor location and a unique ID for each sensor, pictured in Figure 5-11.

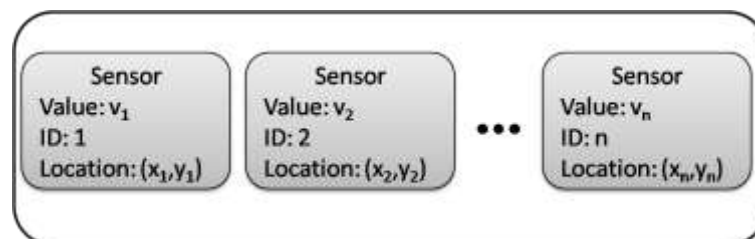


Figure 5-11 Proximity sensor fusion structure

Location is a floating point number in the range of 0 to 100, while ID is a unique identifier for each single sensor unit. The registered sensor data v_i is a positive integer number.

Data Processing

In order to calculate the position of a detectable object a localization algorithm is required. The selected method for planar localization is the weighted average calculation, generically depicted in the following equation, with e_i being an element of a set and w_i its assorted weight.

$$\bar{e} = \frac{\sum_{i=1}^n w_i e_i}{\sum_{i=1}^n w_i}$$

This equation can be adapted to the planar localization problem by using the floating point range of sensor position (x_i, y_i) as set and the current sensor value v_i as weight, resulting in respective formulas.

$$\bar{x} = \frac{\sum_{i=1}^n v_i x_i}{\sum_{i=1}^n v_i}$$

$$\bar{y} = \frac{\sum_{i=1}^n v_i y_i}{\sum_{i=1}^n v_i}$$

While there are different, more advanced localization algorithms, this approach has several advantages. The implementation is easy and computationally inexpensive, thus applicable on a low-power microcontroller. Furthermore it is working with a single active sensor while other common approaches, including trilateration or multilateration require at least three active receivers, which cannot be guaranteed at the edge regions of the Proximity Table.

Determining a height value requires a different approach as every sensor is located on the same plane, resulting in a constant z-position. Acquired values increase if a detectable object is approaching the sensor. The same rule applies for sensor arrays, i.e. the sum of all values is an indicator for the current height. However this is only valid for dense layouts, where distance between sensors is considerably closer than the maximum detection distance. Otherwise it is possible to touch the surface without triggering any sensor. A first attempt at determining height was using fix threshold levels, enabling five different height values. In order to generate a more precise z-position this was later replaced by using linear interpolation between four adaptable fixed points at given totals. Values that are smaller than the minimum fixed point are deemed far off and assigned an infinite height. Higher values are clamped to the minimum height of 1. One example with fixed points at values 40, 100, 200 and 1200 is shown in Figure 5-12.

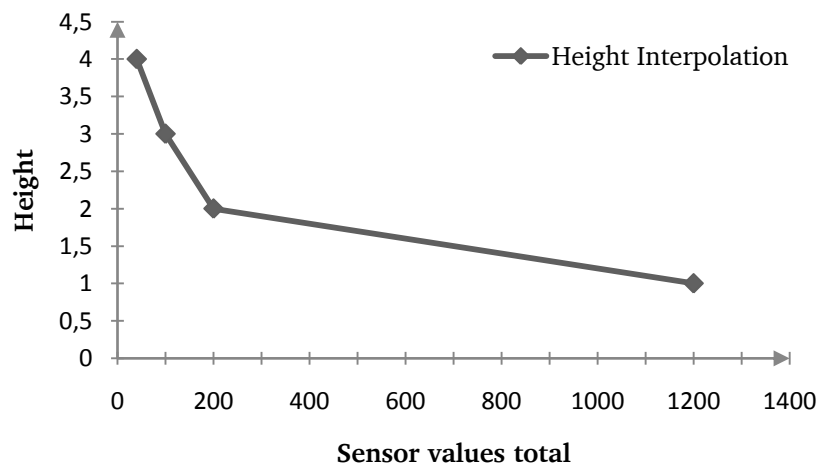


Figure 5-12 Linear interpolation of height values

A major problem at small totals is the influence of sensor noise, causing unreliable localization, most notably if a detected object is just within range of the array. While noise is present in any sensor device its magnitude is increasing together with the amount of sensors used in subsequent calculations. This is especially important for the Proximity Table as it is the only device using all sensors at all times. Figure 5-13 is showing a graph of gradually increasing sensor total over time, while the blue area is the mutual noise of all sensors. This area covers the worst case scenario for occurring noise, i.e. if each sensor does have an inherent noise level fluctuating between 0 and 10, six sensors will have a combined noise level between 0 and 60. Therefore a higher overall sensor value results in a smaller influence of noise.

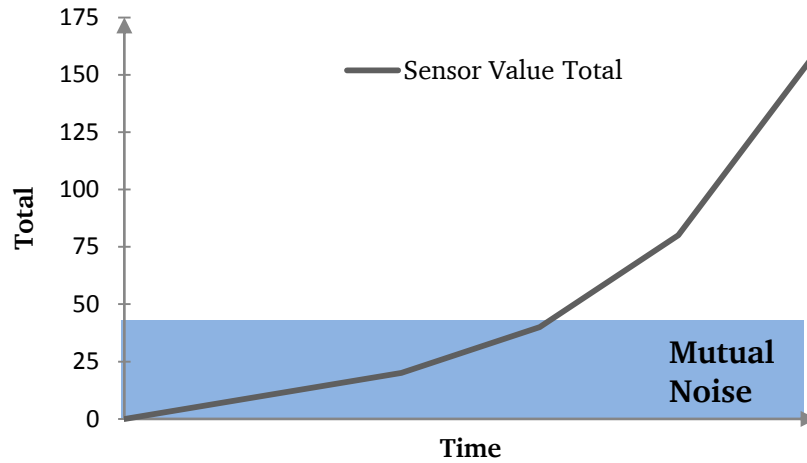


Figure 5-13 Influence of noise at different sensor value totals

In order to deal with noise, three different precautions are taken. The first is a simple threshold level, necessary to avoid calculating locations based on very small values, resulting in exceptionally strong influence of noise. Consequently it is not possible for the data processing to differentiate between high noise and small signals, rendering a localization attempt using those values useless. A threshold level is implemented, ignoring small totals to avoid determining locations of such high uncertainty.

The second part is the application of filtering to the signal submitted by each sensor. Two filtering algorithms are supported. Average filtering is taking the arithmetic mean over a number of samples, resulting in a smoother output.

$$\bar{v} = \frac{1}{n} \sum_{i=1}^n v_i$$

Another filtering method is median filtering, which is using the median over a number of samples. This algorithm has the advantage that short bursts of noise might eventually be completely ignored if applied properly. Computationally it requires a sorting algorithm and picking the middle samples depending if the number of samples n is even or odd. If $v_{(i)}$ is the i -th element of a sorted data set the median is defined as:

$$\tilde{v} = \begin{cases} v_{(\frac{n+1}{2})}, & \text{if } n \text{ is odd} \\ \frac{1}{2} \left(v_{(\frac{n}{2})} + v_{(\frac{n}{2}+1)} \right), & \text{if } n \text{ is even} \end{cases}$$

The third and final step in reducing the influence of noise is the smoothing of the calculated locations. Remaining noise is causing slightly different calculated positions even at static hand positioning. If the hand is gradually moving the detected location will not follow the path directly but instead will fluctuate around the trajectory. Two interpolation algorithms have been implemented. The first method is a simple linear method that calculates an average value using a given amount of previous

locations, similar to the average sensor filtering, although applied in output space rather than on individual sensors.

$$x = \frac{1}{n} \sum_{i=1}^n x_i \quad , \quad y = \frac{1}{n} \sum_{i=1}^n y_i \quad , \quad z = \frac{1}{n} \sum_{i=1}^n z_i$$

This approach results in smooth trajectories of adjacent points but is not able to eliminate the noise-induced location drifting completely. In order to achieve fixed locations a different algorithm is necessary. This second method is using a metric between consecutive points that decides if a new location is calculated. If the value is below a certain threshold t the previously calculated location remains, if it is above the new location is selected.

$$\begin{aligned} x &= \begin{cases} x_{i-1}, & m(x_{i-1}, x_i) \leq t \\ x_i, & m(x_{i-1}, x_i) > t \end{cases} \\ y &= \begin{cases} y_{i-1}, & m(y_{i-1}, y_i) \leq t \\ y_i, & m(y_{i-1}, y_i) > t \end{cases} \\ z &= \begin{cases} z_{i-1}, & m(z_{i-1}, z_i) \leq t \\ z_i, & m(z_{i-1}, z_i) > t \end{cases} \end{aligned}$$

However it is difficult to select a proper threshold level and each selected location is prone to noise. Figure 5-14 is visualizing the difference between non-interpolated locations and both methods using nine calculated locations. Linear interpolation is approximating the actual hand movement well but is not able to stay fixed at the target location. Differential interpolation is able to stay fixed at a position, but is ignoring the majority of calculated locations and is more susceptible to noise.

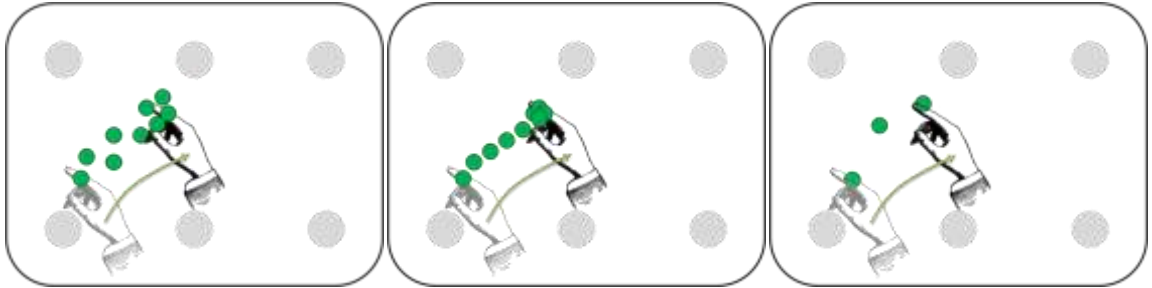


Figure 5-14 Result of no interpolation (1), linear interpolation (2) and differential interpolation (3)

Generally the linear interpolation yields more satisfying results and should be used unless it is crucial for the application to locate fixed positions.

All steps of the noise reduction are combined in the listing of an algorithm in pseudo-code on the following page:

```

int[] filteredData;

foreach(Sensor s)
{
    switch(filteringAlgorithm)
    {
        case Average:
            filteredData = Average(numberOfSensorSamples);
        case Median:
            filteredData = Median(numberOfSensorSamples);

    }
}

float[] position3D;

int total = Sum(filteredData);

if (total < mutualNoiseThreshold) position3D = null;

position3D = weightedAverage(filteredData, sensorLocations); //localizations based on sensor locations and values as weight

switch(interpolationAlgorithm)
{
    case Linear:
        position3D = Average(numberOfPositionSamples);
    case Differential:
        difference = position3D – lastPosition; //difference to previous positions
        if(difference > differentialThreshold) position3D = position3D; //new position if big difference
        else position3D = lastPosition; //keep position if small difference
}

```

Additionally, routines have been integrated to detect multiple objects, adapting the multi-touch paradigm to proximity sensing. It is not possible for a capacitive proximity sensor to differentiate between two objects in range at far distance and a single object at lower distance. In order to work around this limitation the number of active sensors is used. If it is assumed that the detectable objects are similar sized, the region influencing the electric fields is increasing with the number of objects, thus triggering more sensors, shown in Figure 5-15.

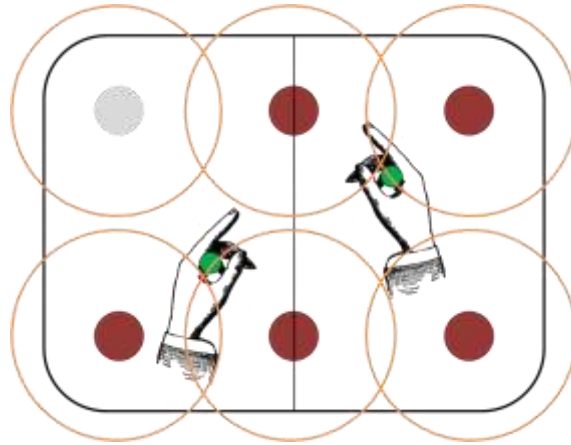


Figure 5-15 Multiple objects increasing number of active sensors

However this does fail if the distribution of sensors is sparse and the objects are close together. Therefore the Proximity Table is divided into left and right regions, which independently track the location of objects as soon as enough sensors are active enabling multi-touch recognition. Since only six sensors are used for the Proximity Table and two of them are located centrally those have to be assigned to both regions, either contributing fully to both independently running weighted average calculations, determining point location, or using a distribution function to partially assign values.

Due to the low number of sensors causing unstable localization it has been decided to refrain from using multi object detection in any application until a prototype with additional sensors is available. Concluding the data processing there is no usage of semantic mapping and the output structure is a list of three-dimensional floating point locations leading to the following data processing virtual input device.

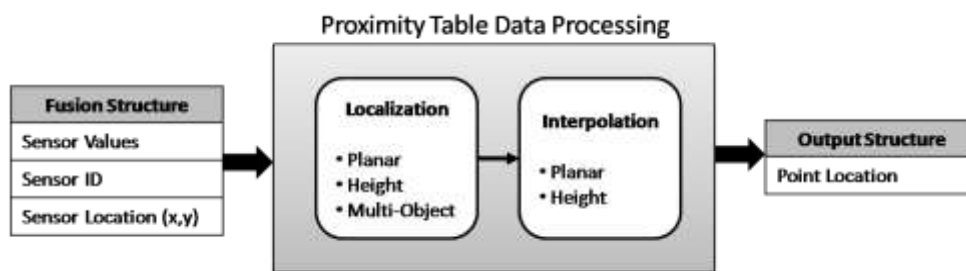


Figure 5-16 Proximity Table data processing virtual input device

5.5. Bed

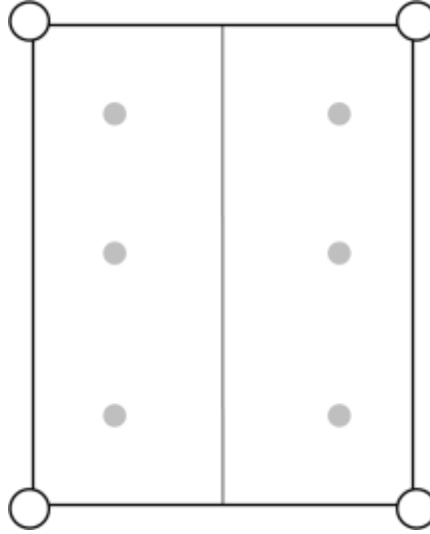


Figure 5-17 Design model of Bed input device

5.5.1. Idea

In the scope of this project the Bed is an input device amending a full size bed with capacitive proximity sensors. The units are attached unobtrusively on the bed frame below the mattress. The aim is to gather information about the orientation and posture of its occupants. In assisted living environments it is useful to monitor sleep activity in order to register unusual, possibly dangerous patterns. Additional functionality is the calculation of the weight distribution of the body, which is an indicator for stress on the vertebral column, thus being useful in discovering unhealthy sleeping positions or unsuited mattresses.

5.5.2. Modeling

The desired output is information about the location and posture of bodies on the Bed. Sensor values and information about sensor location is used to create an intermediary data structure that is semantically mapped to the output structure.

Data Processing

The data fusion structure is identical to the respective Proximity Table structure. In order to determine location and posture, an intermediary data structure is created that divides the Bed into two areas. Sensors are dynamically assigned to these regions based on their predefined positions, stored in the fusion structure, specifically the y-coordinate. The area A is stored as a Boolean value, with 0 assigned to the left side and 1 to the right. Accordingly, a sensor S_i is assigned using the following equation, if the y-axis is located vertical to the regular lying position and its value a floating point value between 0 and 100.

$$A(S_i) = \begin{cases} 0, & y \leq 50 \\ 1, & y > 50 \end{cases}$$

The intermediary structure furthermore contains activity information about each sensor as well as balance information within each area and in between areas.

Bed Intermediary Structure	
Sensor 1 Active	Boolean
Sensor 2 Active	Boolean
Sensor 3 Active	Boolean
Sensor 4 Active	Boolean
Sensor 5 Active	Boolean
Sensor 6 Active	Boolean
Left Area Balance	Float
Right Area Balance	Float
Between Area Balance	Float

Table 5-4 Bed intermediary structure

Sensor activity is defined by a simple threshold level, emulating a simple two-state device. The different balance attributes are calculated using weighted average algorithms within the areas. Information about a person sitting or lying and current position and orientation are gathered by searching for specific configurations of sensor activity. The example shown in Figure 5-18 is visualizing the pattern for one person lying vertically on the left side of the Bed, activating all sensors on the left side.

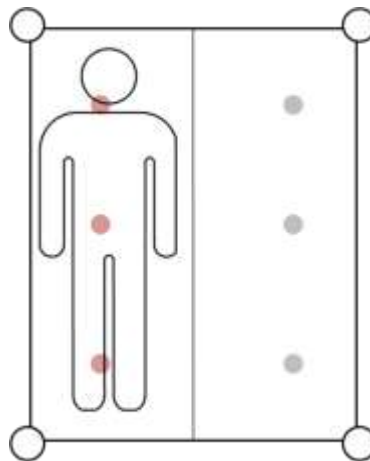


Figure 5-18 One person lying vertically on left side of the Bed

Calculating the strain on the spine is done using the determined balance attributes. Depending on the configuration and resulting body position either single balances or weighted combinations of the balances are used. This calculation is based on a simplified model that is considering equal weight distribution as ideal. Therefore balance points within a range around the average location trigger a good spine strain.

The output data structure is a list of Person data structures containing attributes interesting for the detection of posture and orientation.

Person	
x-Position	Integer
y-Position	Integer
Rotation	Integer
Lying	Boolean
Spine strain	Integer

Table 5-5 Person data structure

Any sensor activity configuration is semantically mapped to a certain Person data structure. Using the previously introduced graphical notation above example can be mapped as shown in Figure 5-19.

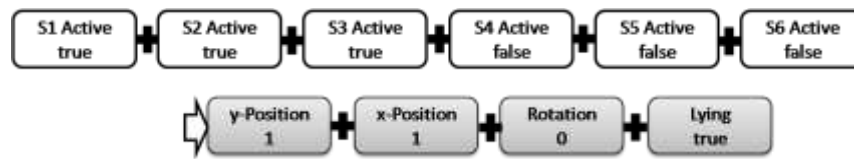


Figure 5-19 Semantic mapping of sensor configuration to Person structure

A simplified notation and resulting full listing of all interesting configurations and their semantic mappings are found in appendix C. Again the result of the data processing is used to model a virtual input device, pictured in Figure 5-20.

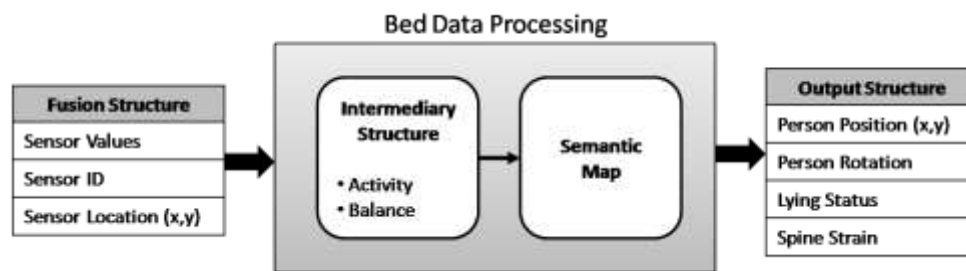


Figure 5-20 Bed data processing virtual input device

5.5.3. Problems

Sensors values in this case do not solely depend on the proximity of a detectable object but also on the pressure exerted on the antennas. As noted in chapter 6.1.2, changes in antenna geometry are reflected by changes in capacitance. Thus the CapSense units are additionally working as pressure sensors in this input device. It was not examined how the resulting value is distributed between proximity and pressure yet a larger influence of pressure is assumed, as acquired values are considerably larger than in sole proximity applications. An appropriate threshold level determining sensor activity depends on the individual geometry of the Bed and thus has to be determined using experimentation.

Currently it is necessary to predefine if either one or two bodies will occupy the Bed as there are several ambiguous configurations, e.g. two persons sitting on the upper left and upper right or a single person lying horizontally on the upper side of the Bed. An automatic detection based on sitting recognition should be implemented in future versions, i.e. the detection of higher than usual sensor values for a short time when a person is getting seated on the mattress for the first time. Another possibility is increasing the number of sensor units and approximate amount of active sensors, similar to multi-object recognition on the Proximity Table.

Another issue is the model for spine strain currently used. The assumptions made do not necessarily reflect forces endured in actual scenarios. It is intended to cooperate with orthopedists in order to create a more accurate bio-mechanical simulation based on concepts derived from mechanical and models. Additional sensor units should help to register more elaborate strain approximation, allowing for a more reliable prediction of potential, spine related health issues.

5.6. Active Floor

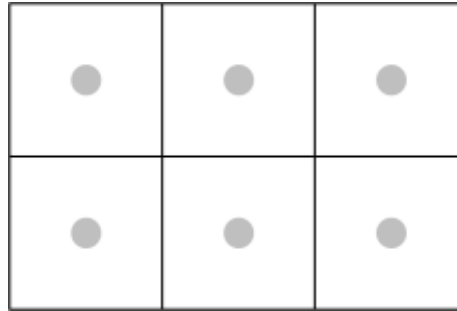


Figure 5-21 Design model of Active Floor

5.6.1. Idea

The Active Floor is the application of capacitive proximity sensors below the carpet or parquet of a room. This setup can be used to locate the position of a human body and its proximity to the ground. In AAL environments there are two primary applications for the Active Floor:

- Fall detection, automatically triggering necessary actions upon registering the fall of an elderly resident
- Location based event triggering, e.g. automated light switching upon entering or leaving a room

The sensors are able to distinguish between a standing person and a lying person, making use of the considerably higher capacitance value a lying body effects.

5.6.2. Modeling

The device is specifically designed to be applied within a room. Therefore the output information is a list of persons that are currently in the room and their respective location and status.

Data Fusion

Fusion data consists of sensor values, sensor ID and a floor tile, represented by a rectangle structure, in this case consisting of the lower left point, width and height. The fusion structure is pictured in Figure 5-22.

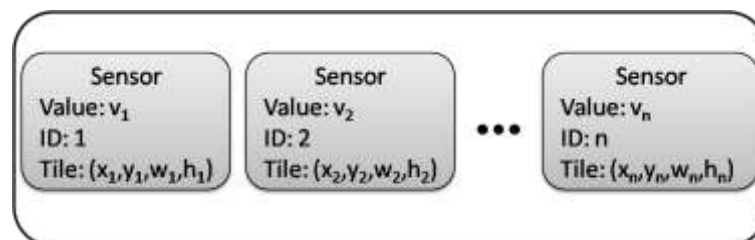


Figure 5-22 Active Floor data fusion structure

Data Processing

The Active Floor needs to distinguish between no activity, a walking/standing person and a lying person. This is realized by employing two threshold levels to the sum of registered values. Current status s therefore is calculated using the following equation.

$$V = \sum_{i=1}^n v_i, \quad s(V) = \begin{cases} 0, & V < t_1 \\ 1, & t_1 \leq V < t_2 \\ 2, & V \geq t_2 \end{cases}$$

Two different methods for calculating the resulting position of a person are proposed and shown in Figure 5-23.

- Direct association with tile. If one is detecting activity it is assumed that the person is positioned in its center. In case of multiple active tiles the center of the most active one is chosen.
- Weighted average calculation between tiles, analog to localization on the Proximity Table. Tile centers are used as fixed points.

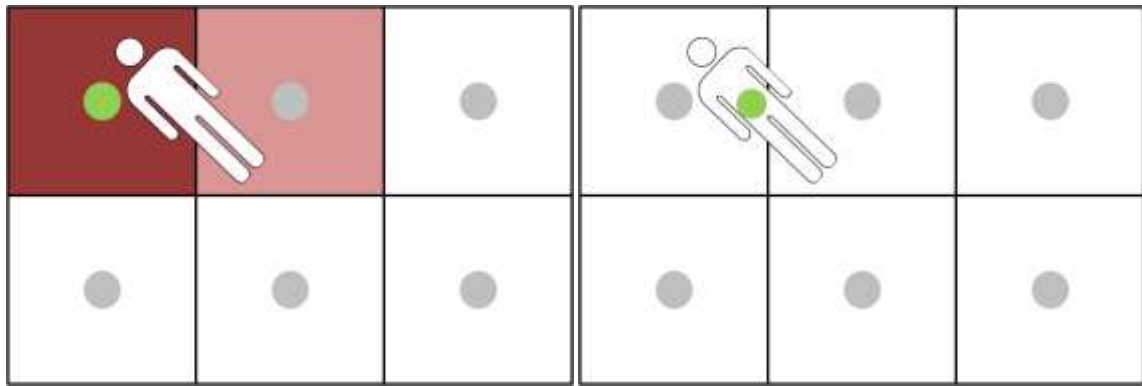


Figure 5-23 (1) Localization using most active tile (2) Localization using weighted average

The resulting output structure is a person similar to the Bed, yet with a different set of attributes.

Person	
x-Position	Integer
y-Position	Integer
Status	Integer

Table 5-6 Active Floor Person data structure

There is no semantic mapping necessary to create the output structure. A virtual input device for the Active Floor data processing is therefore given in Figure 5-24.

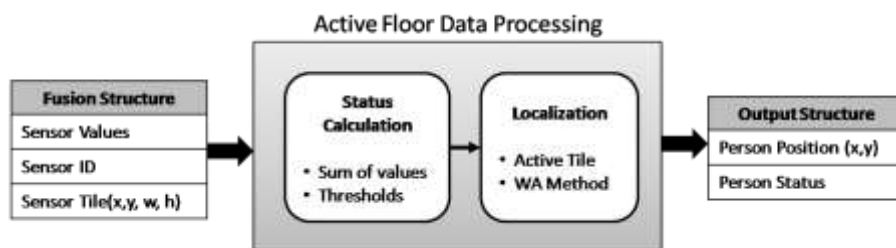


Figure 5-24 Active Floor data processing virtual input device

5.6.3. Problems

Uniformity of Tiles

Functionality of the Active Floor relies on the rough assumption that the signal over the whole area of a tile is uniform, which would require the sensing electrode to be homogenously distributed below the whole tile. Since the sensors of this project rely on wire antennas those need to be bended to cover the whole tile. Yet a longer antenna increases noise level up to a point making reliable measurements impossible (Chapter 6.1.2). Consequently tile size and covered area within a tile have to be reduced. However the results are still good enough for detecting position and falls.

Detecting multiple persons

Though the Active Floor is designed for application in single person environments, it is also possible to detect multiple persons, by applying the thresholds to the detected values of each single tile. However there are several ambiguous configurations. If two or more persons are standing close together the result may be interpreted as a single person lying (Figure 5-25). Accordingly if a single person is lying on two tiles that may result in detecting two standing persons (Figure 5-26).

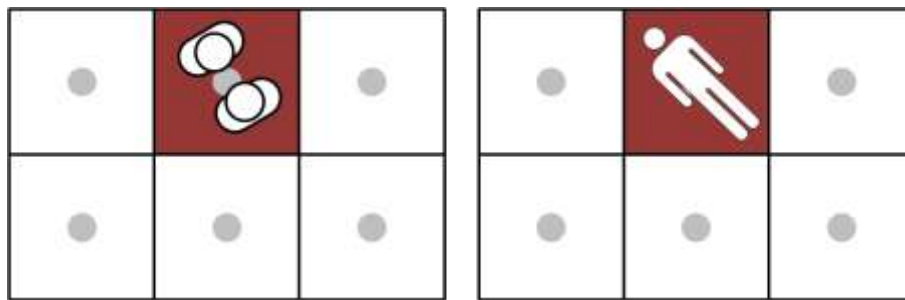


Figure 5-25 First ambiguous configuration of floor tile - two standing or one lying on single tile

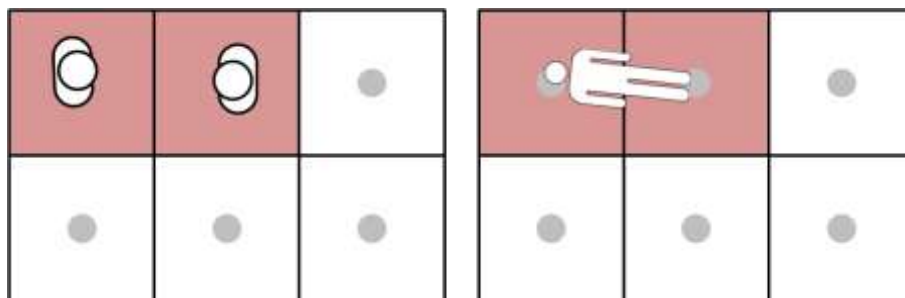


Figure 5-26 Second ambiguous configuration of floor tiles – two standing on neighboring tiles or one lying

In order to avoid misinterpretations it is required to detect the number of persons currently in the room. A simple way is assigning a specific identifier to the tile connected with the room entrance. Activity of this tile is continuously logged, registering entry and leave events, as shown in Figure 5-27.

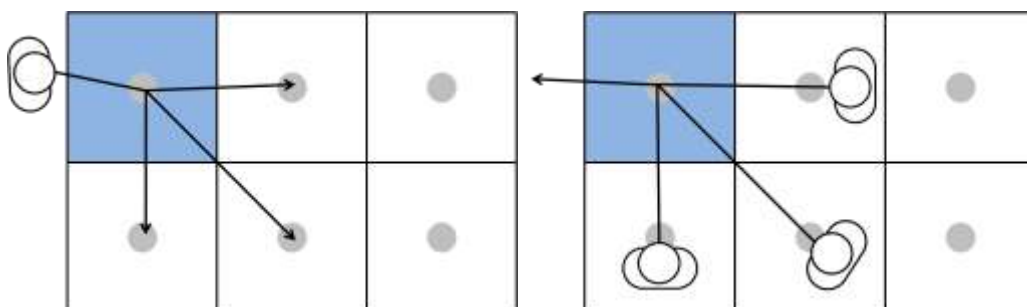


Figure 5-27 Room entry and leave events

6. Acquiring Sensor Data

6.1. Hardware Description

6.1.1. Generic Hardware

Focusing on the modeling process and development of sample applications it was decided to use capacitive proximity sensor boards available on the market. The chosen sensor is the CY3271 PSoC First Touch Starter Kit by Cypress Semiconductor [14]. It provides multifunctional, programmable sensor boards, based on the manufacturers own PSoC series of mixed-signal arrays. Those boards can be connected wireless to a PC using the Cypress' CyFi Low-Power RF solution. Interfacing on the computer is possible using a supplied DLL. Because this library lacks a public documentation a small collection of important classes and methods has been compiled and can be found in appendix A, together with a sample application. It was chosen due to low cost, low power usage and previous experience with PSoC devices.

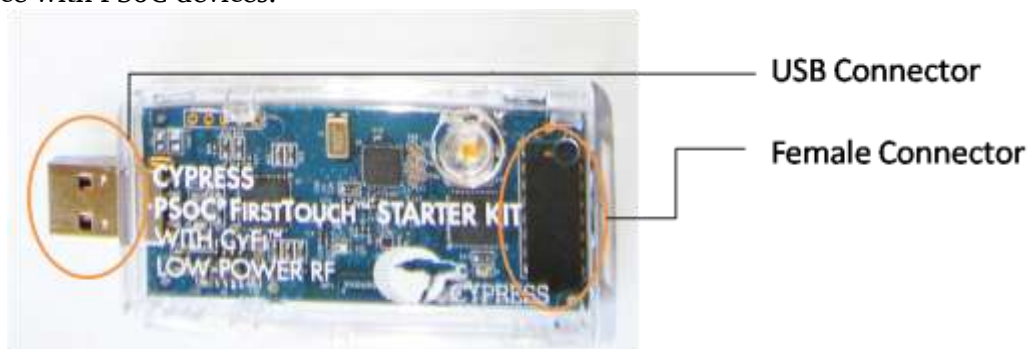


Figure 6-1 CY3271 Kit – PC bridge board

PC Bridge (FTPC)

This device has two primary functions. It is required to flash the other boards using a supplied software programmer. Furthermore it features a CyFi low-power RF-transceiver, acting as network hub, bridging data to the USB port.

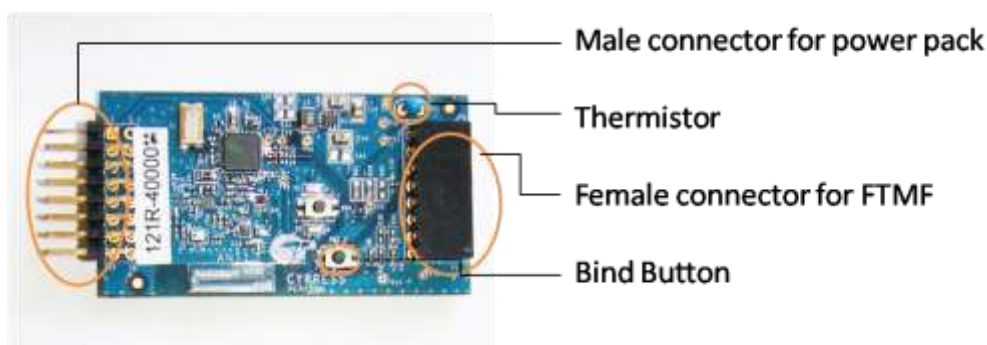


Figure 6-2 CY3271 Kit – RF expansion card

RF Expansion Card (FTRF)

The FTRF features a CyFi transceiver acting as wireless node when attached to one of the battery packs. It can either work standalone as temperature sensor, using an onboard thermistor or bridge sensor data acquired from attached boards.

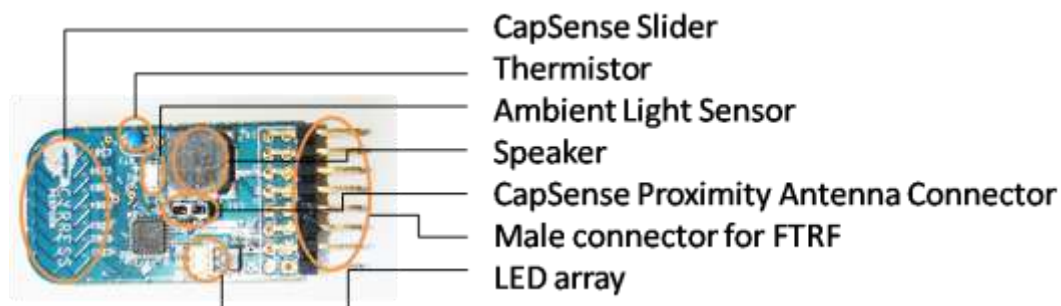


Figure 6-3 CY3271 Kit – Multifunction expansion card

Multifunction Expansion Card (FTMF)

This board features a PSoC device and several sensors and actuators for experimentation.

- 7-element CapSense slider measuring absolute position on the active area.
- CapSense proximity sensor registering grounded objects within range. Requires wire antenna.
- Thermistor for temperature measurement.
- Ambient light-level sensor able to determine illumination.
- LED-cluster featuring red, green or blue light.
- Simple speaker for audio signals.

Status data and measured properties can be forwarded to the FTRF or other devices using the I2C protocol. This project is using the CapSense proximity sensor using custom antennas, which is explained more detailed in the following section.

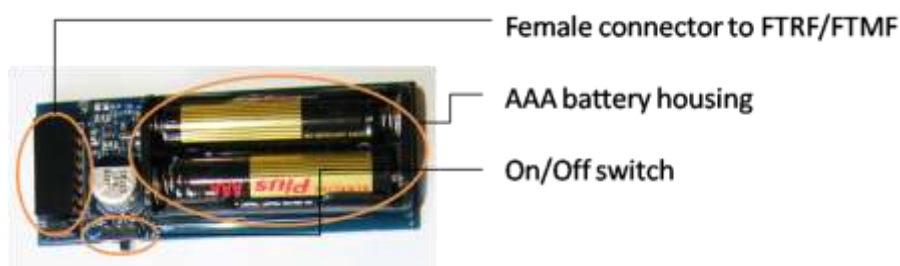


Figure 6-4 CY3271 Kit – AAA battery pack

AAA Power Pack

The power unit is housing two AAA batteries. It can be used to power the FTRF, the FTMF, or both in series.

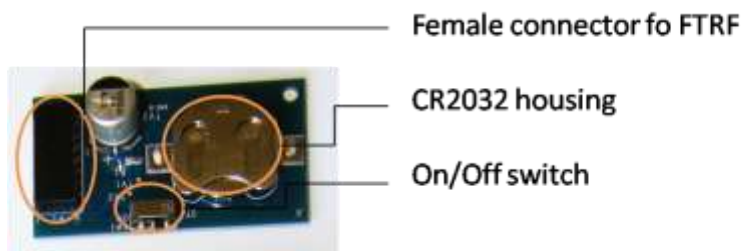


Figure 6-5 CY3271 Kit – CR2032 battery pack

CR2032 Power Pack

This battery housing is featuring a CR2032 coin cell battery. Its primary use is powering the FTRF for ultra low-power temperature measuring applications.

Setup Used

The project is using the setup depicted in Figure 6-6. Due to the focus on fast data submission, which causes comparatively high power consumption, the AAA battery pack is used. The FTRF is bridging the data acquired from the CapSense Proximity sensor on the FTRF to the PC Bridge. Antennas are typically coiled and approximately 100cm in length. In case of the Proximity Table they are fixed onto an acrylic glass tube, as shown below, in other setups they are fixed without stand.

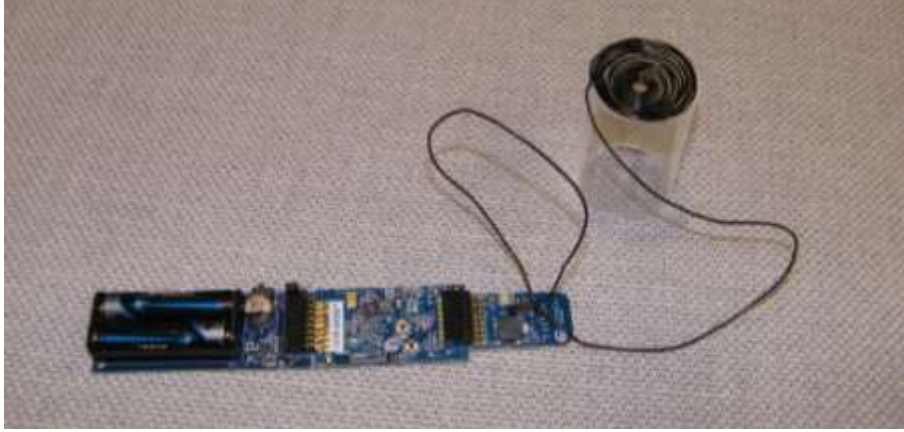


Figure 6-6 Typical sensor setup of this project

6.1.2. Antenna Design

Before determining parameters of the antenna attached to the CapSense proximity sensor it is necessary to discuss the objects the device is supposed to detect. As mentioned previously, this project distinguishes between deliberate interaction, which requires detection of the hand, and passive interaction, sensing the whole body. The first requires more sensitivity as the conductive surface is much smaller and consequently has a smaller effect on sensor capacitance. Therefore the antenna is designed for hand detection and considered good enough for body sensing in the scenarios described in 5. The average size parameters of a human hand are 18.90cm in length and 8.45cm in breadth for male subjects. Female hands are on average 17.22cm in length and 7.48cm in breadth [1]. Roughly approximated as plate capacitor, the following formula for the capacitance of a hand can be derived.

$$C = \epsilon_0 \frac{A}{d} \approx 8.85 \cdot 10^{-12} \frac{F}{m} \cdot \frac{0.18m \cdot 0.08m}{d} \approx 1.27pFm \frac{1}{d}$$

This approximation is useful to calculate the percentage increase of capacitance if a hand is approaching the sensor. If it is desired to sense a hand at a distance of 30cm the maximum mutual capacitance between antenna and hand is given in the following equation.

$$C_{max} = 1.27pFm \cdot \frac{1}{0.3m} \approx 4.23pF$$

However to register that capacitance a plate electrode exceeding the size of the hand would be necessary, which is not viable in many applications that require a good localization of the approaching object. The actual mutual capacitance largely depends on antenna geometry.

Though the used antenna has to be connected to the sensor using a wire its geometry is arbitrary. Plate conductors are working as well as insulated or uncovered wire. It was decided early on to use generic jumper wire of 0.5mm² profile. It is possible to rapidly modify size and shape, in order to test different setups. Disadvantages are less sensitivity compared to plate capacitors and potentially unreliable results if the wires are not fixed properly and change their geometry while in operation. However this

effect might also be desired, e.g. concerning the Bed input device that is generating its values using a combination of presence sensing and registering changes in wire geometry caused by pressure on the wires.

Conclusively a setup was chosen that is sufficiently suited for all applications. Wires of approximately 100cm length are coiled to a round shape, as shown previously in Figure 6-6. The wire is long enough to cause a capacitance that allows a good sensing resolution, close to the manufacturer provided resolution limit of approximately 25cm [13], while avoiding longer wires that cause an increased level of noise. Additionally this setup does expose the majority to the wire to the surface, which makes it susceptible towards geometry changes caused by applied pressure.

6.2. Acquisition Pipeline

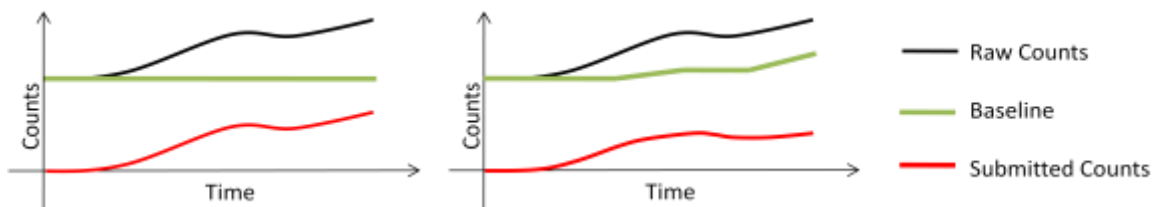
Acquiring physical properties using a sensor and its submission to a PC requires several steps that are depicted in the process pictured in Figure 6-7. The measure and ADC step occur within the sensor circuit using the sigma-delta modulator described in chapter 4.1.1. Data-pre-processing is done by the PSoC microcontroller on the FTMF, which applies algorithms described in the next section. Data transmission from sensor to PC includes several steps, detailed in chapter 6.2.2.



Figure 6-7 Physical property to usable data

6.2.1. Data Pre-Processing

The PSoC microcontroller on the FTMF is applying a combined algorithm to normalize the acquired raw-data and apply noise reduction [13].



Raw counts delivered from the bit stream integrator are used to calculate a baseline. Any further value is in respect to this baseline, thus instead of low-level data an absolute difference count is submitted. It is possible to either use a static baseline, based on the first raw count, or use a dynamically updated one, adapting to lasting changes in capacitance averaging several counts, which affects the output as shown in Figure 6-8. Selection is depending on the desired application, i.e. if a detectable object is remaining in the sensing area for longer periods of time, dynamic update should be disabled. In contrast, a system designed for short interaction profits from using the adapting baseline, as it is less susceptible to changes in capacitance caused by different humidity or temperature.

Update Algorithm Settings

There are several additional settings controlling the behavior of the update algorithm. Parameters chosen for this project are found in appendix B.

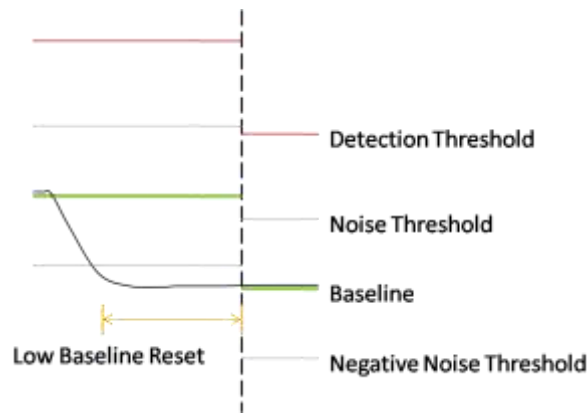


Figure 6-9 Low baseline reset

Noise Threshold determines a level above the baseline. Count values above this threshold do not update the baseline.

Baseline Update Threshold is a parameter regulating the speed of baseline reset. Positive count values below the *Noise Threshold* are accumulated until they exceed the *Baseline Update Threshold*, causing a baseline reset.

Negative Noise Threshold is a negative difference count value that is used to force an update of the baseline if an unusual amount of negative capacitance changes is detected, which might occur if a detectable object is in proximity to the sensor on start up and is being removed later on, as shown in Figure 6-9.

Low Baseline Reset is working together with *Negative Noise Threshold* and determines the time before a baseline update is forced.

Sensors Autoreset decides if a value has to be above *Noise Threshold* to trigger a baseline update, effectively limiting the time a sensor can be active.

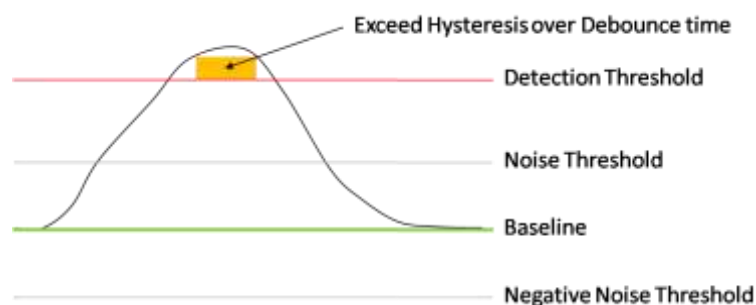


Figure 6-10 Triggering sensor with Hysteresis and Debounce

Detection Threshold can be used to use the proximity sensor as a trigger by sending a signal if the difference count is above this threshold.

Hysteresis is added or subtracted from *Detection Threshold* depending on current sensor status in order to add stickiness, to the trigger behavior. Stickiness indicates the tendency of the sensor to remain in its current state. In case of an inactive sensor the difference value needs to exceed *Detection Threshold*

plus *Hysteresis*. If it is active the submitted value needs to drop below *Detection Threshold* minus *Hysteresis*.

Debounce is determining the time necessary to switch sensor status according to the rules outlined in the *Hysteresis* description, visualized in Figure 6-10.

Scan Speed should be set to a slow speed, as this increases sensitivity, while remaining fast enough for real time usage.

Resolution does not need further tuning. Similar to *Scan Speed* the best available resolution is sufficiently fast for the desired purpose.

6.2.2. Data Transmission

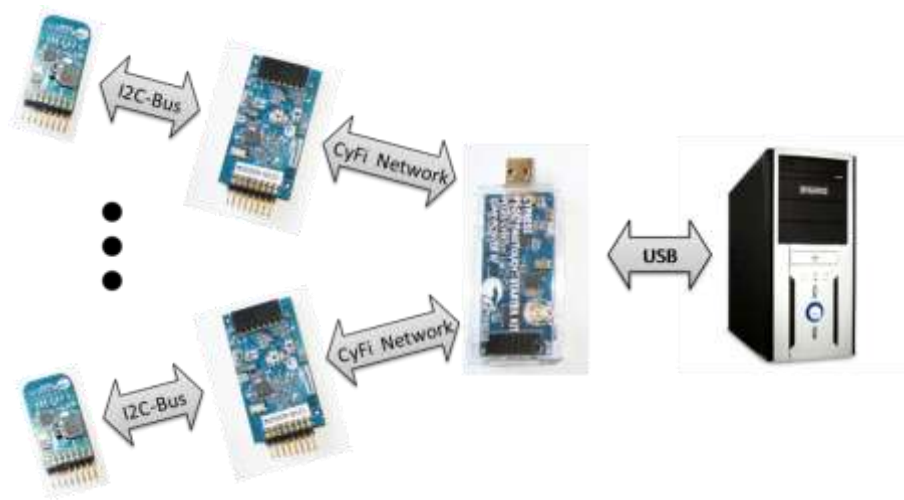


Figure 6-11 Data transmission network

Transferring the acquired sensor data to a PC is requiring several steps, pictured in Figure 6-11.

FTMF to FTRF

Communication between FTMF and FTRF occurs using the I2C bus [45], a bidirectional serial bus system designed for low-speed peripherals attached to other boards. It is controlled by the general purpose PSoC chips and does not require further settings. The used implementation is sending 14-byte data packages from the FTMF, using following structure.

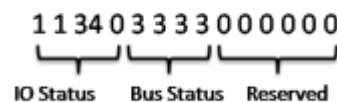


Figure 6-12 I2C Data Package

The four IO status bytes are of interest to further processing. Bytes one and four are status bytes for FTMF onboard LEDs. Byte two and three are the CapSense raw counts output, which is subsequently used for any further processing within the developed software.

FTRF to FTPC

Each FTRF is connecting to the FTPC using the CyFi wireless technology, a 2.4GHz ISM band protocol stack, resulting in a star topology network [15]. Unique IDs are assigned to each RF receiver, which need to be manually bound to the hub once. This two step bind procedure requires the FTPC to be set to bind mode, triggered by sending the appropriate command via software. The second step is pressing

the bind button on the FTRF. At predefined intervals data is polled from the FTMF through the I2C bus, wrapped into a network package together with additional data and sent to the FTPC. The structure of this package is analog to the *Message* class found in appendix A.1.

The designated polling interval of the supplied firmware is in the range between one and five seconds, which is too slow for responsive interaction. Minimum time for the CapSense proximity sensor to complete a measurement at maximum resolution and slow speed is 22ms [13]. Consequently it was attempted to decrease the polling interval by modifying the FTRF firmware closer, in order to better match the actual measurement time. It was possible to achieve data submission every 50ms with a single sensor. However, as soon as multiple sensors are bound, the FTPC is not able to process arriving data and freezes until it is manually disconnected. It seems that either the PSoC chip at its low frequency is not fast enough to handle that many packages in short timeframes or the supplied RAM is too small to buffer that much data. Polling rate was subsequently reduced to achieve a stable processing with six attached FTRF units, resulting in one package every 200ms. The modified firmware is found in appendix B.2.

FTPC to USB

Controlling the wireless network and communication to the PC is using two separate PSoC processors that are connected via I2C bus. The first chip is acting as wireless hub, buffers packages and forwards them to the other chip, which is bridging I2C to USB data and additionally provides programming functionality. Connected to a PC, the FTPC is recognized using the USB-HID [57] standard and therefore does not require specific drivers. However it is beneficial to use the factory supplied DLL for interfacing, which implements the USB-HID standard and provides simple control of the FTPC. This process is described more detailed in appendix A.1. It is possible to either poll data manually or use implemented event handlers that push the data as soon as it does arrive.

Data Conversion in Applications

As described above, CapSense raw counts are submitted using a two byte block of the I2C data packets. In order to use this data for subsequent calculations it needs to be converted into integer values. Those two bytes are basically 16-bit unsigned integer data, which allows transformation either using bit shift or the simple formula below.

$$I_{Res} = b_1 \cdot 256 + b_2$$

I_{Res} is the resulting unsigned integer value and b_x are the respective bytes in decimal representation.

7. Physical Design of Sample Input Devices

This chapter presents the physical prototype input devices that have been built using the modeled devices of chapter 5. Every prototype is installed in the Ambient Assisted Living laboratory of the Interactive Multimedia Appliances Department at the Fraunhofer IGD. This lab is the realistic representation of a three-room apartment featuring a sleeping room, a combined kitchen/living room area and an office section, showcasing ambient assisted living technologies.

7.1. Proximity Table

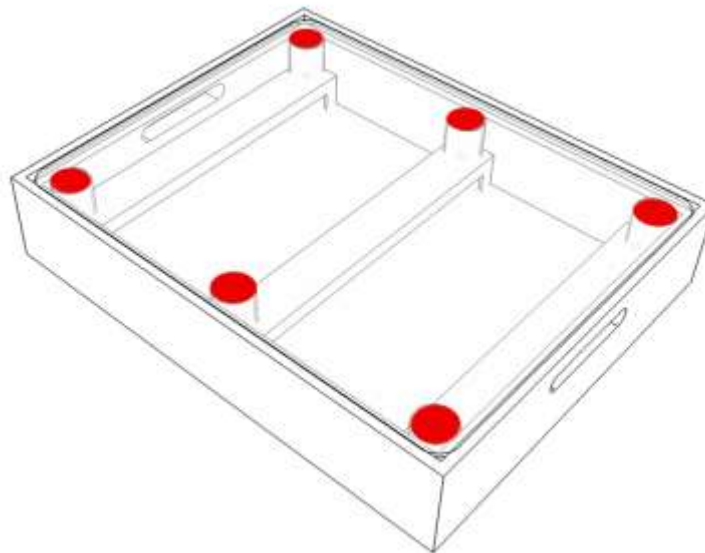


Figure 7-1 Rendering of a Proximity Table model

7.1.1. Concept & Design

Even though this input device is called Proximity Table, it is not restricted to applications for a table. It is designed as a proximity sensor array that can be unobtrusively located within furniture. It was decided to install the first prototype in a portable box for the following reasons:

- Changes in setup occur frequently. The prototype should allow easy maintenance.
- Sensor units are running on batteries that need to be replaced occasionally.
- The prototype needs to be movable for demonstrations in different locations.

Therefore the sensor array is placed inside a wooden box. The single sensor units can be hidden under wooden boards. Antenna holders made of acrylic glass tube are placed on those boards. For the purpose of better integrating the Proximity Table into the laboratory environment of a living room it has a white acrylic glass cover as well as white foil covering the visible wooden parts. The antenna wires are 100cm long. First 80cm are winded to a dense coil and placed on top of the acrylic tubes, so the majority of the antenna is close to the surface.

7.1.2. Dimensions

Proximity Table	
Overall dimension	540mm x 440mm x 105mm
Interior dimension	500mm x 400mm x 100mm
Active sensor area	500mm x 400mm

Table 7-1 Data of Proximity Table prototype

7.1.3. Images

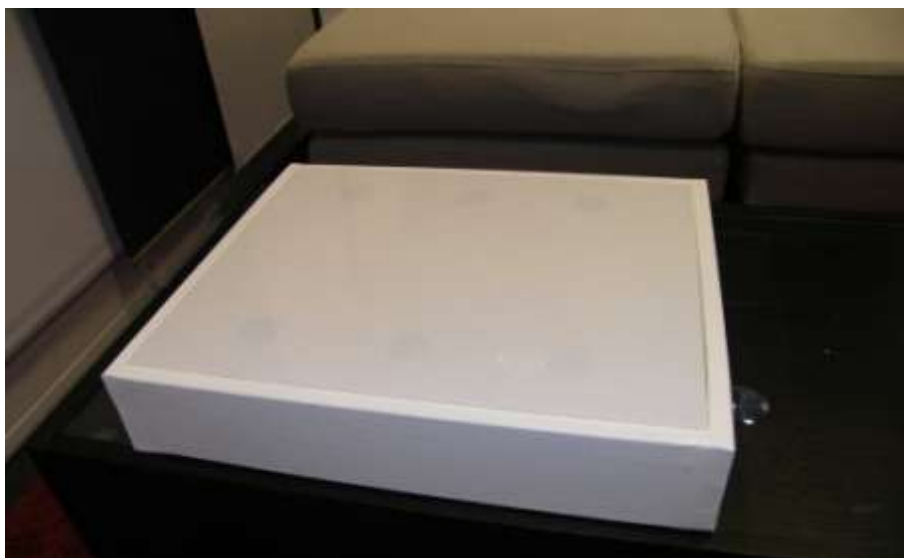


Figure 7-2 Proximity Table input device closed



Figure 7-3 Proximity Table input device open

7.2. Bed

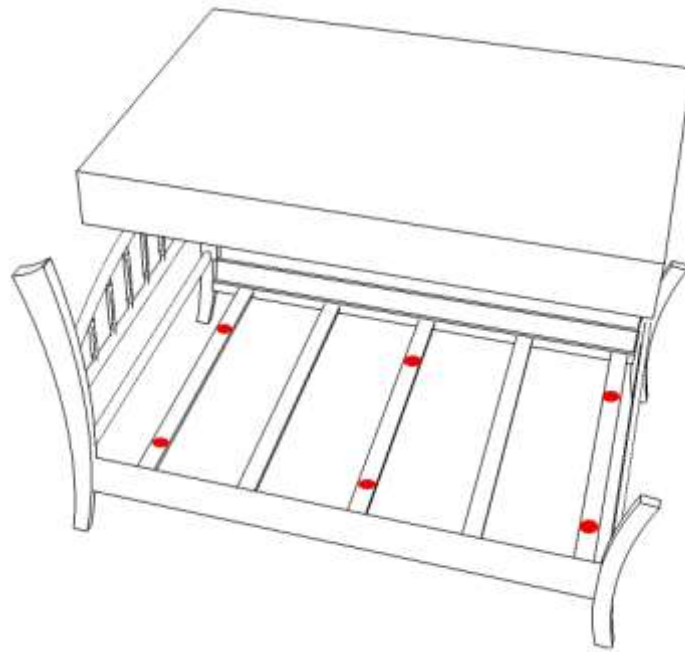


Figure 7-4 Rendering of a Bed model

7.2.1. Concept & Design

The Bed input device is designed for application to a regular full size bed. A prototype device using six sensor units was attached in the sleeping room of the laboratory. The antennas are made of 100cm coiled wire and being attached to the slatted frame below the mattress. All sensor units are placed on the floor below the Bed in order to be reachable easily. The associated software application is running on a television set interfaced by a PC on a nearby cupboard.

7.2.2. Images



Figure 7-5 Bed equipped with Bed input device



Figure 7-6 Antennas attached to slatted frame

7.3. Active Floor

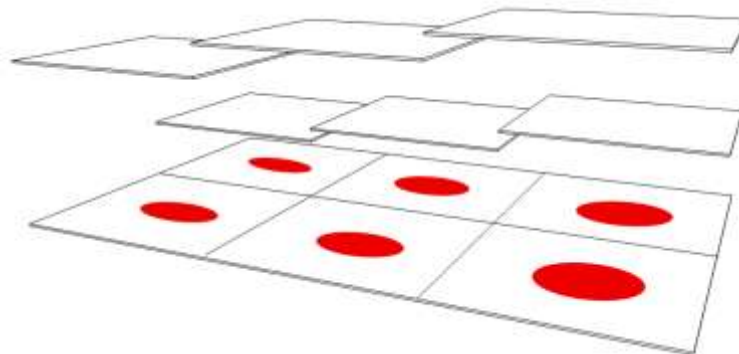


Figure 7-7 Rendering of Active Floor model

7.3.1. Concept & Design

The Active Floor is installed in the sleeping room of the laboratory. Six coiled antennas of 100cm length are attached centered below 100cm x 100cm carpet tiles. Sensors are hidden below bed and cupboard when active. Software is running on the same PC as the Bed application. Overall actively covered area is 300cm x 200cm.

7.3.2. Images



Figure 7-8 Active Floor separated in several tiles



Figure 7-9 Detailed view of floor tile with antenna in the center

8. Sample Applications

8.1. Developed Software

8.1.1. Requirements

Before development of the software started several requirements and features were collected. Those are noted in the following list:

- Interfacing with CyFi hardware and manipulate the data acquisition
- Modeling of the input devices in object-oriented paradigm
- On-the-fly modification of the input device data processing
- External storage of input device data processing parameters
- Logging of input events to a file format that can be easily processed
- Direct visualization of input device functionality
- Providing sample applications showcasing input device features and different input semantics

The software was developed using Microsoft (MS) Visual Studio 2008. It is a .NET 3.5 framework [38] application written in C# language. Any 3D visualization is using the XNA 3.0 framework [37]. In order to work properly following applications have to be installed in advance:

- MS .NET framework 3.5
- Cypress Sense and Control Dashboard. Including drivers for the FTPC. Included on DVD accompanying the hardware.
- MS XNA Framework Redistributable 3.0. Enables using XNA applications.

Usage of the application is restricted to newer MS Windows operating systems (XP, Vista, 7).

8.2. Structure

8.2.1. GUI

The application is based on WinForms window management system. A tree of all included windows is shown in Figure 8-1. A detailed description of all Forms is following in the following section and the chapters associated to each input device.

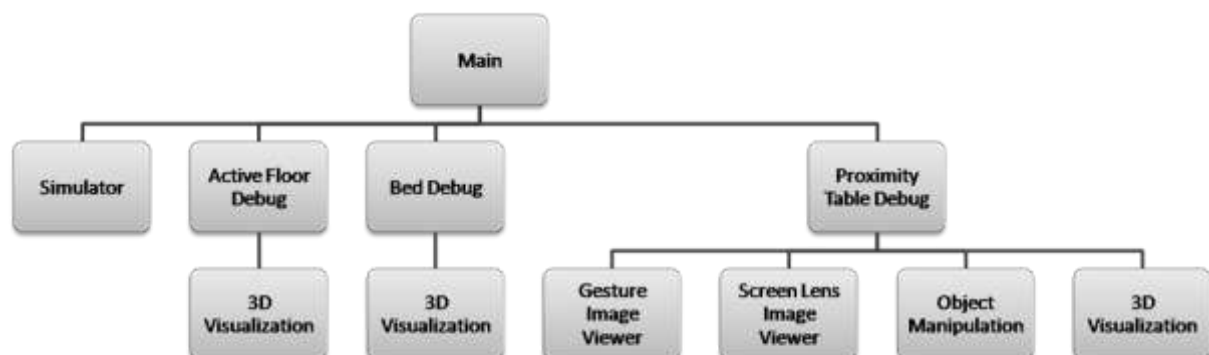


Figure 8-1 GUI structure

8.2.2. Class Structure

In addition to the classes associated to each Form there are numerous other classes that can be assigned to different categories.

General Classes

This category contains all classes that are in use by several other objects.

- *Program*: The main class instantiating the program.
- *Logger*: Provides generic functionality to write data to a log file.
- *Settings*: Structure storing all values acquired from settings file.

Data Acquisition

All classes that are involved in the data acquisition process are listed below.

- *CyFiHub*: Interface to FTTPC. Provides access to acquired data and node IDs.
- *Sensor*: Generic sensor object. Stores a given number of acquired values.
- *FloorSensor*, *ProximitySensor*, *SimSensor*: Derived from *Sensor*, are containing additional information, according to the modeling of 5. The data fusion structure is a list of those objects for every input device.

Data Processing

This list contains the classes implementing the virtual input devices of the modeling process, described in more detail in the appropriate sections.

- *ActiveFloor*
- *Bed*
- *ProximityTable*

Output Structure

Some output structures of the modeled input devices require their own class, listed below.

- *BedPerson*: Contains information about position, orientation, lying status and spine strain
- *FloorPerson*: Contains information about position and status.

Application data processing

Several applications implement additional data processing in order to generate their output.

- *GestureAnalyzer*: Required for finding gestures in subsequent points that are detected by the Proximity Table. Used in some of the demo applications described in chapter 8.5.
 - *Trajectory*: Helper class for gesture analyzing, containing of a three-dimensional vector and its length.
- *AppSwitch*: Used by Proximity Table applications in order to switch between applications without the need for another input device.

Visualization

Combining the WinForms and XNA framework requires several helper classes, necessary to render within a form.

- *ServiceContainer*: Required to use the XNA content manager, handling textures, fonts and 3D models.
- *GraphicsDeviceServices*: Helper class managing the XNA *GraphicsDevice* within a WinForms environment.
- *GraphicsDeviceControl*: Control allowing the use of XNA methods for rendering.
 - *activeFloorVisualizationControl*, *bedVisualizationControl*, *imageWindowControl*, *manipulationWindowControl*, *proximityTableVisualizationControl*, *screenLensWindowControl*, *simulatorControl*: Individual controls derived from *GraphicsDeviceControl* rendering different content.

8.2.3. Functional Structure

The generic functionality is similar for all available modes. Any new data arriving from the FTFC is raising an event, triggering an update of the associated data structure. This data structure is then passed to the appropriate data processing and transformed to the output structure, as described earlier in 5. Associated applications access the output structure and apply further processing before updating the screen content. The process and associated components are displayed in Figure 8-2.

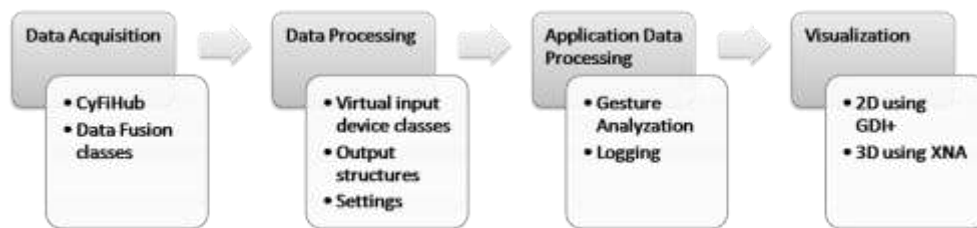


Figure 8-2 Functional processing structure

A detailed description of those steps concerning the actual devices and applications is given in the following sections of this chapter.

8.3. Generic Interface

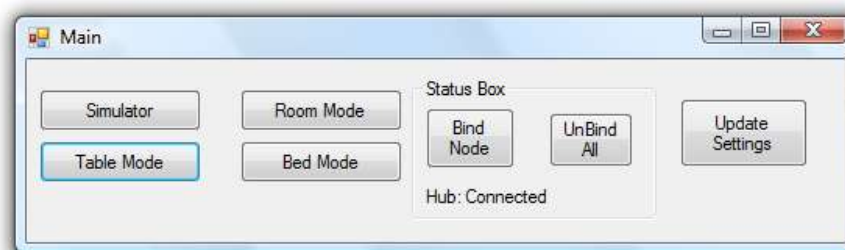


Figure 8-3 GUI Main window

The Main form (Figure 8-3) is the only window showing after the application is started. It allows launching the Simulator and the different input device modes, explained in more detail later in this chapter. The buttons in the status box handle communication with the FTFC. *Bind Node* is initiating the bind mode of the FTFC, which was described earlier in chapter 6.2.2. *UnBind All* can be used to

remove bindings from all nodes simultaneously. Finally *Update Settings* is reading current settings from the settings file.

Event Logging

The event logger is opening a stream to a single file, in order to store events generated by the Bed and Active Floor input devices. A CSV (comma separated value) file format was selected using a separator compatible to MS Excel. The generic structure is shown below in Table 8-1. All events are converted to strings. Any registered event is causing another line to be written in the log file.

Column 1	Column 2	Column 3	Column 4	Column 5	Column 6	Column 7	Column 8
<i>TimeStamp</i>	<i>Sensor 1 Value</i>	<i>Sensor 2 Value</i>	<i>Sensor 3 Value</i>	<i>Sensor 4 Value</i>	<i>Sensor 5 Value</i>	<i>Sensor 6 Value</i>	<i>Event String</i>

Table 8-1 Table structure of event logging CSV file

Settings File

It was decided to use an external file containing settings, in order to store input device and application settings gained through experimentation without modifying the source code itself. Most GUI configurable settings are read from a settings.txt file located in the application directory. This setting file is read initially on program startup and can be updated on the Main screen. A typical example can be found in appendix D.1.

8.4. Simulator

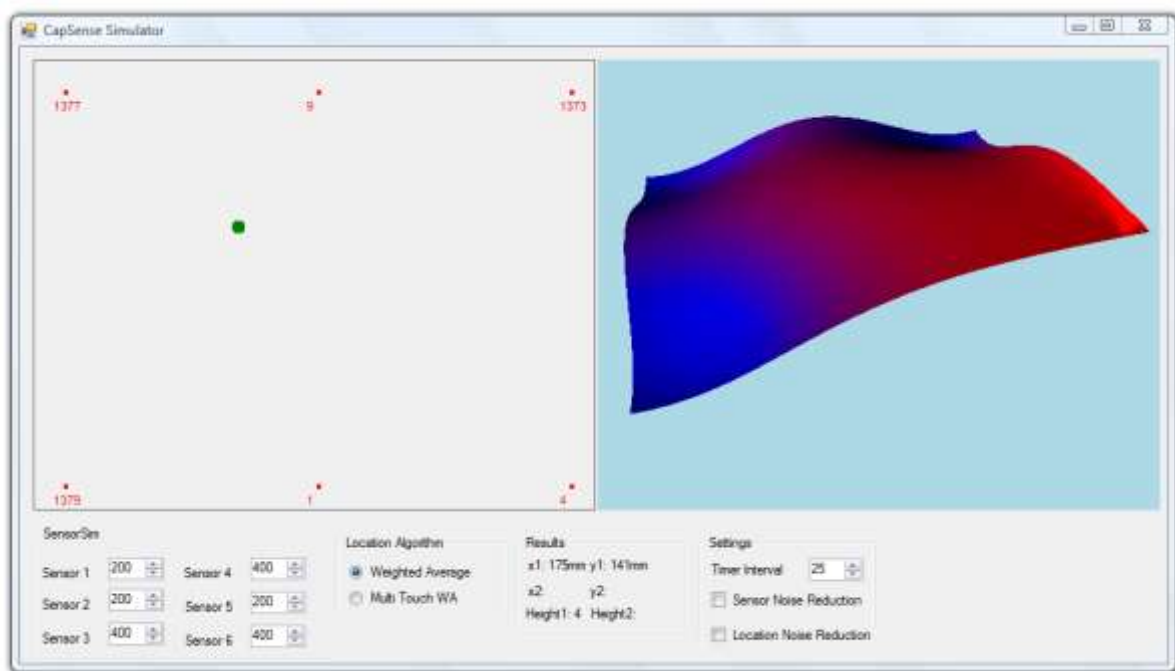


Figure 8-4 GUI Simulator mode

Simulator mode, shown in Figure 8-4, was implemented early in the design process, in order to test several algorithms using artificially created sensor values. It is instantiating six different sensors, each of which is assigned a distance value that is transformed to a sensor value using the following transfer function. The calculated sensor value is v , d the set distance, d_{max} the associated maximum distance the sensor can register and r a small positive random number associated to the noise.

$$v = \begin{cases} 0 + r, & d > d_{max} \\ \left(\frac{d_{max} - d}{18}\right)^3 + r, & d \leq d_{max} \end{cases}$$

This equation is a rough approximation of the non-linear transfer function of a real sensor, without taking any disturbing factors or geometry changes into account. Sensor values are transformed into position and height data, similar to the Proximity Table input device. Implemented data processing algorithms are weighted average and a multi-object detection method. Additionally it is possible to apply noise reducing interpolation algorithms either at sensor value level or to the generated locations. The graphic control on the left shows the location of each sensor together with labels displaying current data value, as well as detected points as green circles, the size of which are indicating the detected height. The 3D visualization on the right side is visualizing all sensor values directly by assigning height coordinates to a two-dimensional triangle mesh. Depending on its current value a sensor is influencing the neighboring region, using a Gaussian distribution function to modify the height data. The resulting overall height data of a vertex at mesh position $h(i,j)$ is calculated using the formula below. Current sensor value is v_u and n the number of sensors. Positive integer coordinates of mesh and sensors are transformed to a function range $[-5,5]$, in order to properly apply the distribution function. x and y are the transformed mesh coordinates; x_s and y_s the transformed sensor coordinates.

$$h(i,j) = \sum_{u=1}^n \frac{v_u}{a \cdot \pi} \cdot e^{\left(-0.5 \cdot \frac{(x-x_s)^2 + (y-y_s)^2}{b}\right)}$$

a and b are modifiers to clamp the resulting values into ranges that can be visualized decently. The resulting height value is further accentuated by using a color gradient from red to blue.

8.5. Proximity Table

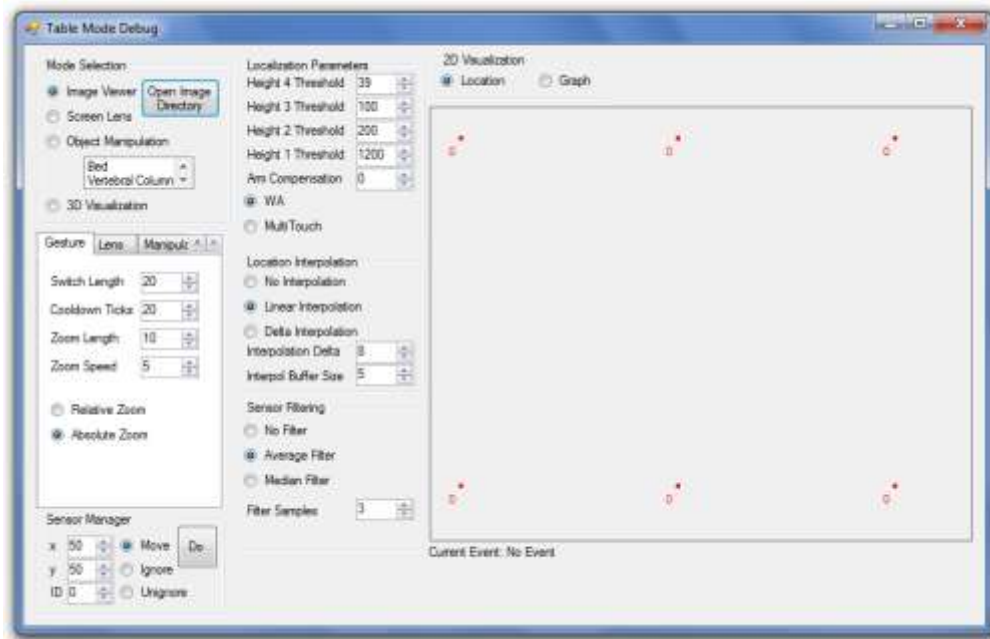


Figure 8-5 GUI Table mode debug

The table mode debug screen shown in Figure 8-5 is the GUI form controlling the data processing and sample applications for the Proximity Table. Located on the right side are a two-dimensional visualization, analog to Simulator mode, and a label displaying events detected in Image Viewer and 3D Object Manipulation modes. Additionally the 2D visualization supports sensor movement using

drag and drop, as well as displaying a history of sensor values in a graph. Controls on the left side are arranged into six groups. *Mode Selection* on the far left is used to launch the demonstration applications, as well as a 3D visualization. Below is a tabbed group setting parameters for each application. The *Sensor Manager* group can be used to relocate sensors and set their activity status. The *Localization Parameters* group is controlling localization algorithm of the data processing, allowing to select weighted average or multi-object recognition. *Height x Threshold* are the four fixed points for height interpolation, while *Arm Compensation* is intended to improve the localization if the Proximity Table is put on a flat surface and the user sitting in front of it. In that case it is likely that the arm of the user is within the active region of the device and will disturb the localization of the hand. The *Location Interpolation* group is setting parameters for the various location interpolation algorithms, described in chapter 5.4.2, with *Interpolation Delta* being the threshold for differential position interpolation and *Interpol Buffer Size* being the number of samples for average position interpolation. Finally the *Sensor Filtering* group allows setting the filtering algorithm and number of samples to reduce the noise of individual sensors.

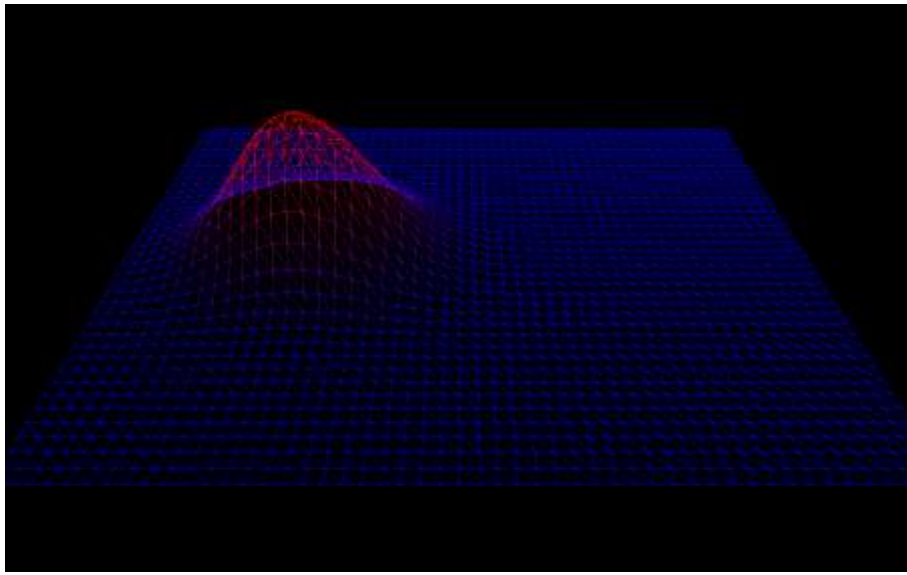


Figure 8-6 3D visualization of Proximity Table mode

The 3D visualization, shown in Figure 8-6, is available from the *Mode Selection* menu. Calculation is similar to 3D visualization of Simulator mode. Instead of single sensor positions and values the location of detected objects is used as center of the Gaussian distribution. Height is mapped to a color gradient and the deviation of the distribution. In this case n is the number of detected points, h_{max} the maximum detectable height, h the calculated height and x_p and y_p the two-dimensional location of a detected point. Additionally a is a constant to scale the visualization and d a height-dependent deviation, scaled using the constants b and c .

$$h(i, j) = \sum_{u=1}^n \frac{h_{max} - h}{a \cdot \pi} \cdot e^{\left(-0.5 \cdot \frac{(x-x_p)^2 + (y-y_p)^2}{d}\right)}$$

$$d = d_{max} - [b \cdot (h_{max} - h)]^c$$

Application Switch

The application switch is an expanding/collapsing menu present in all demo applications, intended to switch between the different apps at run-time without using an additional input device. It was decided to use a collapsible menu to maximize the interaction region and avoid cluttering the display. If an object is detected on the right border of the Proximity Table the menu is expanding and the user can

select the desired application. The application switch does implement a point-and-click paradigm, dividing the screen and menu into several active regions. If the hand stays in any region longer than a given time a click event is triggered.

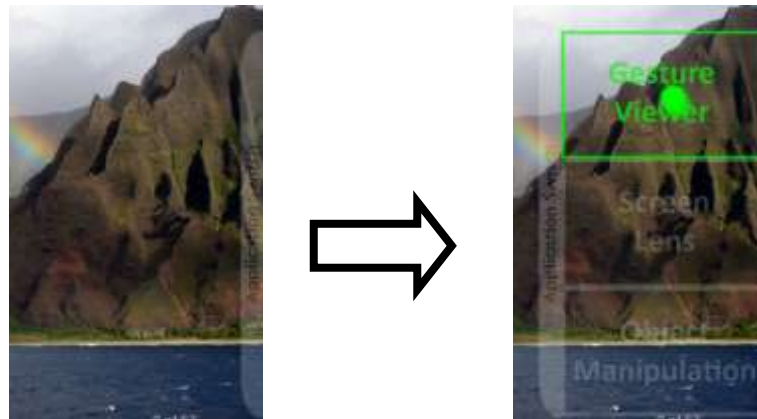


Figure 8-7 Application switch collapsed and expanded

The debug window tab associated with the application switch allows setting the timing of menu appearance, disappearance and the time threshold required to trigger the mentioned click event.

8.5.1. Gesture Image Viewer



Figure 8-8 GUI of Gesture Image Viewer

The idea behind the *Gesture Image Viewer* is to implement another abstraction layer between input device and application. A software module is interpreting detected locations into simple gestures that trigger different input events. It is intended to showcase functionality similar to other gesture controlled applications, e.g. using finger or mouse gestures.

The *Gesture Image Viewer* is a sample application that is using simple gestures, allowing the user to control an image slideshow. Upon launch the user may select a folder containing images. Supported formats are JPEG, PNG and BMP. Images are dynamically stored as textures and rendered using XNA 2D drawing functions. Zooming is emulated by increasing the drawing region outside of current screen size. Additionally the last ten detected points are drawn in decreasing opacity to give the user visual feedback about his latest interaction. In the lower right corner of the screen, the number of images in the folder and the index of the current image are displayed.

Gesture Recognition

The gesture analyzer module is examining the movement of a user's hand within the interaction area of the Proximity Table. In this application the supported gestures are rather simple, depending only on direction and speed of movement. As soon as a hand is entering the detection area several locations are stored in order to determine a direction d , which is calculated using the quotient of differences between subsequent locations, Δx and Δy .

$$d = \frac{\Delta x}{\Delta y}$$

Four different directions are distinguished.

- Positive x direction. Requires a high positive d quotient.
- Negative x direction. Requires a high negative d quotient.
- Positive y direction. Requires a positive d quotient smaller than 1.0.
- Negative y direction. Requires a negative d quotient higher than -1.0.

Speed in two directions is calculated using the following equations, accumulating absolute differences in subsequent locations.

$$s_x = \sum_{i=1}^n |\Delta x_i| \quad , \quad s_y = \sum_{i=1}^n |\Delta y_i|$$

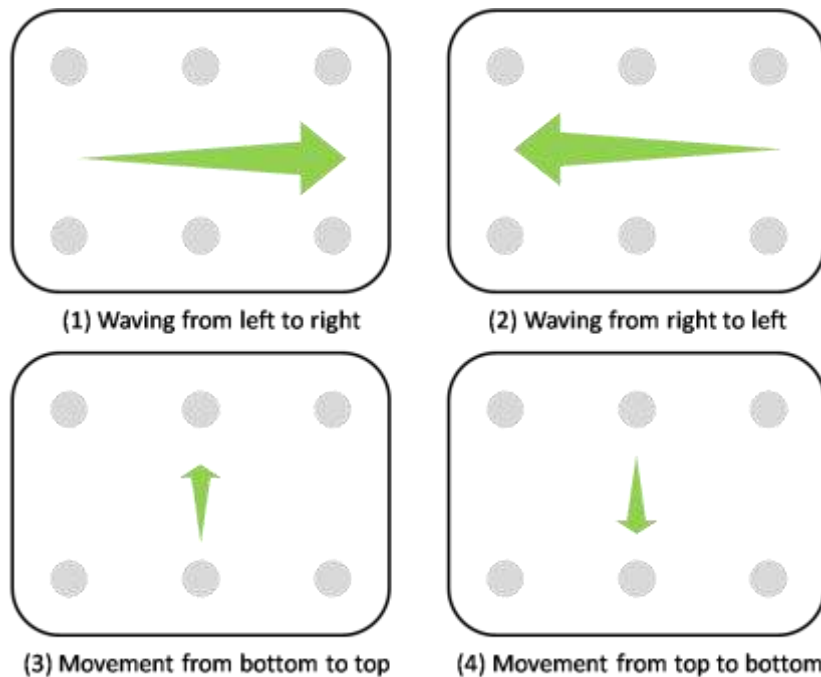


Figure 8-9 Four Gesture Image Viewer gestures

The hand wave from left to right (Figure 8-9(1)) requires movement in positive x direction, while exceeding a given speed s_x . Likewise the wave from right to left (Figure 8-9(2)) is causing an event as soon as the movement in negative x direction exceeds the same threshold. The next gesture is a movement in positive y direction (Figure 8-9(3)). The speed threshold s_y is much smaller than those of the previous gestures as movement speed is directly mapped to an event. Accordingly the fourth gesture is triggering an event if the hand moves in negative y direction with reasonably high s_y (Figure 8-9(4)).

Event Mapping

Any of the four detectable gestures is causing a distinct application action.

- The left to right wave is loading and displaying the next image
- The right to left wave is loading and displaying the previous image
- Movement from top to bottom is zooming out of the image
- Movement from bottom to top is zooming into the image

Zooming is supporting two different selectable modes. The first is directly mapping the current y location to the zoom factor, while the second is increasing/decreasing zoom depending on direction and speed of the movement.

8.5.2. Screen Lens



Figure 8-10 GUI of Screen Lens Image Viewer

Another sample application is the *Screen Lens Image Viewer*, which is implementing regional zoom functionality into an image slideshow. Folder selection and format support are identical to *Gesture Image Viewer*. The idea behind this application is to integrate an input modality that is directly mapping detected locations into application events; in this case, position and strength of the integrated zooming.

Similar to the previous application, images are loaded into textures. The zoom function is using a custom pixel shader to displace pixels within a circular region. Position and height of a detected hand are scaled and parsed to this shader dynamically, determining the center of the zoom area and strength of displacement. Additionally, the current aspect ratio of the image is submitted in order to scale the zoom region to a circle, which is necessary due to quadratic texture coordinate mapping on non-quadratic screens. A full listing of the shader code is found in appendix D.2. The displacement function applied to each pixel is the following:

$$x_{new} = x_{old} - d_x, \quad d_x = \begin{cases} s \cdot (x_{old} - c_x) \cdot r, & x_{old} - c_x \leq n \\ 0, & x_{old} - c_x > n \end{cases}$$
$$y_{new} = y_{old} - d_y, \quad d_y = \begin{cases} s \cdot (y_{old} - c_y), & y_{old} - c_y \leq n \cdot r \\ 0, & y_{old} - c_y > n \cdot r \end{cases}$$

Pixel coordinates in texture space are x and y , d is the displacement, s the displacement strength, c the center of zoom in texture space, r the aspect ratio and n a distance norm.



Figure 8-11 Comparison between high zoom factor (left) and low zoom factor (right)

In order to change images, two active regions have been implemented, chosen to avoid complications with the application switch active region. Similar to the latter a certain configurable staying time is required to cause an event.

- Next image is triggered by staying in close proximity to the upper right side of the Proximity Table
- Previous image is triggered by staying in close proximity to the upper middle part of the Proximity Table

Another optional feature of this application is the *Sticky Lens*. In order to avoid tiring of the arm when zooming through the image, position and zoom factor will get fixed if the location is not changing anymore. This change is determined using a differential threshold, similar to the differential interpolation, as described in chapter 5.4.2. The user may then remove his hand and watch the image with the fixed zoom region. In order to move this fixed lens, the hand has to be brought into the detection area of the Proximity Table again.

8.5.3. 3D Object Manipulation

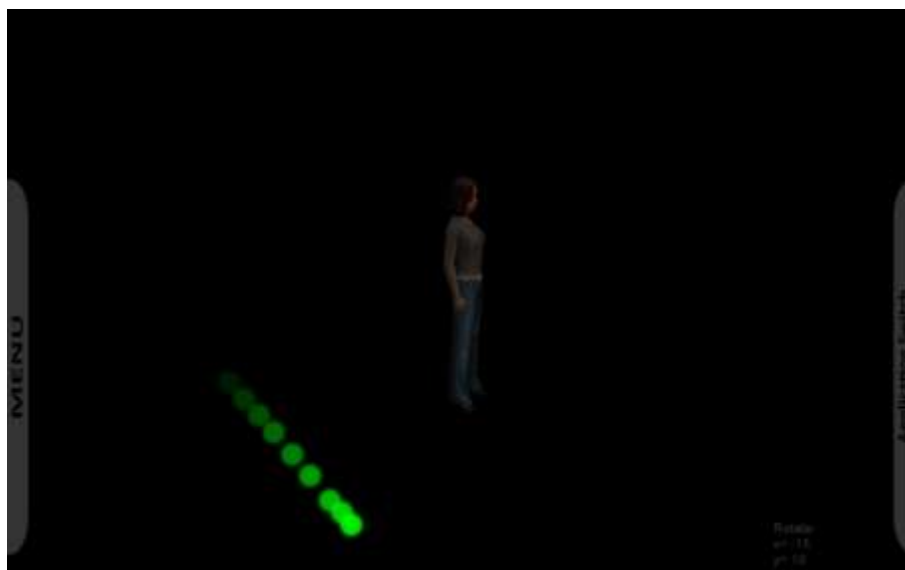


Figure 8-12 GUI of 3D Object Manipulation

The *3D Object Manipulation* is the last demonstration of the Proximity Table. It allows the manipulation of three-dimensional objects similar to regular model viewers. The user is able to perform the standard actions rotate, pan and zoom. This application implements an interaction, based on direction and speed of hand movement. Using those parameters a trajectory is calculated and in turn mapped directly to an application event.

A set of models is precompiled within the project and can be freely selected upon start of the *3D Object Manipulation*. Additionally it is possible to switch models within the application using a menu item. Selection between the manipulation modes and run-time model switching are possible via a menu similar to the application switch, activated if an object is detected in the active region on the left side of the screen.

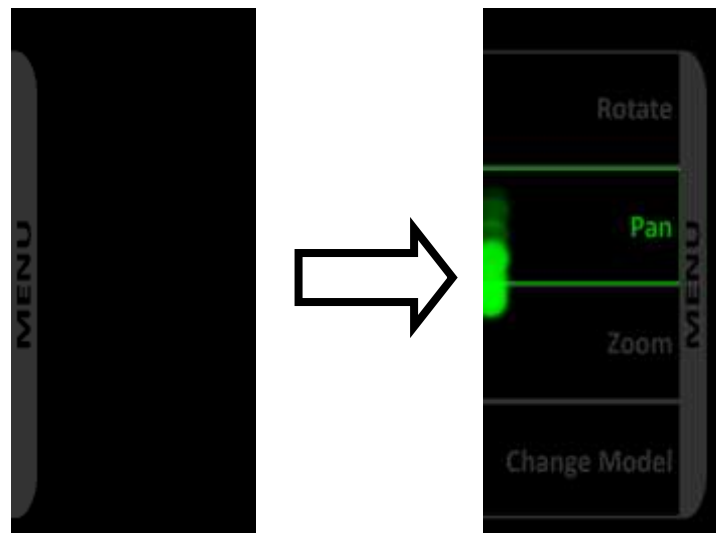


Figure 8-13 3D Object Manipulation menu collapsed and expanded

Menu appearance time and required time for selecting items is again configurable using the GUI or settings file.

Trajectory Calculation

It is desired to achieve a direct and precise manipulation of the model, depending on the strength of the user's interaction. Regarding rotation and pan mode the selected method is converting subsequent detected locations into trajectories that are directly applied to the model using only linear scaling and transformation.

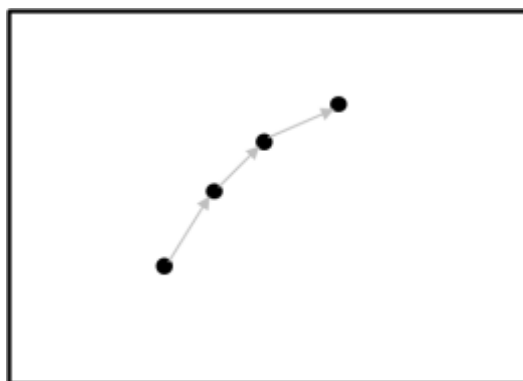


Figure 8-14 Trajectory calculation from detected points

Trajectories are stored as three-dimensional vectors storing direction and length, calculated using two consecutive locations.

As any movement within the interaction area is directly applied to the model a certain prediction of the user's intention is required, in this case based on three assumptions for natural interaction.

- The typical interaction between user and input device is generally taking several seconds
- If a very fast movement is detected it is very likely the user wants to leave the interaction area
- If the detected positions are close to the border of the interactive area, fast movement in outside direction may be considered as intention to leave the interaction area

The first assumption is used to differentiate between false object detection caused by sensor noise and a user interaction. Occasionally if several sensors suffer a high positive noise it is possible that this is registered as an object. However those noise spikes are generally short compared to a deliberate interaction. Therefore any object that remains in the interaction area for a very short time is ignored and does not lead to trajectory generation, resulting in the return of a vector with length 0. This time threshold is configurable by menu and settings file.

Trajectory length and scaling to rotation/pan are chosen in a way that allows fast model manipulation with moderate hand movement. Therefore if the trajectories of fast movements are directly mapped they lead to exaggerated, hardly controllable model movement. Accordingly a maximum trajectory length is defined and any exceeding vector is ignored.

Finally the third assumption is dealing with the user's intention to leave the interaction area in any direction, most notably vertical to the Proximity Table surface. In order to predict this behavior, a scaled version of the current trajectory is added to the detected location. If the resulting position is outside of the interaction area, no trajectory is returned, as the interaction is considered finished. The scaling factor of the trajectory is configurable.

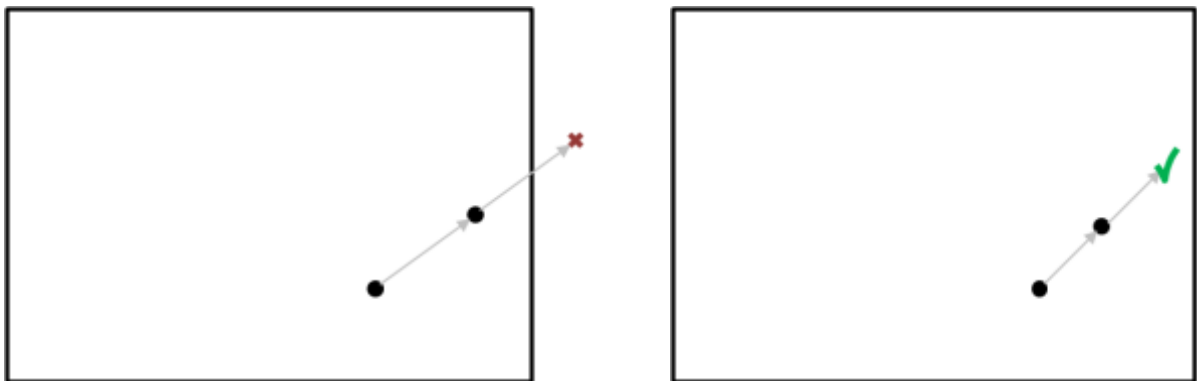


Figure 8-15 Leaving intention prediction

Conclusively, those assumptions are combined in a trajectory prediction method, following the pseudo-code below:

```
if(locationDetected)
{
    repeats++;
}
if(repeats > threshold)
{
    trajectory = detectedLocation - lastPoint;
    futurePoint = detectedLocation + scale * trajectory;
    lastPoint = detectedLocation;
    if(trajectory.Length > maximumLength) return 0;
    if(outOfBoundary(futurePoint) return 0;
    return trajectory;
}
```

Object Manipulation

Pan mode is directly taking the acquired trajectory to calculate the new model position. The trajectory components in x and y direction are t_x and t_y , s_p is a scaling factor and l_t the trajectory length.

$$\begin{aligned}x &= x_{old} + s_p \cdot t_x \cdot l_t \\y &= y_{old} + s_p \cdot t_y \cdot l_t\end{aligned}$$

Rotation mode is using a vector vertical to the trajectory as rotation axis and a rotation value depending on trajectory length. Only rotation around an axis in the xy -plane is supported. The rotation values are r_x and r_y , with s_r being another scaling factor.

$$\begin{aligned}r_x &= r_{x,old} + s_r \cdot t_y \cdot l_t \\r_y &= r_{y,old} + s_r \cdot t_x \cdot l_t\end{aligned}$$

Zoom mode does not require a trajectory. It is using a zooming implementation similar to *Gesture Image Viewer*, based on region and proximity to the table. A variation is caused by the menu on the left side, which requires the table to be divided into four regions, as shown in Figure 8-16.



Figure 8-16 3D Object Manipulation regions in zoom mode

8.6. Bed

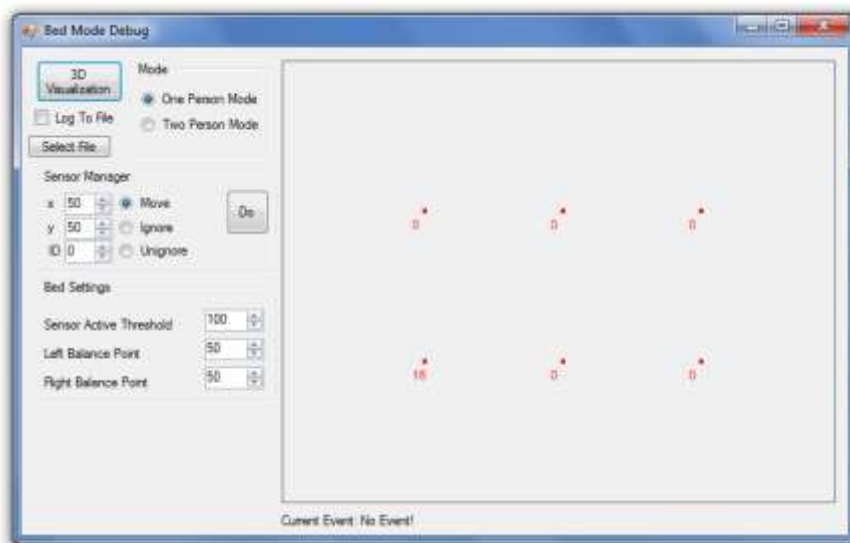


Figure 8-17 GUI Bed mode debug

The GUI element controlling sample applications and settings of the Bed input device is the Bed mode debug window of Figure 8-17. On the left side it is possible to start the visualization and enable event logging to a selected file. The setting between one and two person detection, mentioned in chapter 5.5.3, is available through radio buttons. Sensors can either be moved using the menu or drag and drop similar to Proximity Table debug. All settings controlling the calculation are modified in the *Bed Settings* group. Sensor Active Threshold, Left Balance Points and Right Balance Point are the parameters defined within the Bed modeling chapter, referred to above. The right side consists of the 2D visualization displaying current sensor values and an event label showing any occurring event.

8.6.1. Position and Posture Detection

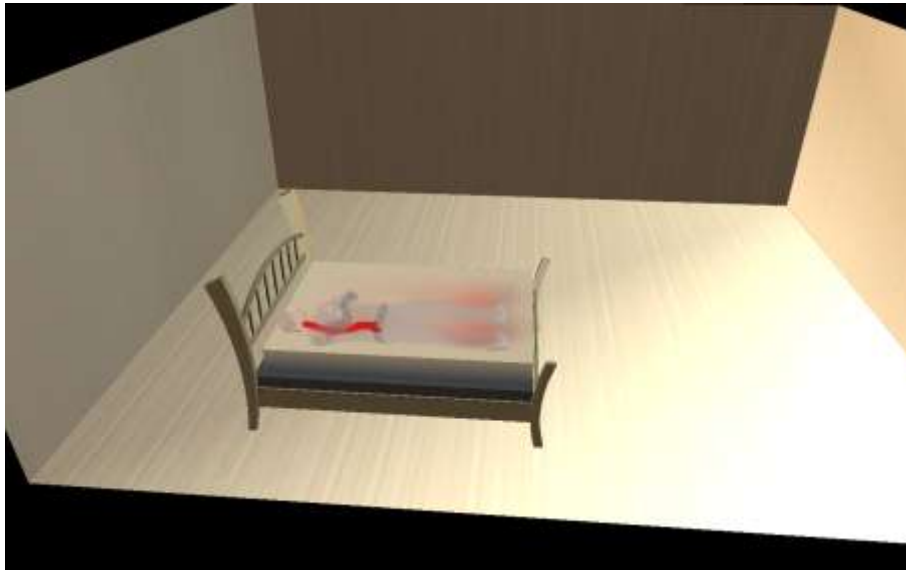


Figure 8-18 3D visualization of position and posture detection

The Position and Posture Detection of Figure 8-18 is the demonstration application for the Bed. It is the three-dimensional rendering of a room and a bed. On this bed sensor values, detected persons including their position and posture, as well as spine stress are visualized. It is possible to modify the viewport using the keyboard. Keys Q and A control rotation around the x-axis, keys W and S rotation around the y-axis, keys E and D rotation around z-axis and keys O and L are used for zooming.

All sensor values are displayed directly using a mapping to a Gaussian distribution and color gradient seen in *Simulator* mode, albeit using different scaling, gradients and colors. They visualize the current pressure on the mattress, as shown in Figure 8-19.

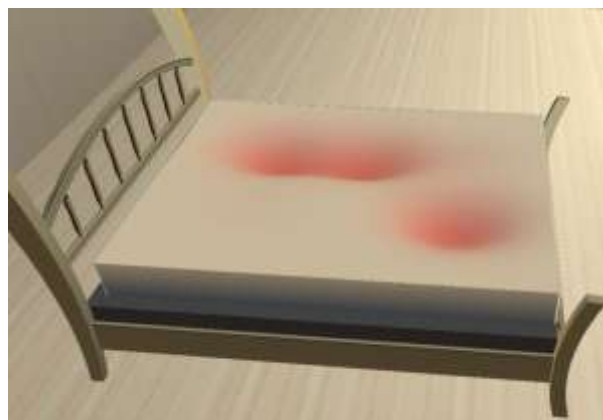


Figure 8-19 Mattress pressure visualization

Any detected person is displayed translucently at the detected position, using the determined posture. The spine is rendered into the bodies using red color to indicate bad spine stress and green color to indicate good spine stress, as shown in Figure 8-20.

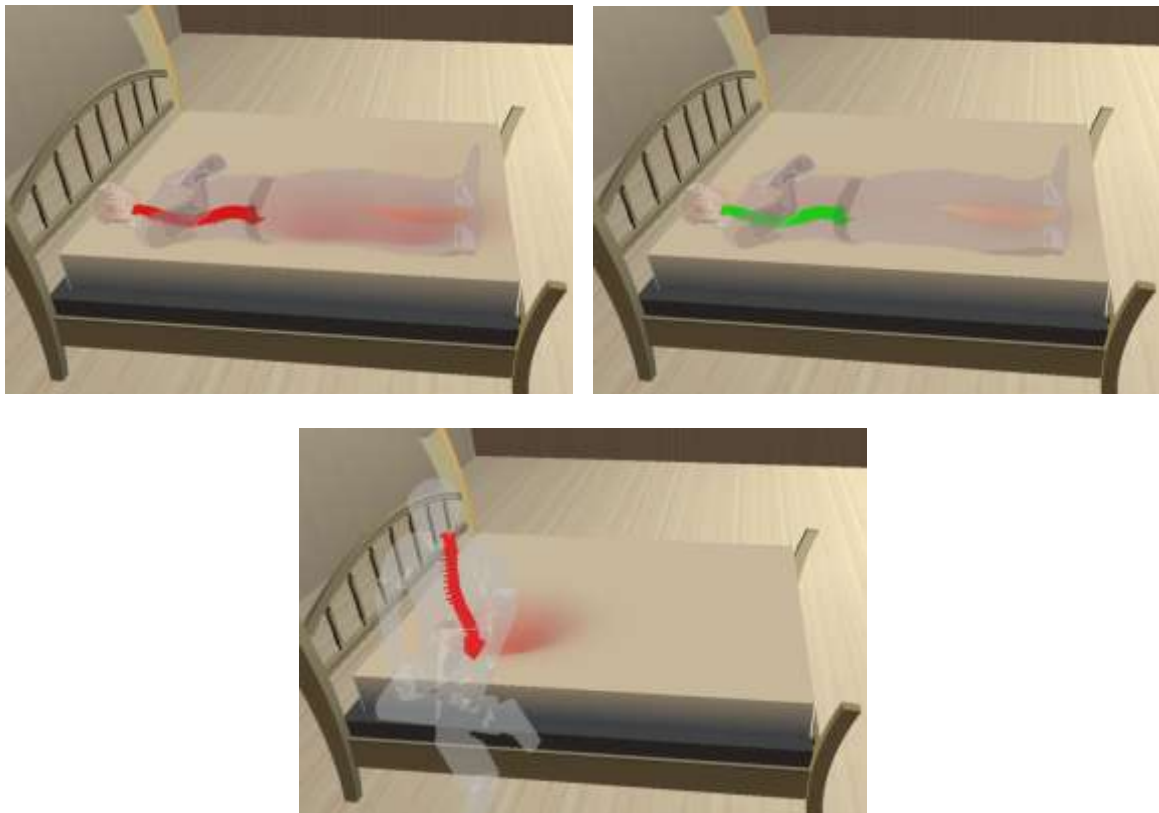


Figure 8-20 Visualization of different postures and spine stress

8.7. Active Floor

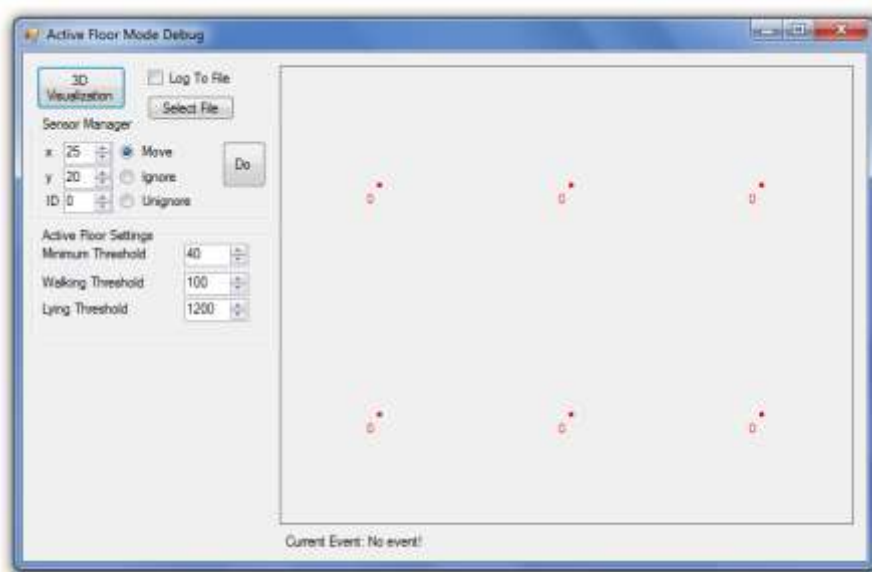


Figure 8-21 GUI Active Floor mode debug

Finally the Active Floor mode debug window is controlling settings and applications for the Active Floor input device. The setup is similar to the Bed debug window, including event logging, using a

different default logging file. The *Active Floor Settings* group is setting the minimum threshold for weighted average calculation and the two thresholds differentiating between walking and lying as defined in chapter 5.6.2.

8.7.1. Fall Detection

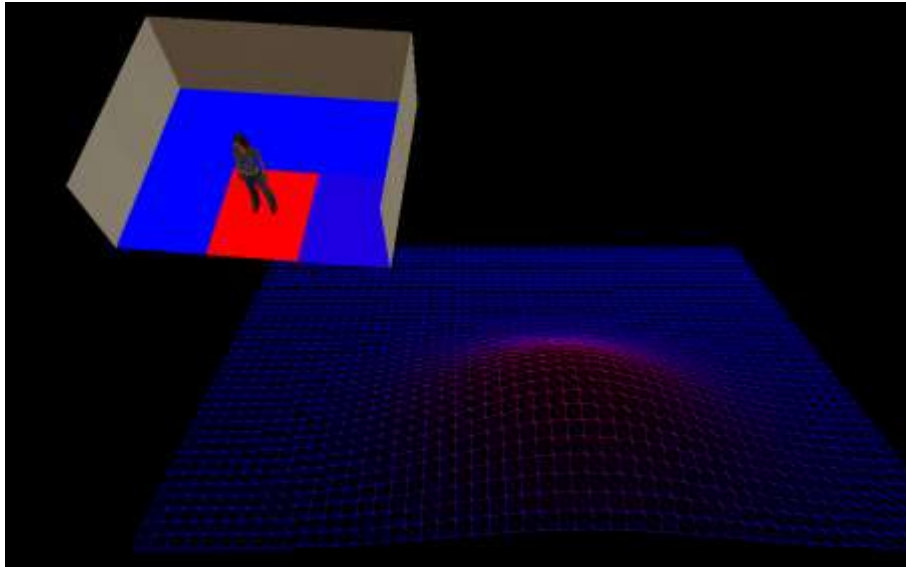


Figure 8-22 3D visualization of fall detection

The Fall Detection application is visualizing the input data of the Active Floor and generated events. The lower right side is displaying the location of a detected object similar to the 3D visualization of the Proximity Table.

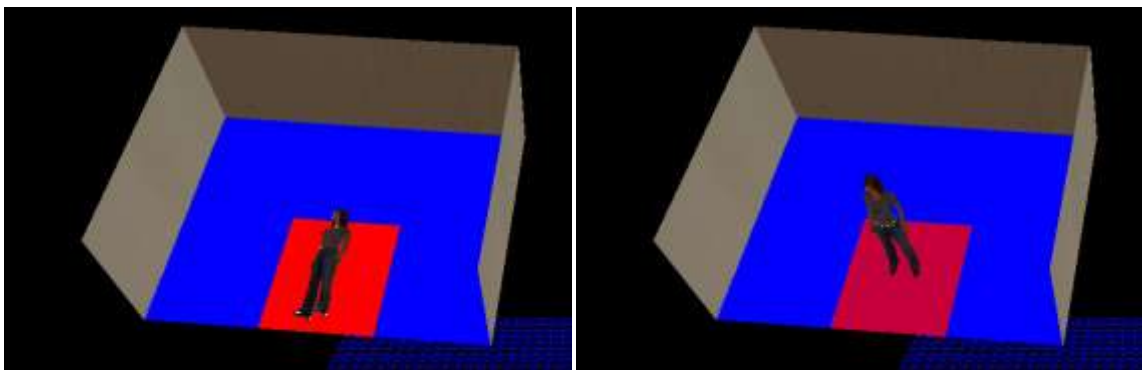


Figure 8-23 Room visualization with person lying or standing

Registered events are displayed as a small rendered room in the upper left side. As soon as a floor tile is active, a person is drawn in its center. If the sensor values exceed the lying threshold the model is drawn lying.

Additionally an audio signal is given as soon as a fall has been detected, acting as alarm function in case of emergency.

9. Evaluation Proximity Table

A small user survey was conducted over two days, September 4th and September 7th 2009. Employees and guests of the Fraunhofer IGD in Darmstadt were invited to try out the Proximity Table and its demonstration applications. The study was trying to collect feedback about input devices based on this technology and suggested applications, by people not involved with the project. The aim was to investigate the prospects of this technology, as well as receiving clues about how to improve the hardware and software in future iterations. In order to get a qualitative feedback a group of tech-savvy users, acquainted with different input devices, was chosen. Each participant was given a short introduction about the technology, interaction with the Proximity Table and features of the software application. Feedback was gathered by discussion with the 18 participants and through assessing an evaluation form, the users were asked to fill out (Appendix E). Results for the different software demonstrations are displayed in the table below. Participants could give each application a rating between 1 (worst) and 10 (best). Approval rating in the table below is the arithmetic average over all submitted results.

	Gesture Image Viewer	Screen Lens Image Viewer	3D Object Manipulation
Operations	Browse, Zoom	Browse, Regional Zoom	Rotate, Zoom, Pan
Interaction Paradigm	Hand Gestures	Active Regions, Direct Mapping	Menu, Hand Gestures, Active Regions
Approval Rating [1 (worst) -10 (best)]	8.67	8.67	6.44
Rating Details (Percentage of Users)			
• Negative (1-3)	0% (0 of 18)	0% (0 of 18)	5.6% (1 of 18)
• Undecided (4-7)	5.6% (1 of 18)	22.2% (4 of 18)	77.8% (14 of 18)
• Positive(8-10)	94.4% (17 of 18)	77.8% (14 of 18)	16.6% (3 of 18)

Table 9-1 Evaluation results of different software applications

Negative opinions focused mainly on three points:

- Lack of precision
- Sluggishness of the interaction
- Tiring of the arm after longer usage

The first two objections can be attributed to the technical limitations of the sensors used. Hard-wired systems, less susceptible to noise should be able to determine the position of the hand at a higher frequency and with improved precision. The main reason causing a tiring of the arm is the large interaction area in combination with an uncomfortable sitting position when interacting with the Proximity Table. This is caused by the placement of the device on a low-height living room table and the users sitting on a couch, which is forcing them to bend over and use a stretched out arm to interact. Suggestions by the participants included to move the system on a table where it can be manipulated in a standing position, as well as the development of smaller tablet-like devices that are low-weight and can be placed on a user's lap. While the latter is currently hard to realize, a different placing of the device within the laboratory seems reasonable.

The least favorite software application was the 3D object manipulation, with only three users giving a good rating. Participants complained that it was less intuitive than the other demonstrations and requires more training. This is not surprising, as this application is intended for the professional user and a certain level of training can be presumed. However some ideas about changing the timing and size of the menus and restricting certain camera values to feasible ranges might help to increase the usability of this software.

Positive feedback likewise features three main categories:

- Novelty of the device
- Intuitive interaction
- Simplicity of usage

The initial response of the participants towards the Proximity Table was mostly positive. User's showed interest as they did not use such devices before. Some had previous experience with other devices, tracking spatial position based on cameras and the differences to this system. The participants learned to use the Proximity Table pretty quickly. Most were only able to use it for a few minutes, yet managed to control the supplied applications without many issues. Others said usage did not require much force and should be possible even for handicapped people.

Concerning the demonstration applications both image viewers were well received, particularly the version controlled by gestures, that was liked by 17 of 18 participants. The majority was quickly able to learn the required gestures and the events they generate. Users could imagine using the screen lens for watching family photos and reading smaller text.

Conclusively the participants consider the Proximity Table an interesting and intuitive technology for interaction and acknowledge its advantages for certain applications. However, so far they are not satisfied with the precision and speed of the prototype. Regarding the supposed target group of elderly people in AAL environments, software design is a crucial point. Seeing that several participants had difficulties controlling some demonstrations, any application needs to be designed and evaluated with respect to physical and mental limitations of the target group.

10. Amending Current Assisted Living Environments

Capacitive proximity sensors are a versatile sensor type that has numerous applications in ambient assisted living environments, additionally to the devices already presented.

The setting of environment parameters, e.g. lighting and temperature, according to the current resident location is a common application of AAL. This can be useful to save energy and protect the person requiring assistance from harm, e.g. by automatically turning on lighting if the user is approaching a stairway. A simple system that is able to detect the room a person currently occupies, can be realized using capacitive proximity sensors based on wire electrodes hidden in a door frame (Figure 10-1 (1)). The direction of movement can be determined by comparing the time difference between the value maxima of both sensors. In order to successfully determine position the premises have to be modeled in a room map, shown in Figure 10-1 (2). The door frame of the main entrance, marked with an arrow, is determining the total amount of persons currently in the premises. The door frames associated with each room are tracking the number of people in individual rooms.

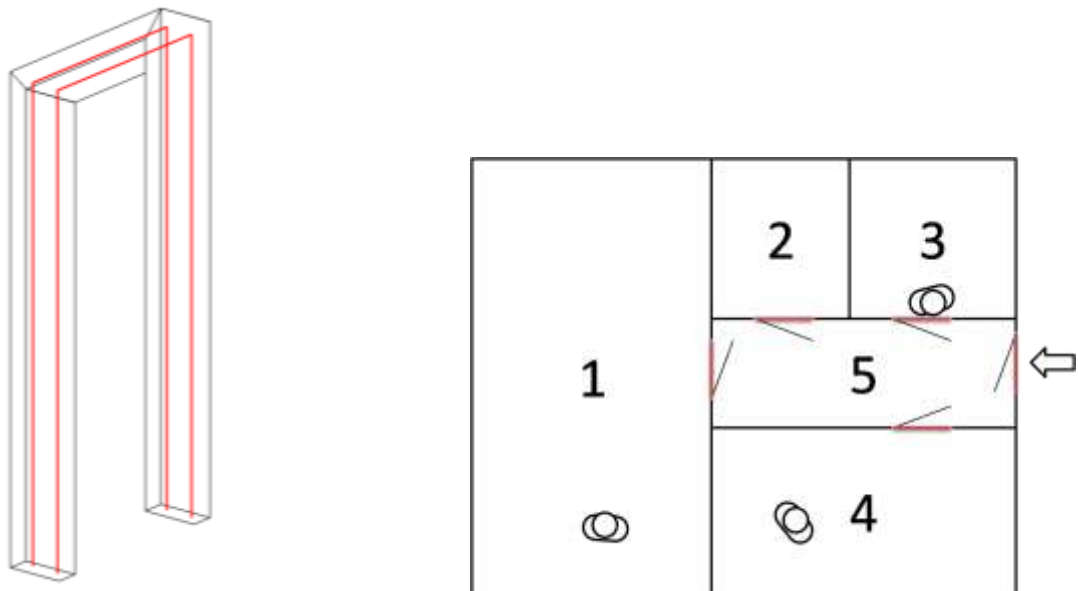


Figure 10-1 (1) Door frame sensors (2) Room map

One group of applications especially suited for capacitive proximity sensors is emergency prevention in possibly dangerous areas, by detecting the presence of bodies and sending alerting signals before impending accidents. A common example is warning a user before touching a hot oven plate, most notably patients who suffer from varying levels of dementia. Such systems can be realized by placing a sensor within the oven itself and connect it to audiovisual output devices that alarm the user before touching the heated area, in order to prevent serious injuries (Figure 10-2).

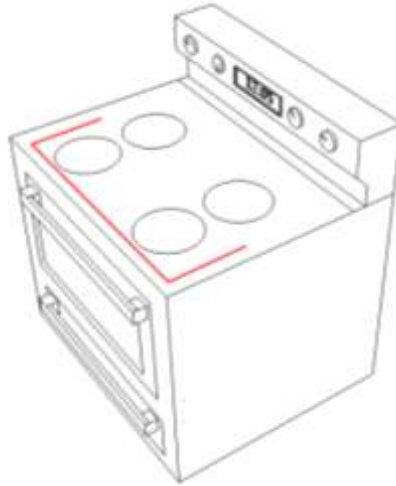


Figure 10-2 Oven amended with capacitive proximity sensors

For many elderly people it is crucial to receive medication at distinct times of the day. An intelligent medication box equipped with capacitive proximity sensors is able to register the number of times a person is accessing the box and at which time of the day those events occur. Unusual behavior can either directly trigger reminders for the user or alert the caregiver in more serious occurrences. However additional sensor equipment is required to detect which medicine the user has been taking, respectively failed to do so.

Another interesting application for capacitive sensors is respiration monitoring [34]. Using highly sensitive sensor devices and noise reducing spread spectrum transmission technology, it is possible to detect chest motion caused by breathing over a distance of at least 30cm. This allows the placement of mobile monitoring devices in close proximity to the patient, for example on a nightstand. These mobile devices could include other contact free instruments to measure additional body parameters, e.g. infrared thermometers determining body temperature.

11. Conclusion and Outlook

This thesis has presented a generic approach to model advanced input devices from capacitive proximity sensors. The CMR taxonomy has been expanded, in order to model advanced input devices and their complex data processing on included microprocessors. Furthermore several demonstration units based on generic hardware have been modeled and designed, showcasing potential applications in AAL environments. In order to debug these input devices and evaluate their usability, software has been developed that implements additional processing of the input data and presents several usage scenarios.

11.1. Outlook

Capacitive proximity detectors are a powerful sensor device that can be produced cheaply and applied flexibly. They are easily concealable and have the ability to emulate different other sensor devices. However, while capacitive sensing has gained considerable market share in touch screen devices and non-mechanical buttons, the application of proximity sensing is rare. The last few years have shown that there is a considerable interest in novel input devices, albeit mainly in entertainment and multimedia applications. If devices based on capacitive proximity sensing will ever reach a considerable market share is impossible to predict. On the other hand it is very likely that the input devices we use today will be replaced by more advanced units in the near future. Capacitive proximity sensors do have the potential to be an integral part of this progress.

11.2. Future Works

This thesis is merely a starting point on the way to real-world deployment of input devices based on capacitive proximity sensors. One of the main handicaps is the usage of general purpose sensor devices that are not particularly well-suited for high-speed interaction at a decent resolution. While they do allow easy experimentation and reconfiguration, units specifically designed for the task will provide better results. Considering the deliberate interaction of the Proximity Table, an integration of highly sensitive, hard-wired sensor devices should increase the spatial resolution and interaction area considerably, allowing for detecting gestures more precisely, which in turn might lead to more advanced input semantics.

The potential of an input device providing spatial position is largely unexplored on the application side. Presented software is merely using the third degree-of-freedom in situations where precision is not required. It can be assumed that advances in sensor design will be reflected in more advanced data processing algorithms that might handle coordinated movement of both hands.

In order to evaluate capacitive proximity input devices it is crucial to compare their efficiency in specific tasks to common units, like mouse or touch screen. However, it is also necessary to develop a methodology that allows quantifying the performance of devices supporting spatial interaction.

A. Interfacing with CY3271

A.1. Important Classes

An assembly called RFCommunicatorLib.DLL is included on the Kit CD. This file can be loaded into Visual Studio and opened via the Object Browser. The interesting namespace is Cypress.RFCommunicator.

Class RFCommunicator

Attributes		
Name	Type	Description
Instance	RFCommunicator	References singleton class
IsConnected	Boolean	Connection status of Bridge
Event Delegates		
Name	Description	
BindEnded	Triggered when bind operation has ended	
DataArrived	Triggered of new Message arrives	
DeviceAttached	Triggered if FTPC attached	
DeviceRemoved	Triggered if FTPC removed	
UnBindEnded	Triggered if unbind operation has finished	
Methods		
Name	Return Type	Description
BindNode	Byte	Put bridge in bind mode waiting for nodes to respond. Returns status code.
GetMessage	RFCommunicator.Message	Fetch most recent message.
Start	Boolean	Start FTPC. Returns success.
StartReading	Boolean	Start reading data from FTPC. Returns success
Stop	Boolean	Stop FTPC. Returns success
StopReading	Boolean	Stop reading data from FPC. Returns success.
UnBindNodeAll	RFCommunicator.CommunicationResult	Unbind all nodes.Return result

Table A-1 RFCommunicator class overview

Class NodeInfo

Attributes		
Name	Type	Description
<i>nodeId</i>	Byte	Unique ID of a node in the network
<i>nodeStatus</i>	Byte	Current status of the node

Table A-2 NodeInfo class overview

Class **HubInfo**

Attributes		
Name	Type	Description
<i>currentProtocolState</i>	Byte	Current protocol Status
<i>deviceType</i>	Byte	Type of connected device. Always EzI2C hub in this project
<i>networkSize</i>	Byte	Number of connected nodes.

Table A-3 HubInfo class overview

Class **Message**

Attributes		
Name	Type	Description
<i>Data</i>	Byte Array	Payload of the network package
<i>DateTime</i>	System.DateTime	Timestamp of the package
<i>Length</i>	Byte	Size of the Message
<i>NodeID</i>	Byte	Unique ID of the node that did send the Message

Table A-4 Message class overview

A.2. Example Code

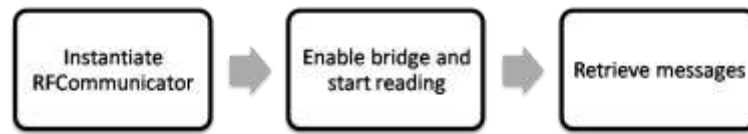


Figure A-1 Sample application algorithm

This example program is using the .NET Framework 3.5 and is written in C#. Create a new WinForms project. Requires a form with two labels called label1 for displayed data and label2 for hub status, as well as a Timer timer1, which should poll data every few hundred milliseconds.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using Cypress.RFCommunicator;

namespace MinimumCyFi
{
    public partial class Form1 : Form
    {
        RFCommunicator hub = RFCommunicator.Instance;
        byte[] data = new byte[Cypress.RFCommunicator.Message.MESSAGE_LEN];

        public Form1()
        {
            InitializeComponent();
            hub.Start();
            hub.StartReading();
            hub.DataArrived += new DataArrivedEventDelegate(hub_DataArrived);
            timer1.Enabled = true;
        }

        void hub_DataArrived(Cypress.RFCommunicator.Message message)
        {
            data = message.Data;
        }

        private void timer1_Tick(object sender, EventArgs e)
        {
            if (hub.IsConnected) label2.Text = "Hub Status: Connected";
            else
            {
                label2.Text = "Hub Status: Disconnected";
                return;
            }
            string s = "";
            foreach (byte b in data)
            {
                s = s + b.ToString() + " ";
            }
            label1.Text = s;
        }
    }
}
```

B. Hardware Modifications

B.1. Multifunction Expansion Board Firmware Settings

Noise Threshold	5
Baseline Update Threshold	200
Negative Noise Threshold	10
Low Baseline Reset	50
Sensors Autoreset	Enabled
Detection Threshold	1
Hysteresis	0
Debounce	0
Scan Speed	Slow
Resolution	16

Table B-1 Proximity table sensor firmware settings

Noise Threshold	5
Baseline Update Threshold	200
Negative Noise Threshold	10
Low Baseline Reset	50
Sensors Autoreset	Disabled
Detection Threshold	1
Hysteresis	0
Debounce	0
Scan Speed	Slow
Resolution	16

Table B-2 Active Floor & Bed sensor firmware settings

B.2. Modified FTRF Firmware

General

The firmware is owned by Cypress Semiconductor but licensed for modification, which is happening in the polling loop part of the FTRF source file. Several unnecessary functions are deactivated. Only the modified part of this larger source code file is displayed.

```
for (;;)
{
    //-----
    // Watch Start Binding Button activity
    //-----
    if (CheckBindButton())
    {
        CYFISNP_BindStart(ON_THE_FLY_DEV_ID);
        LED_GRN_ON;
    }
}
```

```

    }

    // -----
    // Watch "Force New Report" button
    // -----
    // if (CheckWakeButton())
    // {
    //     sendNewTxMsg();
    // }

    // -----
    // Run SNP less frequently to save energy
    // -----
    if (--snpRunScaler == 0)
    {
        snpRunScaler = SNP_RUN_SCALER;
        CYFISNP_Run();    // Poll SNP machine
    }

    // -----
    // Process received SNP data packets
    // (only supports update report rate for now)
    // -----
    if (CYFISNP_RxDataPend() == TRUE)
    {
        CYFISNP_API_PKT *pRxApiPkt;
        pRxApiPkt = CYFISNP_RxDataGet();
        reportTimeSec = pRxApiPkt->payload[0]; // Update report rate
        CYFISNP_RxDataRelease();
    }

    // -----
    // Periodic 1 sec events
    // -----
    if (CYFISNP_TimeExpired(&oneSecTimer) == TRUE)
    {
        CYFISNP_TimeSet(&oneSecTimer, sleepTicksPerSec);

        // -----
        // Blink Heartbeat LED
        // -----
        // if (--ledHeartbeatSec == 0)
        // {
        //     ledHeartbeatSec = LED_HEARTBEAT_SEC;
        //     blinkRedLed(50);    // *100 uS ON
        //     Calibrate_ILO();    // Calibrate ILO against IMO
        // }

        // -----
        // If SNP reconnect timeout, then try to reconnect.
        // (If very power sensitive, may want to defer for 5 minutes)
        // -----
        if (CYFISNP_eProtState == CYFISNP_CON_MODE_TIMEOUT)
        {
            CYFISNP_Jog();
        }

        // -----

```

```

// Periodic weather station report
// -----
if (--reportTimerSec == 0)
{
    reportTimerSec = reportTimeSec;
    sendNewTxMsg();
}
}

```

Proximity FTRF

In order to speed up the data transfer the sleepTicksPerSec variable has been reduced to a lower value. The final value was chosen to avoid FTFC freezing.

```

#ifndef MAIN_H
#define MAIN_H
#include <m8c.h> // part specific constants and macros
#include "PSoCAPI.h" // PSoC API definitions for all User Modules
#include "psocGpioInt.h"

//-----
//
// #defines
//
//-----

#define ON_THE_FLY_DEV_ID 0x00

#define LED_HEARTBEAT_SEC 1 // Heartbeat LED blink rate (in Sec)
#define REPORT_TIME_SEC 1 // Default reporting time (in Sec)

#define SNP_RUN_SCALER 2 // Run SNP every 2nd cycle (every ~30mS)
static BYTE snpRunScaler; // Slow-down CYFISNP_Run() to save energy

#define DEBOUNCE_TIME 2

#define LOW_POWER_TEST 1

#define ChipSelect_ON (ChipSelect_Data_ADDR != ChipSelect_MASK)
#define ChipSelect_OFF (ChipSelect_Data_ADDR &= ~ChipSelect_MASK)

#define IS_DataReady_ON (DataReady_Data_ADDR & DataReady_MASK)

#define SLAVE_ADDRESS 0x05

// -----
// Wait up to 300 mS for Weather Station board to respond
// -----
#define DATA_READY_TIME (300 / CYFISNP_TIMER_UNITS)

#define I2C_PAYLOAD_MAX 8
//-----
//
// Global Variables
//
//-----

static CYFISNP_API_PKT txApiPkt; // Local Tx Data buffer

```

```

WORD ledHeartbeatSec = LED_HEARTBEAT_SEC;

WORD reportTimeSec = REPORT_TIME_SEC;
WORD reportTimerSec = REPORT_TIME_SEC;

WORD sleepTicksPerSec = 15; // 64 sleep timer ticks is "nominally" 1 sec
WORD oneSecTimer;

BYTE rxBuffer[I2C_PAYLOAD_MAX];
//BYTE rxData[CYFISNP_FCD_PAYLOAD_MAX];
//BYTE status;

#define LED_RED_OFF (RedLED_Data_ADDR &= ~RedLED_MASK)
#define LED_RED_ON (RedLED_Data_ADDR |= RedLED_MASK)
#define LED_RED_TOG (RedLED_Data_ADDR ^= RedLED_MASK)
#define IS_LED_RED_ON (RedLED_Data_ADDR & RedLED_MASK)

#define LED_GRN_OFF (GreenLED_Data_ADDR &= ~GreenLED_MASK)
#define LED_GRN_ON (GreenLED_Data_ADDR |= GreenLED_MASK)
#define LED_GRN_TOG (GreenLED_Data_ADDR ^= GreenLED_MASK)
#define IS_LED_GRN_ON (GreenLED_Data_ADDR & GreenLED_MASK)

//Thermistor ON and OFF defines
#define Port2_DataShade_ON (Port_2_Data_SHADE |= Vtherm_div_MASK)
#define Port2_DataShade_OFF (Port_2_Data_SHADE &= ~Vtherm_div_MASK)
#define Vtherm_div_OFF (Vtherm_div_Data_ADDR = Port2_DataShade_OFF)
#define Vtherm_div_ON (Vtherm_div_Data_ADDR = Port2_DataShade_ON)

#endif

```

Active Floor and Bed FTRF

Again sleepTicksPerSec has been modified, in this case to guarantee reduced power consumption and even less freezes. Active Floor and Bed do not require the frequent polling rate of the Proximity Table.

```

#ifndef MAIN_H
#define MAIN_H

#include <m8c.h> // part specific constants and macros
#include "PSoCAPI.h" // PSoC API definitions for all User Modules
#include "psocGpioInt.h"
//-----
//
//#defines
//
//-----
#define ON_THE_FLY_DEV_ID 0x00

#define LED_HEARTBEAT_SEC 1 // Heartbeat LED blink rate (in Sec)
#define REPORT_TIME_SEC 1 // Default reporting time (in Sec)

#define SNP_RUN_SCALER 2 // Run SNP every 2nd cycle (every ~30mS)
static BYTE snpRunScaler; // Slow-down CYFISNP_Run() to save energy

```

```

#define DEBOUNCE_TIME 2
#define LOW_POWER_TEST 1

#define ChipSelect_ON (ChipSelect_Data_ADDR != ChipSelect_MASK)
#define ChipSelect_OFF (ChipSelect_Data_ADDR &= ~ChipSelect_MASK)

#define IS_DataReady_ON (DataReady_Data_ADDR & DataReady_MASK)

#define SLAVE_ADDRESS 0x05

// -----
// Wait up to 300 mS for Weather Station board to respond
// -----
#define DATA_READY_TIME (300 / CYFISNP_TIMER_UNITS)

#define I2C_PAYLOAD_MAX 8

//-----
//
// Global Variables
//
//-----

static CYFISNP_API_PKT txApiPkt; // Local Tx Data buffer

WORD ledHeartbeatSec = LED_HEARTBEAT_SEC;

WORD reportTimeSec = REPORT_TIME_SEC;
WORD reportTimerSec = REPORT_TIME_SEC;

WORD sleepTicksPerSec = 30; // 64 sleep timer ticks is "nominally" 1 sec
WORD oneSecTimer;

BYTE rxBuffer[I2C_PAYLOAD_MAX];
//BYTE rxData[CYFISNP_FCD_PAYLOAD_MAX];
//BYTE status;

#define LED_RED_OFF (RedLED_Data_ADDR &= ~RedLED_MASK)
#define LED_RED_ON (RedLED_Data_ADDR != RedLED_MASK)
#define LED_RED_TOG (RedLED_Data_ADDR ^= RedLED_MASK)
#define IS_LED_RED_ON (RedLED_Data_ADDR & RedLED_MASK)

#define LED_GRN_OFF (GreenLED_Data_ADDR &= ~GreenLED_MASK)
#define LED_GRN_ON (GreenLED_Data_ADDR != GreenLED_MASK)
#define LED_GRN_TOG (GreenLED_Data_ADDR ^= GreenLED_MASK)
#define IS_LED_GRN_ON (GreenLED_Data_ADDR & GreenLED_MASK)

//Thermistor ON and OFF defines
#define Port2_DataShade_ON (Port_2_Data_SHADE != Vtherm_div_MASK)
#define Port2_DataShade_OFF (Port_2_Data_SHADE &= ~Vtherm_div_MASK)
#define Vtherm_div_OFF (Vtherm_div_Data_ADDR = Port2_DataShade_OFF)
#define Vtherm_div_ON (Vtherm_div_Data_ADDR = Port2_DataShade_ON)
#endif

```

C. Bed Full Posture Listing

C.1. Sensor Activity Mapping

Instead of using the full graphical listing of chapter 5.2.3 a simple graphical representation of active sensors is used, as shown in Figure C-1.

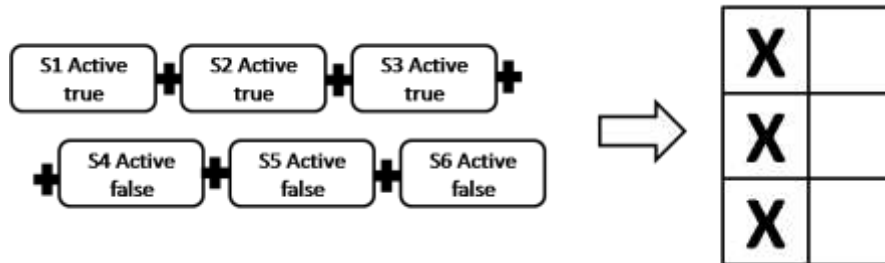


Figure C-1 Symbolic notation for sensor activity

C.1.1. One Person

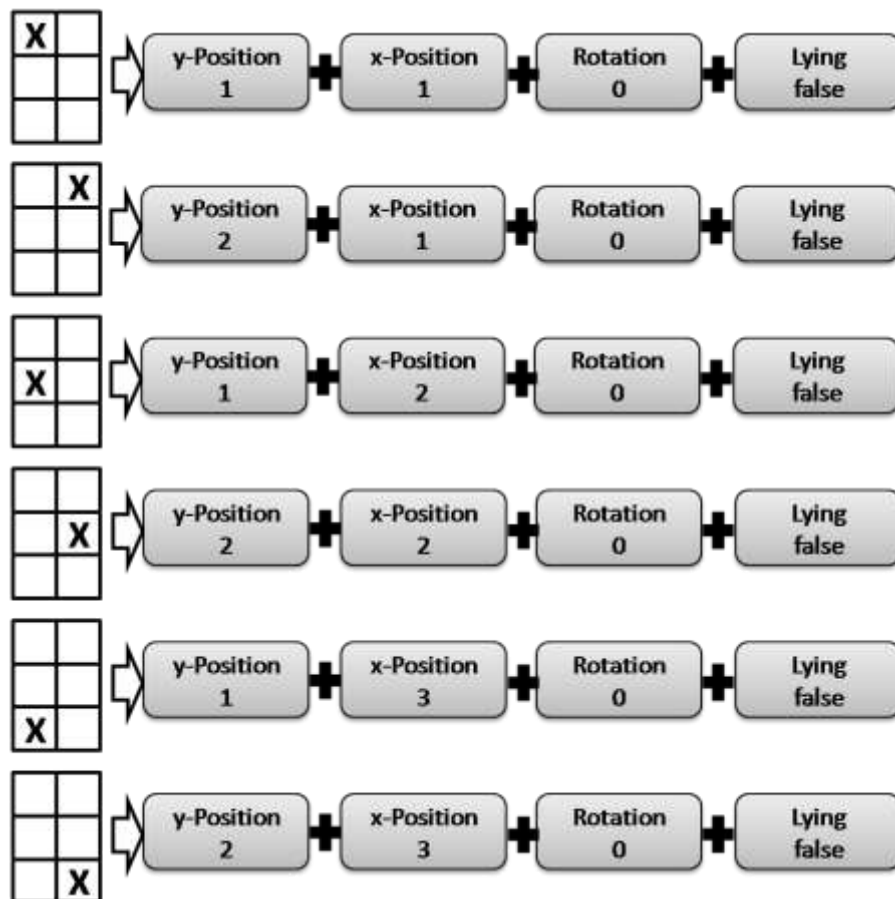


Figure C-2 Sensor activity mapping of one person sitting

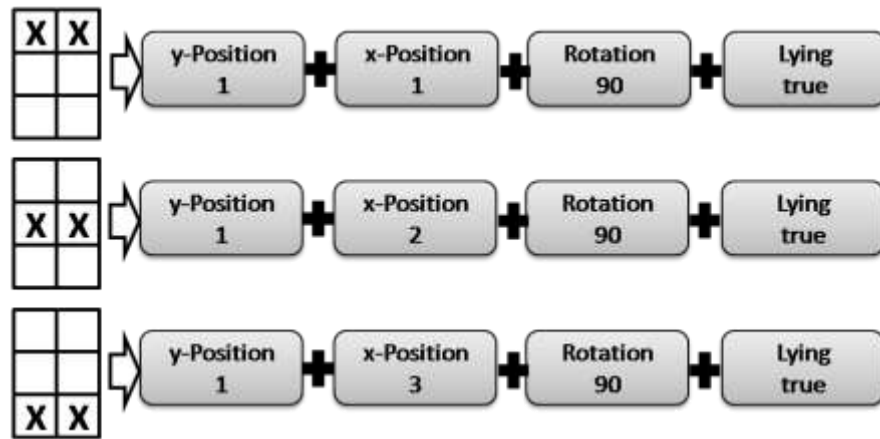


Figure C-3 Sensor activity mapping of one person lying horizontally

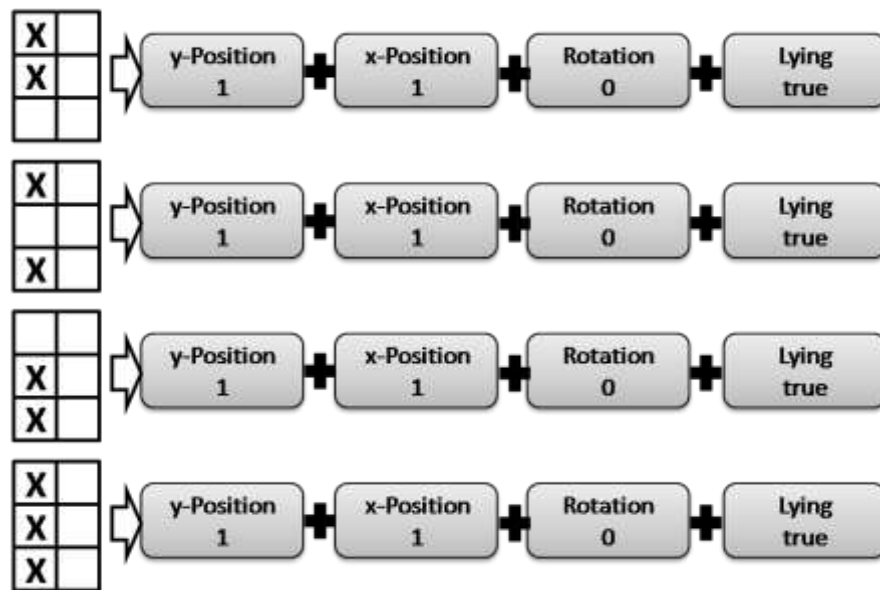


Figure C-4 Sensor activity mapping of one person lying on the left side

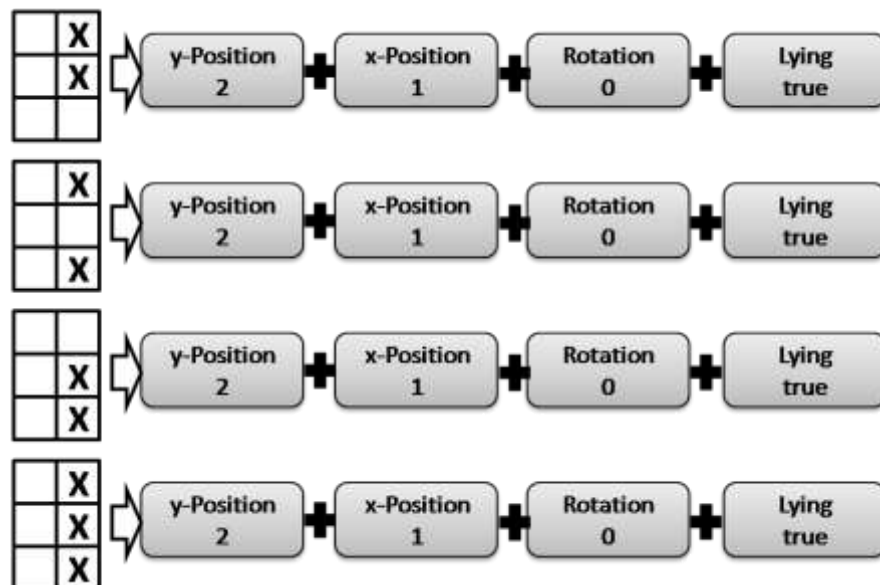


Figure C-5 Sensor activity mapping of one person lying on the right side

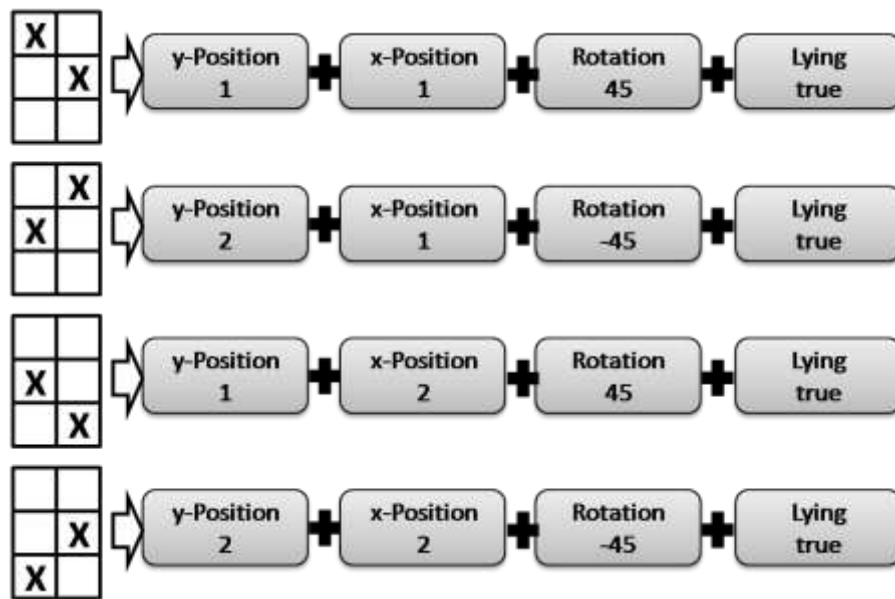


Figure C-6 Sensor activity mapping for one person lying at a 45 degree angle on bed

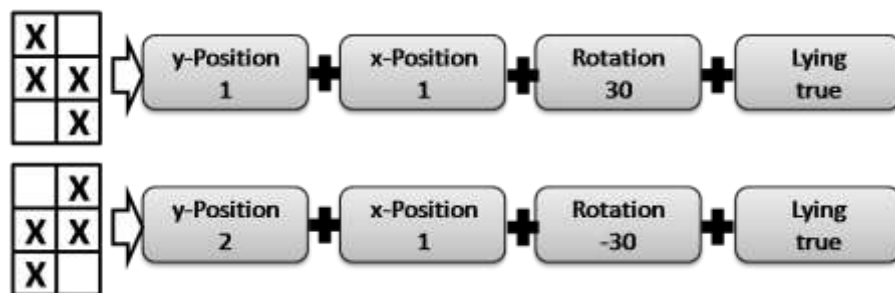


Figure C-7 Sensor activity mapping for one person lying at a 30 degree angle on bed

C.1.2. Two Persons

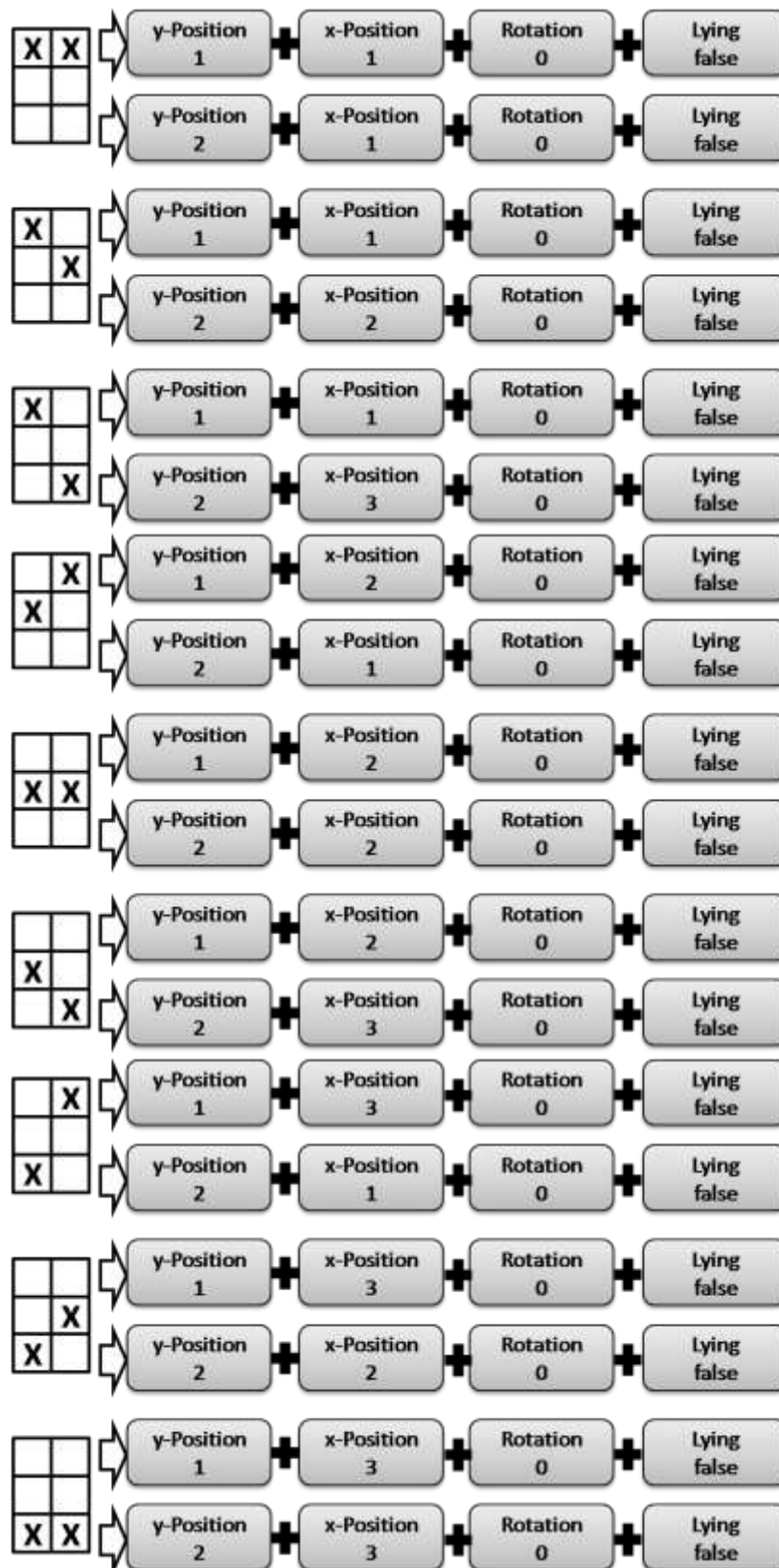


Figure C-8 Sensor activity mapping of two persons sitting

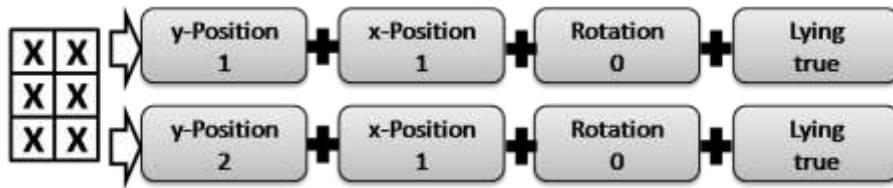


Figure C-9 Sensor activity mapping of two persons sitting

C.2. Spine Strain Mapping

NOAL is the number of active sensors in left area. NOAR is the number of active sensors in the right area. LA and RA are balances of the left or right area. Again this mapping is rough at best.

C.2.1. One Person



Figure C-10 Spine strain mapping of one person sitting

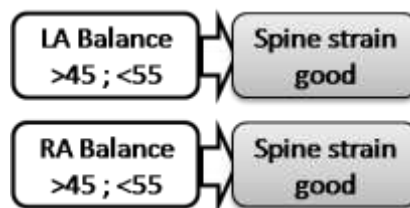


Figure C-11 Spine strain mapping of one person lying vertically

C.2.2. Two Persons



Figure C-12 Spine strain mapping of two persons sitting

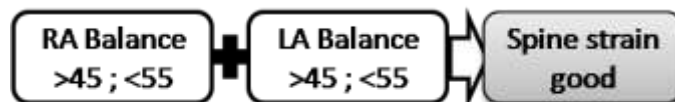


Figure C-13 Spine strain mapping of two persons lying

D. Software Details

D.1. Typical Setting File

[Bed Mode Settings]

SensorActiveThreshold = 100

LeftBalancePoint = 50

RightBalancePoint = 50

[Room Mode Settings]

MinimumThreshold = 40

WalkingThreshold = 100

LyingThreshold = 1200

[Table Mode Settings]

DefaultImageDirectory = C:\Users\User\Pictures\Nature\Set 2

ActiveMode = 0

InterpolationAlgorithm = 1

LocationAlgorithm = 0

FilteringAlgorithm = 1

FilteringSamples = 3

Height4Threshold = 39

Height3Threshold = 100

Height2Threshold = 200

Height1Threshold = 1200

InterpolationDelta = 8

InterpolationBufferSize = 5

ArmCompensation = 0

PredictionVectors = 1

GestureSwitchLength = 20

GestureCooldown = 20

GestureZoomLength = 10

GestureZoomSpeed = 5

GestureZoomMode = 0

ScreenLensLongTicks = 8

ScreenLensZoomStrength = 5

StickyLensActive = 1

StickyLensDelta = 10

StickyLensMaxRepeats = 10

ManipulationAppearTicks = 20

ManipulationMenuTicks = 10

ManipulationRotationSpeed = 5

ManipulationPanSpeed = 5

ManipulationZoomSpeed = 5

ManipulationMaximumTrajectoryLength = 15

ManipulationIgnoreticks = 3

MenuAppearTicks = 20

MenuChooseTicks = 10

MenuDisappearTicks = 60

D.2. Screen Lens Shader Code

```
float2 center;
float aspect;
float strength;

//----- TEXTURE PROPERTIES -----
// This is the texture that SpriteBatch will try to set before drawing
texture ScreenTexture;

// Our sampler for the texture, which is just going to be pretty simple
sampler TextureSampler = sampler_state
{
    Texture = <ScreenTexture>;
};

//----- PIXEL SHADER -----
// This pixel shader will displace texture coordinates according to a center at a given strength

float4 PixelShaderFunction(float2 TextureCoordinate : TEXCOORD0) : COLOR0
{
    float4 color = 0.0;
    float2 displacement = 0.0;
    //float displacement = 1.0;
    float2 dif;
    dif.x = TextureCoordinate.x - center.x;
    dif.y = (TextureCoordinate.y - center.y) * aspect;
    if(length(dif) < 0.15)
    {
        displacement.x = strength * dif.x * aspect;
        displacement.y = strength * dif.y;
    }

    float2 coord = TextureCoordinate - displacement;
    color = tex2D(TextureSampler, coord);
    return color;
}

//----- TECHNIQUES -----
// This technique is pretty simple - only one pass, and only a pixel shader
technique BlackAndWhite
{
    pass Pass1
    {
        PixelShader = compile ps_2_0 PixelShaderFunction();
    }
}
```

E. Evaluation Proximity Table

E.1. Evaluation Sheet

EVALUATION PROXIMITY TABLE

1. Hatten Sie das Gefühl dass die geforderten Anwendungen einfach und präzise zu erledigen waren?

Wenig Zutreffend Sehr Zutreffend

--	--	--	--	--	--	--	--	--	--

2. Wie hat Ihnen die Gesture Image Viewer Applikation gefallen?

Gar Nicht Sehr Gut

--	--	--	--	--	--	--	--	--	--

3. Wie hat Ihnen die Screen Lens Applikation gefallen?

Gar Nicht Sehr Gut

--	--	--	--	--	--	--	--	--	--

4. Wie hat Ihnen die 3D Object Manipulation Applikation gefallen?

Gar Nicht Sehr Gut

--	--	--	--	--	--	--	--	--	--

5. Finden Sie die Bedienung des Proximity Table intuitiv?

Wenig Intuitiv Sehr Intuitiv

--	--	--	--	--	--	--	--	--	--

6. Halten Sie ein Eingabegerät wie den Proximity Table für eine interessante Möglichkeit der Interaktion?

Wenig Zutreffend Sehr Zutreffend

--	--	--	--	--	--	--	--	--	--

7. Könnten Sie sich vorstellen ein Gerät wie den Proximity Table längerfristig zu nutzen?

Wenig Zutreffend Sehr Zutreffend

--	--	--	--	--	--	--	--	--	--

8. Was hat Ihnen am Proximity Table besonders gut gefallen?

--

9. Was hat Ihnen am Proximity Table gar nicht gefallen?

--

Wir bedanken uns für Ihre Teilnahme!

Figure E-1 Scan of the Proximity Table evaluation sheet

E.2. Evaluation Results

The total number of submitted sheets was 18. Each question was answered by giving a number from 1 as worst to 10 as best.

	Average	Lowest	Highest	Median
Question 1	5.5	2	8	5.5
Question 2	8.67	7	10	9
Question 3	8.67	6	10	9
Question 4	6.44	3	10	7
Question 5	7.67	3	10	8
Question 6	8.89	7	10	9
Question 7	6.67	3	10	7

Table E-1 Evaluation sheet results

List of Tables

Table 4-1 Possible manipulation operators	11
Table 4-2 Six-tuple of a 270-degree volume knob	12
Table 4-3 Comparison chart of input device taxonomies	12
Table 4-4 Necessary services and available technologies in assisted living.....	16
Table 5-1 Mechanical mouse sensors	19
Table 5-2 Capacitive proximity sensor input devices based on mode.....	22
Table 5-3 Structure setup	23
Table 5-4 Bed intermediary structure	32
Table 5-5 Person data structure	32
Table 5-6 Active Floor Person data structure.....	35
Table 7-1 Data of Proximity Table prototype	44
Table 8-1 Table structure of event logging CSV file.....	52
Table 9-1 Evaluation results of different software applications	65
Table A-1 RFCommunicator class overview	i
Table A-2 NodeInfo class overview	i
Table A-3 HubInfo class overview.....	ii
Table A-4 Message class overview.....	ii
Table B-1 Proximity table sensor firmware settings.....	iv
Table B-2 Active Floor & Bed sensor firmware settings	iv
Table E-1 Evaluation sheet results	xvii

List of Images

Figure 3-1 Speed Drawing of a Computer	2
Figure 4-1 Black box setup of a capacitive proximity sensor	4
Figure 4-2 Capacitive sensing procedure	5
Figure 4-3 Measurement circuit	6
Figure 4-4 A young Leon Theremin playing his instrument	7
Figure 4-5 School of Fish sensor array	8
Figure 4-6 Thracker attached to a LCD screen	9
Figure 4-7 (1) Keys on the Zuse Z3 (2) The first computer mouse (3) Ivan Sutherland in front of his Sketchpad	9
Figure 4-8 Transaction tree categorization	10
Figure 4-9 Buxton table for continuous, hand-controlled input devices	11
Figure 4-10 (1) Person in perceptive space (2) Interactive workbench (3) HoloWall	14
Figure 4-11 (1) Mixed-Reality game using the Sony EyeToy (2) Woman using Nintendo Wii Balance Board	14
Figure 4-12 (1) Group of people playing Rock Band game (2) Woman using Microsoft Project Natal demonstration unit	15
Figure 4-13 (1) Apple iPhone (2) Cowon S9 personal media player (3) Palm Pre phone	15
Figure 4-14 (1) Tunstall Fall Detector (2) Fujitsu Service Robot enon (3) iRobot Roomba vacuum cleaner	17
Figure 5-1 Modeling mouse and linking to cursor	19
Figure 5-2 Input Device Processing Model	20
Figure 5-3 Proximity Mode	21
Figure 5-4 State Mode	21
Figure 5-5 Data processing in the CMR-Model	22
Figure 5-6 Data processing pipeline	23
Figure 5-7 Generic semantic mapping	23
Figure 5-8 Full CMR model	24
Figure 5-9 Data processing black box detailed	24
Figure 5-10 Design model of Proximity Table input device	25
Figure 5-11 Proximity sensor fusion structure	25
Figure 5-12 Linear interpolation of height values	26
Figure 5-13 Influence of noise at different sensor value totals	27
Figure 5-14 Result of no interpolation (1), linear interpolation (2) and differential interpolation (3) ..	28
Figure 5-15 Multiple objects increasing number of active sensors	30

Figure 5-16 Proximity Table data processing virtual input device.....	30
Figure 5-17 Design model of Bed input device.....	31
Figure 5-18 One person lying vertically on left side of the Bed.....	32
Figure 5-19 Semantic mapping of sensor configuration to Person structure	33
Figure 5-20 Bed data processing virtual input device	33
Figure 5-21 Design model of Active Floor	34
Figure 5-22 Active Floor data fusion structure	34
Figure 5-23 (1) Localization using most active tile (2) Localization using weighted average	35
Figure 5-24 Active Floor data processing virtual input device	35
Figure 5-25 First ambiguous configuration of floor tile - two standing or one lying on single tile	36
Figure 5-26 Second ambiguous configuration of floor tiles – two standing on neighboring tiles or one lying.....	36
Figure 5-27 Room entry and leave events	36
Figure 6-1 CY3271 Kit – PC bridge board.....	37
Figure 6-2 CY3271 Kit – RF expansion card	37
Figure 6-3 CY3271 Kit – Multifunction expansion card	38
Figure 6-4 CY3271 Kit – AAA battery pack.....	38
Figure 6-5 CY3271 Kit – CR2032 battery pack	38
Figure 6-6 Typical sensor setup of this project	39
Figure 6-7 Physical property to usable data	40
Figure 6-8 (1) Static baseline update (2) Dynamic baseline update	40
Figure 6-9 Low baseline reset	41
Figure 6-10 Triggering sensor with Hysteresis and Debounce	41
Figure 6-11 Data transmission network	42
Figure 6-12 I2C Data Package.....	42
Figure 7-1 Rendering of a Proximity Table model.....	44
Figure 7-2 Proximity Table input device closed.....	45
Figure 7-3 Proximity Table input device open.....	45
Figure 7-4 Rendering of a Bed model	46
Figure 7-5 Bed equipped with Bed input device	46
Figure 7-6 Antennas attached to slatted frame.....	47
Figure 7-7 Rendering of Active Floor model.....	47
Figure 7-8 Active Floor separated in several tiles	48
Figure 7-9 Detailed view of floor tile with antenna in the center.....	48
Figure 8-1 GUI structure.....	49
Figure 8-2 Functional processing structure	51

Figure 8-3 GUI Main window	51
Figure 8-4 GUI Simulator mode.....	52
Figure 8-5 GUI Table mode debug	53
Figure 8-6 3D visualization of Proximity Table mode.....	54
Figure 8-7 Application switch collapsed and expanded	55
Figure 8-8 GUI of Gesture Image Viewer.....	55
Figure 8-9 Four Gesture Image Viewer gestures.....	56
Figure 8-10 GUI of Screen Lens Image Viewer	57
Figure 8-11 Comparison between high zoom factor (left) and low zoom factor (right)	58
Figure 8-12 GUI of 3D Object Manipulation.....	58
Figure 8-13 3D Object Manipulation menu collapsed and expanded	59
Figure 8-14 Trajectory calculation from detected points	59
Figure 8-15 Leaving intention prediction	60
Figure 8-16 3D Object Manipulation regions in zoom mode.....	61
Figure 8-17 GUI Bed mode debug.....	61
Figure 8-18 3D visualization of position and posture detection	62
Figure 8-19 Mattress pressure visualization	62
Figure 8-20 Visualization of different postures and spine stress	63
Figure 8-21 GUI Active Floor mode debug	63
Figure 8-22 3D visualization of fall detection.....	64
Figure 8-23 Room visualization with person lying or standing	64
Figure 10-1 (1) Door frame sensors (2) Room map	67
Figure 10-2 Oven amended with capacitive proximity sensors	68
Figure A-1 Sample application algorithm.....	iii
Figure C-1 Symbolic notation for sensor activity	ix
Figure C-2 Sensor activity mapping of one person sitting	ix
Figure C-3 Sensor activity mapping of one person lying horizontally	x
Figure C-4 Sensor activity mapping of one person lying on the left side	x
Figure C-5 Sensor activity mapping of one person lying on the right side.....	x
Figure C-6 Sensor activity mapping for one person lying at a 45 degree angle on bed.....	xi
Figure C-7 Sensor activity mapping for one person lying at a 30 degree angle on bed.....	xi
Figure C-8 Sensor activity mapping of two persons sitting	xii
Figure C-9 Sensor activity mapping of two persons sitting	xiii
Figure C-10 Spine strain mapping of one person sitting	xiii
Figure C-11 Spine strain mapping of one person lying vertically	xiii
Figure C-12 Spine strain mapping of two persons sitting	xiii



Figure C-13 Spine strain mapping of two persons lying xiii

Figure E-1 Scan of the Proximity Table evaluation sheet xvi

Bibliography

- [1] A. Agnihotri, B. Purwar, N. Jeebun, and S. Agnihotri, "Determination of Sex By Hand Dimensions," *The Internet Journal of Forensic Science*, vol. 1, no. 2, January 2006.
- [2] Michel Assenheimer et al., "The T-Scan technology: electrical impedance as a diagnostic tool for breast cancer detection," *Physiological Measurement*, pp. 1-8, February 2001.
- [3] Max Baars. (2004, December) The Theremin - How it works. [Online]. <http://www.thereminvox.com/article/articleview/134/1/2/>
- [4] D.C. Barber and B.H. Brown, "Applied Potential Tomography," *Journal of Physics E: Scientific Instruments*, pp. 723-733, September 1984.
- [5] Larry K. Baxter, *Capacitive Sensors*.: Wiley-IEEE Press, 1996.
- [6] M. S. Brandstein and H. F. Silverman, "A practical methodology for speech source localization with microphone arrays," *Computer Speech & Language*, vol. 11, no. 2, pp. 91-126, November 1997.
- [7] Bill Buxton. (2009, January) Taxonomies of Input. [Online]. <http://www.billbuxton.com/input04.Taxonomies.pdf>
- [8] Stuart K. Card, Jock D. Mackinlay, and George G. Robertson, "The Design Space of Input Devices," in *Proceedings of the SIGCHI conference on Human factors in computing systems: Empowering people*, Seattle, Washington, United States, 1990, pp. 117-124.
- [9] J. Chen, Kwong Karic, D. Chang, J. Luk, and R. Bajcsy, "Wearable sensors for reliable fall detection," in *Engineering in Medicine and Biology Society, 2005. IEEE-EMBS 2005. 27th Annual International Conference of the*, Shanghai, 2006, pp. 3551-3554.
- [10] Jonny Lee Chung. (2008, August) Johnny Chung Lee - Projects - Wii. [Online]. <http://johnnylee.net/projects/wii/>
- [11] D Cohen, "Magnetic Fields around the Torso: Production by Electrical Activity of the Human Heart," *Science*, pp. 652-654, May 1967.
- [12] James L. Crowley and Yves Demazeau, "Principles and Techniques for Sensor Data Fusion," LIFIA (IMAG), Grenoble, Paper 1993.
- [13] Cypress Semiconductor, "CapSense Sigma-Delta Data Sheet," Cypress Semiconductor, San Jose, Data Sheet 2008.
- [14] Cypress Semiconductor, "CY3271 Kit Guide," Cypress Semiconductor, San Jose, Data Sheet 2008.
- [15] Cypress Semiconductor, "CYRF 7936 - 2.4GHz CyFi Transceiver," Cypress Semiconductor, Data Sheet 2009.
- [16] DFKI - German Research Center for Artificial Intelligence. (2009, July) Living Lab Innovative Retail Laboratory - IRL. [Online]. <http://www.dfki.de/web/living-labs-en/irl>
- [17] Arnaud Doucet, Simon Godsill, and Christophe Andrieu, "On sequential Monte Carlo sampling methods for Bayesian filtering," *Statistics and Computing*, vol. 10, no. 3, pp. 197-208, Juli 2000.
- [18] Benj Edwards. (2008, December) The computer mouse turns 40. [Online]. <http://www.macworld.com/article/137400/2008/12/mouse40.html>
- [19] Ina Fried. (2009, July) Bill Gates offers the world a physics lesson. [Online]. http://news.cnet.com/8301-13860_3-10286732-56.html
- [20] Fujitsu Frontech Limited. (2009, July) Fujitsu Service Robot (enon). [Online]. <http://www.frontech.fujitsu.com/en/forjp/robot/servicerobot/>
- [21] Gartner, Inc. (2009, May) Gartner Says Worldwide Mobile Phone Sales Declined 8.6 Per Cent and Smartphones Grew 12.7 Per Cent in First Quarter of 2009. [Online]. <http://www.gartner.com/it/page.jsp?id=985912>
- [20] Albert Glinsky, *Theremin - Ether Music and Espionage*.: University of Illinois Press, 2000.

-
-
- [23] S. Grimnes and O.G. Martinsen, *Bioimpedance and Bioelectricity Basics.*, 2nd ed. London, England: Academic Press, 2008.
- [24] Chen Hong. (2006, October) Robots may force chefs out of kitchen. [Online]. http://www.chinadaily.com.cn/china/2006-10/18/content_710686.htm
- [25] IDS Imaging Development Systems GmbH. (2009, September) IDS Imaging Development Solutions. [Online]. http://www.ids-imaging.de/frontend/files/IDS_P_uEye_ME_engl_04_09_web.pdf
- [26] IFR Statistical Department, "The world market increased by 3% in 2007.," IFR Statistical Department, Frankfurt, Press Release 2008.
- [27] InTouch Technologies, Inc. (2009, July) InTouch Technologies, Inc. - Products: RP-7 Robot. [Online]. http://intouchhealth.com/products_rp7robot.html
- [28] Kai Jauslin. (2009, August) Gesturespace. [Online]. <http://iad.projects.zhdk.ch/gesturespace/>
- [29] Douglas W. Jones. (2009, July) Punched Cards: A brief illustrated technical history. [Online]. <http://www.cs.uiowa.edu/~jones/cards/history.html>
- [30] N Kerrouche, C.N. McLeod, and W.R.B. Lionheart, "Time series of EIT chest images using singular value decomposition and Fourier transform," *Physiological Measurement*, pp. 147-157, February 2001.
- [31] Konami Digital Entertainment, Inc. (2009, September) DDR Community. [Online]. <http://www.ddronlinecommunity.com/>
- [32] Bastian Leibe et al., "The Perceptive Workbench: Toward Spontaneous and Natural Interaction in Semi-Immersive Virtual Environments," in *IEEE Virtual Reality Conference 2000 (VR 2000)*, New Brunswick, 2000, p. pp.13.
- [33] Lee Hoi Leong, Shinsuke Kobayashi, Noboru Koshizuka, and Ken Sakamura, "CASIS: a context-aware speech interface system," in *Proceedings of the 10th international conference on Intelligent user interfaces*, San Diego, 2005, pp. 231-238.
- [34] Rob MacLachlan. (2009, August) Spread Spectrum Capacitive Sensor. [Online]. <http://www.cs.cmu.edu/~ram/capsense/>
- [35] Dieter Meschede, , Dieter Meschede, Ed. Berlin Heidelberg New York: Springer-Verlag, 2002, p. 446.
- [36] Microsoft Corporation. (2009, September) Xbox.com | Project Natal. [Online]. <http://www.xbox.com/en-US/live/projectnatal/>
- [37] Microsoft Corporation. (2009, September) XNA Developer Center. [Online]. <http://msdn.microsoft.com/en-us/xna/default.aspx>
- [38] Microsoft Corporation. (2009, September).NET Framework Developer Center. [Online]. <http://msdn.microsoft.com/de-de/netframework/default.aspx>
- [39] Frank G. Miskelly, "Assistive technology in elderly care," *Age and Ageing*, vol. 30, no. 6, pp. 455-458, 2001.
- [40] MIT Media Laboratory. (2009, September) Smart Rooms. [Online]. <http://vismod.media.mit.edu/vismod/demos/smartroom/>
- [41] Jürgen Nehmer, Martin Becker, Arthur Karshmer, and Rosemarie Lamm, "Living assistance systems: an ambient intelligence approach," in *Proceedings of the 28th international conference on Software engineering*, Shanghai, China , 2006, pp. 43-50.
- [42] Nintendo. (2009, September) Us.Wii.Com - The Global Wii Experience Website in English. [Online]. <http://us.wii.com/>
- [43] Nintendo. (2009, September) Wii Fit. [Online]. <http://www.nintendo.com/wiifit/>
- [44] Atsushi Nishino and Akihiko Yoshida, "Humidity Sensor of capacitance change type," US4217623, August 12, 1980.
- [45] NXP Semiconductors, "UM10204 I2C-bus specification and user manual," NXP Semiconductors,

-
- Whitepaper 2007.
- [46] Richard L. Owens. (2009, August) Optical Mouse technology. [Online]. <http://www.mstar.net/users/rlowens/OpticalMouse/>
 - [47] Alex Pentland, "Smart Rooms," *Scientific American*, vol. 4, no. 274, April 1996.
 - [48] Ganesh Raaja. (2008, September) Basics and implementation of capacitive proximity sensing. [Online]. <http://www.planetanalog.com/features/showArticle.jhtml?articleID=210600561>
 - [49] Jun Rekimoto and Nobuyuki Matsushita, "Perceptual Surfaces: Towards a Human and Object Sensitive Interactive Display," in *Proceedings of the Workshop on Perceptual User Interfaces*, Banff, Canada, 1997.
 - [50] Joshua Reynolds Smith, *Electric Field Imaging*.: Massachusetts Institute of Technology, 1999.
 - [51] Joshua Smith et al., "Electric Field Sensing for Graphical Interfaces," *Computer Graphics and Applications, IEEE*, pp. 54-60, May/June 1998.
 - [52] Sony Computer Entertainment Europe. (2009, September) EyeToy. [Online]. <http://www.eyetoy.com>
 - [53] Horst Steg, Hartmut Strese, Claudia Loroff, Jérôme Hull, and Sophie Schmidt, "Europe Is Facing a Demographic Challenge - Ambient Assisted Living Offers Solutions," Ambient Assisted Living Joint Program, Berlin, Report 2006.
 - [54] Wilhelm Stork et al., "Personal Health Monitoring System mit innovativer mikrosystemtechnischer Sensorik," in *36. Jahrestagung der Deutschen Gesellschaft Für Biomedizinische Technik (DGBMT)*, Karlsruhe, 2002, pp. 903-906.
 - [55] Daniel Terdiman. (2007, November) Activision: 'Guitar Hero III' nets \$115 in first week's sales. Just kidding. [Online]. http://news.cnet.com/8301-13772_3-9813112-52.html
 - [56] John Underkoffler, Brygg Ullmer, and Hiroshi Ishii, "Emancipated pixels: real-world graphics in the luminous room," in *Proceedings of the 26th annual conference on Computer graphics and interactive techniques*, New York, 1999, pp. 385-392.
 - [57] USB Implementer's Forum, "Device Class Definition for Human Computer Interface Devices," USB Implementer's Forum, Firmware Specification 2001.
 - [58] Alessandro Valli, "The design of natural interaction," *Multimedia Tools and Applications*, vol. 38, no. 3, pp. 295-305, July 2008.
 - [59] John von Neumann, "First Draft of a Report on the EDVAC," University of Pennsylvania, Philadelphia, 1945.
 - [60] Raphael Wimmer, Matthias Kranz, Sebastian Boring, and Albrecht Schmidt, "A Capacitive Sensing Toolkit for Pervasive Activity Detection and Recognition," in *Pervasive Computing and Communications, 2007. PerCom '07. Fifth Annual IEEE International Conference on*, White Plains, 2007, pp. 171-180.
 - [61] Raphael Wimmer, Albrecht Schmidt, Paul Holleis, and Matthias Kranz, "Thracker - Using Capacitive Sensing for Gesture Recognition," in *Distributed Computing Systems Workshops, 2006. ICDCS Workshops 2006. 26th IEEE International Conference on*, 2006, pp. 64- 68.
 - [62] Christopher R. Wren et al., "Perceptive Spaces for Performance and Entertainment: Untethered Interaction using Computer Vision and Audition," *Applied Artificial Intelligence*, vol. 11, no. 4, pp. 267-284, June 1997.
 - [63] Thomas G. Zimmerman, Joseph A. Paradiso, Joshua R. Smith, David Allport, and Neil Gershenfeld, *Applying Electric Field Sensing To Human-Computer Interfaces.*, 1995.